第9章: スタイリング & UI ポリッシュ

> 機能が完成したアプリの**見た目と使い心地**を本格的に仕上げる章です。「動くだけ」のアプリを「使いたくなる」アプリに変えていきます。
>
> ここまでの章で書籍管理アプリの「動き」（データの保存・取得・編集・削除）は完成しました。本章では、その動きを包む「見た目」と「触り心地」（UI/UX、User Interface / User Experience：ユーザーが画面を見て触る部分の体験）に焦点を当てます。

この章で学ぶこと

テーマ	内容	なぜ重要か
Tailwind CSS 実践	CSSカスタムプロパティ（CSS Custom Properties / CSS変数：`--変数名` で色やサイズを一元管理する仕組み）の設定	デザインの一貫性を保つため
レスポンシブデザイン	レスポンシブ（responsive：反応する）デザイン。画面サイズに応じてレイアウトを変える仕組み。スマホ/タブレット/PC全てに対応	ユーザーは様々なデバイスでアクセスするため
UIコンポーネント改善	トースト通知（toast：パンが焼き上がる時のように下からひょいと出るメッセージ。操作結果を一時的に表示）、ページネーション（pagination：大量データを複数ページに分割表示する仕組み）	ユーザー体験の向上のため
アニメーション	ホバー（hover：マウスを要素の上に乗せた時）エフェクト、トランジション（CSS Transition：プロパティの変化を時間をかけて滑らかにつなぐ機能）	操作のフィードバックを伝えるため
ダークモード	dark mode：背景が黒・濃灰色など暗い配色に切り替わるモード。夜間や暗い場所で目の負担を軽減	ユーザーの好みに対応するため
アクセシビリティ	a11y（accessibility：「a」と「y」の間に11文字あるので a11y と略す。障害のある方を含む全ての人がWebを利用できるようにすること）	すべての人に使いやすいアプリにするため

書籍管理アプリの見た目を本格的に仕上げていく章です。Tailwind CSS を活用したレスポンシブデザイン、アニメーション、ダークモード、アクセシビリティ（accessibility：Webコンテンツをすべての人が利用できるようにするための取り組み）まで網羅的に学びます。

> **デザインが重要な理由：** 同じ機能のアプリでも、見た目が整っているだけで「使いやすそう」「信頼できそう」という印象を与えます。ポートフォリオ（portfolio：作品集）に載せる際も、デザインの良さは大きなアピールポイントになります。
>
> **本章で出てくる主要な技術用語の早見表：**
>
> - **ユーティリティファースト（utility-first）**：「`bg-red-500`（背景赤）」のような細かいパーツ単位のクラスを組み合わせてスタイルを作る考え方。Tailwind CSS が採用している。
> - **BEM（ベム、Block Element Modifier）**：「.card__title--active」のように、ブロック名と要素名と修飾子をハイフンとアンダースコアで区切るCSSクラス命名規則。
> - **スコープドスタイル（scoped style）**：スタイルを特定のコンポーネントの中だけに閉じ込めて、他のコンポーネントに影響しないようにする仕組み。
> - **CSS-in-JS**: JavaScript の中に直接 CSS を書く方式（styled-components や Emotion などのライブラリ）。
> - **shadcn/ui（シャドシーエヌ・ユーアイ）**：コピー＆ペーストで自分のプロジェクトに取り込む形のUIコンポーネント集。Tailwind CSS と Radix UI ベース。
> - **CSS Modules**:「`Button.module.css`」のような特別な命名のCSSファイルを使うと、クラス名が自動で一意になり、コンポーネントごとにスコープが分離される仕組み。

この章のゴールと地図

この章は「見た目」の章です。文章だけだとイメージしにくいので、**クラスを付ける前と後の画面**をできるだけ絵で並べながら進めます。まず、この章を読み終えたときにできるようになっていてほしいことを確認しましょう。

この章のゴール（読み終えたらチェック）：

- [ ] `bg-blue-500 px-4 py-2` のような Tailwind のクラス名を見て、「どんな見た目になるか」を言える
- [ ] `padding`（内側の余白）・`border`（枠線）・`margin`（外側の余白）の違いを図で説明できる
- [ ] `sm:` `md:` `lg:` を使って、画面の幅ごとに並び方を変えられる（「`sm:` はスマホ用」という勘違いをしない）
- [ ] `hover:` `focus:` `dark:` など「状態によって変わる見た目」を書ける
- [ ] Tailwind CSS v3 と v4（2026年時点の最新版）の設定方法の違いが分かり、どちらのプロジェクトでも迷わない
- [ ] 文字と背景の色の組み合わせが「読みやすいか」を数値（コントラスト比）で確かめられる

この章の地図: 0〜2節で「見た目の土台」を作り、3〜4節で「部品と動き」を足し、5〜6節で「使う人への配慮」を仕上げます。



図9-1: この章の地図（0節から順に進み、最後に練習問題で確認する）

この章を終えると、書籍一覧の画面は次のように変わります。左は「クラスを1つも付けていない」素の HTML、右はこの章で仕上げた後の画面イメージです。



図9-2: 画面イメージ — 同じデータでも、クラスを付ける前（左）と後（右）でここまで印象が変わる

> **ヒント:** この章のクラスは、ブラウザだけで試せる公式サイト **Tailwind Play**（ `https://play.tailwindcss.com` ）に貼り付けると、インストールなしで見た目をすぐ確認できます。「このクラスを足すと何が変わる？」と迷ったら、1つずつ足したり消したりして目で確かめるのが一番の近道です。

目次

0. 前提知識: CSSとTailwindの超基礎
1. Tailwind CSS 実践
2. レスポンシブデザイン
3. UI コンポーネントの改善
4. アニメーション
5. ダークモード（発展）
6. アクセシビリティ
7. 最終的な画面の説明
8. コンポーネント構成図（最終版）
9. 発展: アプリでは使っていない重要なスタイリング機能
10. 練習問題（解答・解説つき）

---

0. 前提知識: CSSとTailwindの超基礎

0.1 CSSって何？

CSS（Cascading Style Sheets、カスケーディング・スタイル・シーツ） はHTMLの「見た目」を指定する言語です。「カスケード」とは「滝のように上から下へ流れる」という意味で、後から書いたスタイルが前のものを上書きする性質に由来します。色・フォント・余白・配置などを記述します。

コピー

```
<!-- HTMLファイル側 -->
<!-- button タグはクリックできるボタンを作るHTML要素 -->
<!-- class="btn-primary" はこのボタンに「btn-primary」という名札を付ける指定 -->
<!-- ↑ CSS側で .btn-primary { ... } と書くと、ここに見た目が当たる -->
<button class="btn-primary">送信</button>
```

コピー

```
/* CSSファイル側 */
/* ピリオド「.」で始まる名前は「クラスセレクタ」と呼ばれ、 */
/* HTML の class="btn-primary" を持つ要素にスタイルを当てる */
.btn-primary {
  /* 背景色を指定。#3b82f6 は16進数で青色を表す */
  background-color: #3b82f6;
  /* 文字色を白に。color プロパティは「文字色」を指す */
  color: white;
  /* 内側の余白。上下8px、左右16px。padding は要素内部の余白 */
  padding: 8px 16px;
  /* 角を丸める。8px の半径で四隅を丸くする */
  border-radius: 8px;
  /* 枠線を消す。none = 何も無い */
  border: none;
}
```

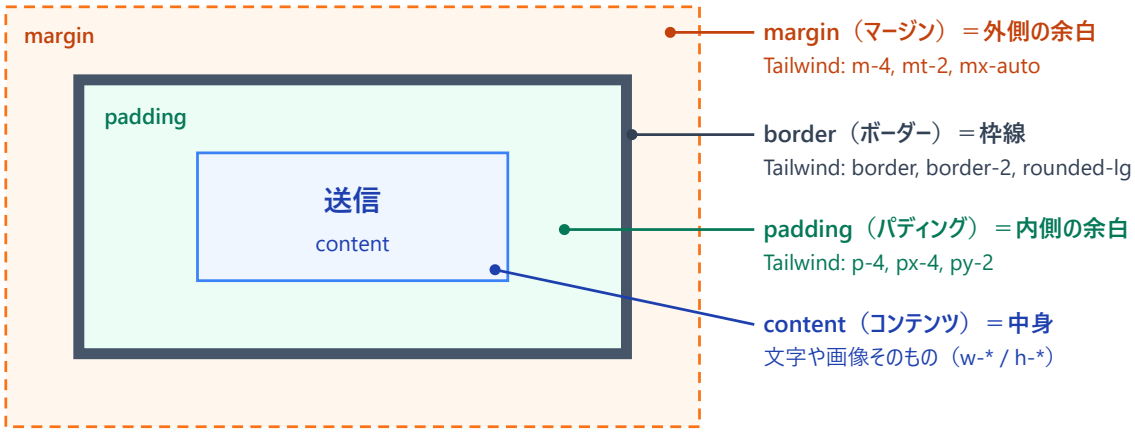
> **ボックスモデル（box model）の補足：** HTML の全ての要素は、上から順に「コンテンツ（中身） → padding（内側の余白） → border（枠線） → margin（外側の余白）」という4層構造で囲まれた箱として描画されます。これを「ボックスモデル」と呼びます。`padding` は箱の中の余白、`margin` は箱と外の他の要素との間の余白、`border` はその境界線です。この違いを理解しておくと、後で出てくる Tailwind の `p-4` / `m-4` / `border` の使い分けがスムーズになります。

▼ 表示結果:

コピー

```
┌──────┐
│ 送信 │ ← 青背景、白文字、角丸のボタン
└──────┘
```

上の補足で出てきた「ボックスモデル」を図にすると、次のような**入れ子の箱**になります。額縁に入れた絵をイメージすると覚えやすいです。



たとえ: 額縁の絵 ― 絵＝content／台紙＝padding／額縁＝border／壁との間隔＝margin

図9-3: ボックスモデル（内側から content → padding → border → margin の順に包まれる）

ポイントは、**padding は箱の「内側」なので背景色が塗られる**のに対し、**margin は箱の「外側」なので透明**だということです。この違いが次のつまずきにつながります。

> **つまずきポイント: 「ボタンを大きくしたい」のに margin を付けてしまう**
>
> ボタンの青い部分そのものを大きくしたいのに `m-4`（外側の余白）を付けると、青い部分は小さいまま、周りに**透明なすき間**ができるだけです。「中を広げたい → padding」「周りとの間を空けたい → margin」と覚えましょう。

コピー

```
<!-- NG: ボタンを大きくしたくて m-4（外側の余白）を付けた -->
<!-- → 青い部分は小さいまま。周りに見えないすき間ができるだけ -->
<button class="m-4 bg-blue-500 text-white">送信</button>
```

コピー

```
<!-- OK: ボタンの「内側」を広げたいので px-4 py-2（内側の余白）を付ける -->
<!-- → 青い部分そのものが大きくなり、押しやすくなる -->
<button class="px-4 py-2 bg-blue-500 text-white">送信</button>
```

▼ **表示の違い**（点線は「margin が取っている、目に見えない場所」を分かりやすく描いたもの）

NG: m-4 bg-blue-500 text-white

送信

青い部分は文字ぴったりの小ささ。点線の内側は透明なすき間

OK: px-4 py-2 bg-blue-500 text-white

送信

青い部分そのものが広がり、ボタンらしく押しやすい

0.2 主要なCSSプロパティ

プロパティ	何を変える？	例
`color`	文字色	`color: red;`
`background-color`	背景色	`background-color: #fff;`
`font-size`	文字サイズ	`font-size: 16px;`
`font-weight`	文字の太さ	`font-weight: bold;`
`padding`	内側の余白	`padding: 8px 16px;`
`margin`	外側の余白	`margin: 16px;`
`border`	枠線	`border: 1px solid #ccc;`
`border-radius`	角丸	`border-radius: 8px;`
`display`	表示方式	`display: flex;`
`width` / `height`	幅・高さ	`width: 100%;`

0.3 Tailwind CSS とは

通常のCSSは「CSSファイルを別に作って、クラスを定義して、HTMLに `class` を書く」という流れですが、これを毎回やると**クラス命名で消耗**します。「このボタン用のクラス名は何にしよう…」と毎回悩むのは意外と大変なのです。

Tailwind CSS（テイルウィンド・シーエスエス） は「あらかじめ大量のクラスが用意されていて、`bg-blue-500`（背景を青の500段階の色に）のようなクラス名をHTMLに直接書くだけでスタイルが当たる」という仕組みです。この考え方を**ユーティリティファースト（utility-first：小さな機能単位のクラスを組み合わせて作る方式）** と呼びます。

ちなみに従来のCSS設計では **BEM**（Block Element Modifier、`.card__title--active` のようにブロック・要素・修飾子で命名する規則）や **CSS Modules**（`Button.module.css` のようなファイルで自動的にクラス名をユニーク化する仕組み）、**CSS-in-JS**（styled-components 等、JS の中にCSSを書く方式）といった様々なアプローチがありました。Tailwind はそれらと並ぶ「もう一つの選択肢」です。

▼ 同じボタンを Tailwind で書くと:

コピー

```
<!-- bg-blue-500: 背景色を青の500段階目に -->
<!-- text-white  : 文字色を白に -->
<!-- px-4        : padding（内側余白）の横方向を 1rem（16px） -->
<!-- py-2        : padding の縦方向を 0.5rem（8px） -->
<!-- rounded-lg  : 角を「大きめ」に丸める（border-radius: 0.5rem） -->
<button class="bg-blue-500 text-white px-4 py-2 rounded-lg">
  送信
</button>
```

CSSファイルを書く必要がありません。HTMLにクラスを書くだけで完結します。

では、上のボタンのクラスを**1つずつ足していくと、見た目がどう変わるか**を並べてみましょう。1つのクラスが1つの変化を担当していることがよく分かります。

足したクラス	見た目（ブラウザでの表示）
① クラスなし	送信 ← ただの文字に見える
② + bg-blue-500	送信 ← 背景が青に
③ + text-white	送信 ← 文字が白に
④ + px-4 py-2	送信 ← 内側に余白
⑤ + rounded-lg	送信 ← 角が丸く
⑥ + hover:bg-blue-700	送信 → 乗せると 送信

図9-4: Before/After — クラスを1つ足すごとに、見た目が1か所ずつ変わる

> **注意:** ①で「ボタンなのに、ただの文字に見える」のは間違いではありません。Tailwind は読み込まれた瞬間に、ブラウザが勝手に付ける枠線や灰色の背景をいったん全部リセットします（この仕組みを **Preflight（プリフライト：飛行前の点検、の意味）** と呼びます）。「真っ白な状態から、必要な見た目だけをクラスで足していく」のが Tailwind の考え方です。

Tailwind はどうやって CSS を作っているのか

「クラス名を書くだけで見た目が変わる」のは魔法ではありません。Tailwind は**あなたが書いたファイルを文字として読み、見つけたクラス名の分だけ CSS を自動で書き出している**のです。料理にたとえると、「注文票（クラス名）を読んだ料理

人（Tailwind）が、注文された料理（CSS）だけを作る」イメージです。

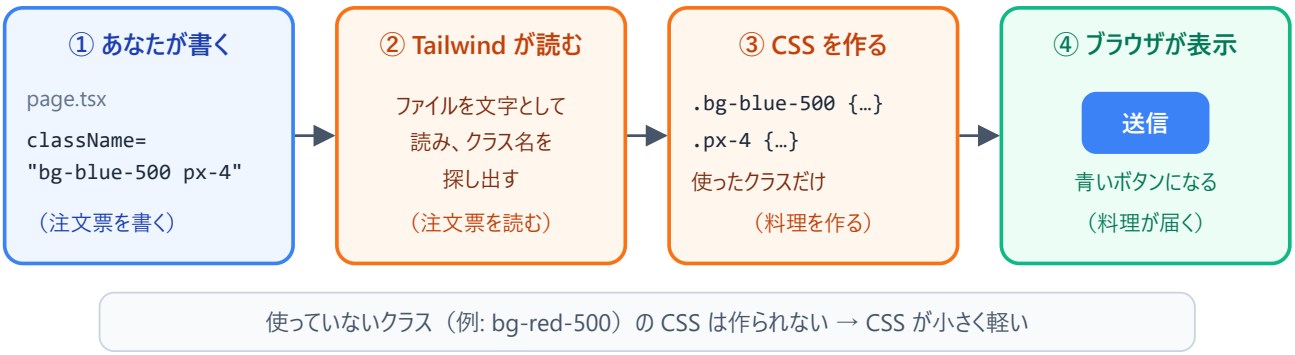


図9-5: Tailwind がクラス名から CSS を作る流れ（Next.js では npm run dev 中に自動で行われる）

この流れを、実際に自分の手で確かめてみましょう。

試してみよう: クラスが CSS に変わる瞬間を見る

> **▼ このコードがやること（先に日本語で）**: Tailwind を Next.js の外で小さく動かして、`bg-blue-500` などのクラスが本当に CSS に変換される様子を確かめます。HTML に書いたクラスの分だけ CSS ができる（図9-5の②③）ことを、出来上がったファイルを読んで体験するのが目的です。アプリ本体とは別の練習用フォルダで試すので、失敗しても安心です。

まず、練習用のフォルダを作って Tailwind をインストールします。

コピー

```
# 練習用のフォルダを作って移動する（アプリのフォルダとは別の場所で OK）
mkdir tw-practice
cd tw-practice
# package.json（道具の一覧表）を作る
npm init -y
# Tailwind 本体と、コマンドで動かすための CLI をインストールする
npm install tailwindcss @tailwindcss/cli
```

次に、2つのファイルを作ります。

コピー

```
/* input.css: Tailwind を読み込むだけの1行 */
@import "tailwindcss";
```

コピー

```
<!doctype html>
<html lang="ja">
  <body>
    <!-- この1行のクラスが、どんなCSSに変わるかを確かめる -->
    <button class="bg-blue-500 text-white px-4 py-2 rounded-lg hover:bg-blue-700">
      送信
    </button>
  </body>
</html>
```

最後に、Tailwind に CSS を作らせます。

コピー

```
# input.css を元に、フォルダ内のファイルからクラスを探して output.css を作る
npx @tailwindcss/cli -i input.css -o output.css
```

▼ **実行結果**（ターミナルの表示）

コピー

```
≈ tailwindcss v4.3.3

Done in 35ms
```

▼ **実行結果**（できた `output.css` の抜粋。全体は約190行で、ここでは「色などの値」と「クラス本体」の部分だけを抜き出しています）

コピー

```
@layer theme {
  :root, :host {
    --color-blue-500: oklch(62.3% 0.214 259.815);
    --color-blue-700: oklch(48.8% 0.243 264.376);
    --color-white: #fff;
    --spacing: 0.25rem;
    --radius-lg: 0.5rem;
  }
}
@layer utilities {
  .rounded-lg {
    border-radius: var(--radius-lg);
  }
  .bg-blue-500 {
    background-color: var(--color-blue-500);
  }
  .px-4 {
    padding-inline: calc(var(--spacing) * 4);
  }
  .py-2 {
    padding-block: calc(var(--spacing) * 2);
  }
  .text-white {
    color: var(--color-white);
  }
  @media (hover: hover) {
    .hover\:bg-blue-700:hover {
      background-color: var(--color-blue-700);
    }
  }
}
```

▼ コードを1つずつ分解して解説

解説1: `@import "tailwindcss";` で Tailwind を読み込む

コピー

```
@import "tailwindcss";
```

- `@import`（インポート）は「別の CSS をここに取り込む」という、CSS に元からある命令です。`"tailwindcss"` を取り込むと、Tailwind の仕組み一式が使えるようになります。
- これは **Tailwind CSS v4** の書き方で、1.1 節のアプリの `globals.css` も同じ1行から始まります。古い記事で見かける `@tailwind base;` などの3行は **v3** の書き方で、v4 ではこの1行に置き換わりました（詳しくは 1.1.1 節）。

解説2: `npx @tailwindcss/cli -i input.css -o output.css` の意味

コピー

```
npx @tailwindcss/cli -i input.css -o output.css
```

- `npx` は「インストールした道具をその場で実行する」コマンドです。`@tailwindcss/cli` が Tailwind をコマンドで動かす道具です。
- `-i input.css` は「入力（input）はこのファイル」、`-o output.css` は「出力（output）はこのファイルに書いて」という指定です。
- Tailwind はフォルダの中のファイル（今回は `index.html`）を文字として読み、クラス名を探してから `output.css` を書き出します。図9-5の②③がまさにこの瞬間です。

> **用語: CLI（シーエルアイ、Command Line Interface）** … ターミナルに文字のコマンドを打って動かす道具のこと。Next.js のアプリでは CLI の代わりに `@tailwindcss/postcss` という部品が `npm run dev` の裏で同じ仕事をしてくれるので、普段は自分でこのコマンドを打つ必要はありません。

解説3: 出力された CSS を読んでみる

コピー

```
.px-4 {
  padding-inline: calc(var(--spacing) * 4);
}
```

- **クラス1つにつき、CSS のルールが1つ**作られています。`index.html` に書いた6つのクラスの分だけが並び、書いていない `bg-red-500` などは1行も出てきません。
- `var(--spacing)` は `0.25rem`（4px）なので、`calc(var(--spacing) * 4)` は「4px × 4 = 16px」です。`padding-inline` は「左右の padding」という意味なので、`px-4` は「左右に16pxの内側余白」になります。
- `hover:bg-blue-700` は `@media (hover: hover)` の中に入っています。これは「**マウスのようにホバーできる端末のときだけ**効かせる」という意味で、スマホで指が触れたときに色が戻らなくなる問題を防いでいます（v4 からの動き）。
- 色の値 `oklch(...)` は、人の目の感じ方に近い新しい色の書き方です。中身を覚える必要はなく、「`--color-blue-500` という名前の青」と読めれば十分です。

> **用語: oklch（オーケーエルシーエイチ）** … 色を「明るさ・鮮やかさ・色合い」の3つの数字で表す、CSS の新しい色の書き方。Tailwind v4 の標準の色はこの形式で定義されている。

0.4 Tailwindクラス命名のパターン

ほぼ全クラスは「**プロパティの省略** + `-` + **値**」の形をしています。

クラス	意味	通常のCSS換算
`bg-red-500`	背景色を赤500	`background-color: #ef4444;`
`text-white`	文字色を白	`color: white;`
`text-xl`	文字サイズXL	`font-size: 1.25rem;`
`p-4`	全方向 padding 16px	`padding: 1rem;`
`px-4`	横方向 padding	`padding-left: 1rem; padding-right: 1rem;`
`py-2`	縦方向 padding	`padding-top: 0.5rem; padding-bottom: 0.5rem;`
`m-4`	全方向 margin	`margin: 1rem;`
`rounded-lg`	角丸 大	`border-radius: 0.5rem;`
`flex`	flexコンテナに	`display: flex;`
`items-center`	縦方向中央揃え	`align-items: center;`
`gap-2`	子要素間の隙間	`gap: 0.5rem;`
`w-full`	幅100%	`width: 100%;`
`hover:bg-blue-700`	マウスホバー時の色	`:hover { background: ...; }`

> **つまずきポイント: クラス名を `${}` で組み立てると、色が付かない**
>
> 「色の名前を変数にして `bg-${color}-100` と書けば短く済む」と考えがちですが、これは**動きません**。図9-5の②のとおり、Tailwind はファイルを**文字として読むだけ**で、プログラムを実行しません。ファイルの中に `bg-blue-100` という**完成した文字**が無い限り、その CSS は作られないのです。

コピー

```
// NG: クラス名を文字列の組み立てで作っている
// → ファイルの中に "bg-blue-100" という文字が無いので、CSS が作られない
<span className={`bg-${color}-100 text-${color}-700`}>{label}</span>
```

コピー

```
// OK: 完成したクラス名を「辞書」に丸ごと書いておき、そこから選ぶ
const colorClasses = {
  blue: "bg-blue-100 text-blue-700",
  green: "bg-green-100 text-green-700",
};
<span className={colorClasses[color]}>{label}</span>;
```

本当に CSS が作られないのか、先ほどの練習用フォルダで確かめてみましょう。

試してみよう: 組み立てたクラス名が無視されることを確かめる

> **▼ このコードがやること（先に日本語で）**：読書状況バッジの色を「文字列をつなげて」作る NG 版と、「完成したクラス名を辞書に書いておく」OK 版を用意し、Tailwind がそれぞれから CSS を作れるかを比べます。`tw-practice` の中に `ng` と `ok` の2つのフォルダを作り、どちらにも 0.3 節と同じ1行だけの `input.css` を置いて試します。

コピー

```
// ng/StatusBadge.tsx（NG 版）
type Props = { color: "blue" | "green" | "yellow"; label: string };

export function StatusBadge({ color, label }: Props) {
  // NG: クラス名を文字列の組み立てで作っている
  return <span className={`bg-${color}-100 text-${color}-700`}>{label}</span>;
}
```

コピー

```
// ok/StatusBadge.tsx（OK 版）
type Props = { color: "blue" | "green" | "yellow"; label: string };

// OK: 完成したクラス名を「辞書」に丸ごと書いておく
const colorClasses = {
  blue: "bg-blue-100 text-blue-700",
  green: "bg-green-100 text-green-700",
  yellow: "bg-yellow-100 text-yellow-700",
};

export function StatusBadge({ color, label }: Props) {
  return <span className={colorClasses[color]}>{label}</span>;
}
```

コピー

```
# NG 版のフォルダで CSS を作る
cd ng
npx @tailwindcss/cli -i input.css -o output.css
# OK 版のフォルダで CSS を作る
cd ../ok
npx @tailwindcss/cli -i input.css -o output.css
```

▼ 実行結果（それぞれの `output.css` を VS Code で開き、`@layer utilities` の部分を見たもの）

コピー

```
---- ng/output.css（中身が空っぽ！）----
@layer utilities;

---- ok/output.css ----
@layer utilities {
  .bg-blue-100 {
    background-color: var(--color-blue-100);
  }
  .bg-green-100 {
    background-color: var(--color-green-100);
  }
  .bg-yellow-100 {
    background-color: var(--color-yellow-100);
  }
  .text-blue-700 {
    color: var(--color-blue-700);
  }
  .text-green-700 {
    color: var(--color-green-700);
  }
  .text-yellow-700 {
    color: var(--color-yellow-700);
  }
}
```

▼ コードを1つずつ分解して解説

解説1: NG 版はなぜ空っぽなのか

コピー

```
<span className={`bg-${color}-100 text-${color}-700`}>{label}</span>
```

- Tailwind から見ると、このファイルに書かれているのは `bg-${color}-100` という**ただの文字の並び**です。`color` に何が入るかは、アプリを実行するまで分かりません。
- そのため Tailwind は「`bg-blue-100` が使われている」と気付けず、CSS を1つも作りません。ブラウザでは `class="bg-blue-100"` が付いているのに**色が付かない**、という不思議な状態になります。

解説2: OK 版は「完成した文字」を書いておく

コピー

```
const colorClasses = {
  blue: "bg-blue-100 text-blue-700",
  green: "bg-green-100 text-green-700",
  yellow: "bg-yellow-100 text-yellow-700",
};
```

- 辞書（オブジェクト）の中に `"bg-blue-100"` などの**完成したクラス名**がそのまま書いてあるので、Tailwind は文字として見つけられます。
- 実行時は `colorClasses[color]` で、`color` に合った文字列を取り出すだけです。本章 4.4 節の `AnimatedButton` の `variantStyles` もこの「辞書パターン」です。2.2 節の `statusBadge` は `switch` 文ですが、各 `case` に**完成したクラス名**を書いている点は同じなので、正しく動きます。

> **用語: 辞書パターン（マッピング）** ... 「キー → 値」の対応表を先に作っておき、キーで値を選ぶ書き方。Tailwind ではクラス名を必ず完成した形で書くための定番のやり方。

もう1つ、クラス名の書き方でよくあるつまずきがあります。

> **つまずきポイント: 同じ種類のクラスを2つ書いたら、後ろが勝つ...とは限らない**
>
> `className="p-2 p-4"` と `className="p-4 p-2"` は、どちらも結果は同じ **16px（p-4）** になります。CSS では「**CSS ファイルの中で後に書かれたルール**」が勝つので、`className` の中の順番は関係ありません。Tailwind は数字の小さい順に CSS を並べるため、`.p-2` の後に `.p-4` が来て、`p-4` が常に勝ちます。

コピー

```
<!-- NG: 「後ろに書いた p-2 で上書きできる」と思っている -->
<!-- → 実際は p-4（16px）が勝ち、思った通りにならない -->
<div class="p-4 p-2">本棚</div>
```

コピー

```
<!-- OK: 同じ種類（padding）のクラスは1つだけ書く -->
<div class="p-2">本棚</div>
```

実際に Tailwind が作った CSS（`p-2` と `p-4` を両方使ったときの抜粋）は次のとおりで、`.p-4` が後ろにあることが分かります。

コピー

```
@layer utilities {
  .p-2 {
    padding: calc(var(--spacing) * 2);
  }
  .p-4 {
    padding: calc(var(--spacing) * 4);
  }
}
```

状況によってクラスを切り替えたいときは、「両方書いて後ろで上書き」ではなく、**条件に合うほうだけを付ける**のが正しいやり方です。そのための小さな道具を作ってみましょう。

条件でクラスを付け外しする `cn` 関数

> **▼ このコードがやること（先に日本語で）**：「今のページなら濃い色、それ以外は薄い色、押せないときは半透明」のように、状態によって付けるクラスを選んで1つの文字列にまとめる関数を作ります。条件が成り立たない部分は捨てられるので、同じ種類のクラスが2つ同時に付くことがありません。クラス名は完成した形で書いているので、Tailwind もちゃんと見つけられます。

コピー

```
// cn.ts
// 条件つきでクラス名をつなげる小さな関数
// false / null / undefined / "" はまとめて捨てる
function cn(...classes: (string | false | null | undefined)[]): string {
  return classes.filter(Boolean).join(" ");
}

// ページ番号ボタンのクラスを作る関数
function pageButtonClass(isCurrent: boolean, isDisabled: boolean): string {
  return cn(
    // どの状態でも共通のクラス
    "rounded-lg px-3 py-1 text-sm",
    // 現在のページだけ濃い色
    isCurrent && "bg-indigo-600 text-white",
    // それ以外は薄い色
    !isCurrent && "bg-gray-100 text-gray-700",
    // 押せないときは半透明
    isDisabled && "opacity-50",
  );
}

console.log(pageButtonClass(true, false));
console.log(pageButtonClass(false, false));
console.log(pageButtonClass(false, true));
```

▼ **実行結果**（`node cn.ts` で実行）

コピー

```
rounded-lg px-3 py-1 text-sm bg-indigo-600 text-white
rounded-lg px-3 py-1 text-sm bg-gray-100 text-gray-700
rounded-lg px-3 py-1 text-sm bg-gray-100 text-gray-700 opacity-50
```

▼ コードを1つずつ分解して解説

解説1: `...classes` で好きな数の引数を受け取る

コピー

```
function cn(...classes: (string | false | null | undefined)[]): string {
```

- `...classes`（残余引数）は「渡された引数を**いくつでも配列にまとめて**受け取る」書き方です。`cn("a", "b", "c")` と呼ぶと、`classes` は `["a", "b", "c"]` になります。

- 型 `(string | false | null | undefined)[]` は「文字列か、false・null・undefined のどれかが並んだ配列」という意味です。`isCurrent && "..."` は条件が成り立たないと `false` になるので、`false` も受け取れるようにしています。

解説2: `filter(Boolean)` で不要なものを捨てて `join(" ")` でつなぐ

コピー

```
return classes.filter(Boolean).join(" ");
```

- `filter(Boolean)` は「**true 扱いになるものだけを残す**」定番の書き方です。`false`・`null`・`undefined`・`""`（空文字）が捨てられます。
- `join(" ")` は配列をスペース区切りの1つの文字列にします。これで `className` にそのまま渡せる形になります。

解説3: `条件 && "クラス"` で付けるかどうかを決める

コピー

```
isCurrent && "bg-indigo-600 text-white",
!isCurrent && "bg-gray-100 text-gray-700",
```

- `A && B` は「A が true なら B、false なら false」になる式です。`isCurrent` が true のときだけ濃い色のクラスが残ります。
- `!isCurrent`（`!` は「〜でない」）と組み合わせると、**どちらか片方だけ**が必ず付きます。実行結果の1行目と2行目で、背景色のクラスが1つずつしか入っていないことを確かめてください。

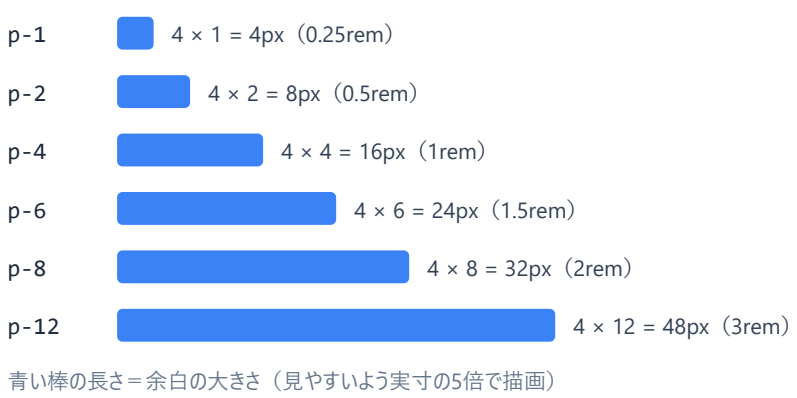
> **ヒント:** 実際の開発では、同じ働きをする `clsx` というライブラリや、さらに「後から書いたクラスで上書きされたように整理してくれる」`tailwind-merge` を組み合わせた `cn` 関数がよく使われます（shadcn/ui も採用）。仕組みは上の数行と同じなので、まずは自作版で考え方をつかみましょう。

0.5 数値スケール

Tailwind の数値（`p-4`, `text-xl`）は**4の倍数のpx**などの規則があります。

Tailwind	値	px換算
`p-1`	0.25rem	4px
`p-2`	0.5rem	8px
`p-4`	1rem	16px
`p-8`	2rem	32px
`text-xs`	0.75rem	12px
`text-sm`	0.875rem	14px
`text-base`	1rem	16px
`text-lg`	1.125rem	18px
`text-xl`	1.25rem	20px
`text-2xl`	1.5rem	24px

余白の数字を「長さ」として並べると、次のようになります。数字が2倍になると余白も2倍になる、という素直な関係です。



読み方のコツ

「数字 × 4px」が大きさ

```
p-4 → 16px
m-2 → 8px
gap-6 → 24px
```

v4 は半端な数も使える

`p-13` → 52px
（v3 では p-13 は無い）

図9-6: 余白の数値スケール（p- だけでなく m-・gap-・w- なども同じ物差しを使う）

> **ヒント**: Tailwind v4 では、余白の大きさが `calc(var(--spacing) * 数字)`（`--spacing` は 0.25rem）という**計算式**で作られます（0.3節の実行結果を参照）。そのため `p-13`（52px）や `w-72`（288px）のように、表に無い数字も自由に使えます。v3 では決められた数字しか使えなかったので、古い記事を読むときは注意しましょう。

0.6 色のスケール

色は `red-50` 〜 `red-950` の11段階で、数字が大きいほど濃くなります。

コピー

```
50  100 200 300 400 500 600 700 800 900 950
↑ 一番薄い              ↑ 標準              ↑ 一番濃い
```

> **コツ:**「背景は薄い色（100, 200）、文字は濃い色（700, 800, 900）」という組み合わせがおすすめ。`bg-blue-100 text-blue-800` で柔らかい青のバッジになります。

実際の色で見てみましょう。下は本アプリのメインカラー（primary）に使うインディゴの11段階です（1.1節の `globals.css` で定義する値と同じ色）。

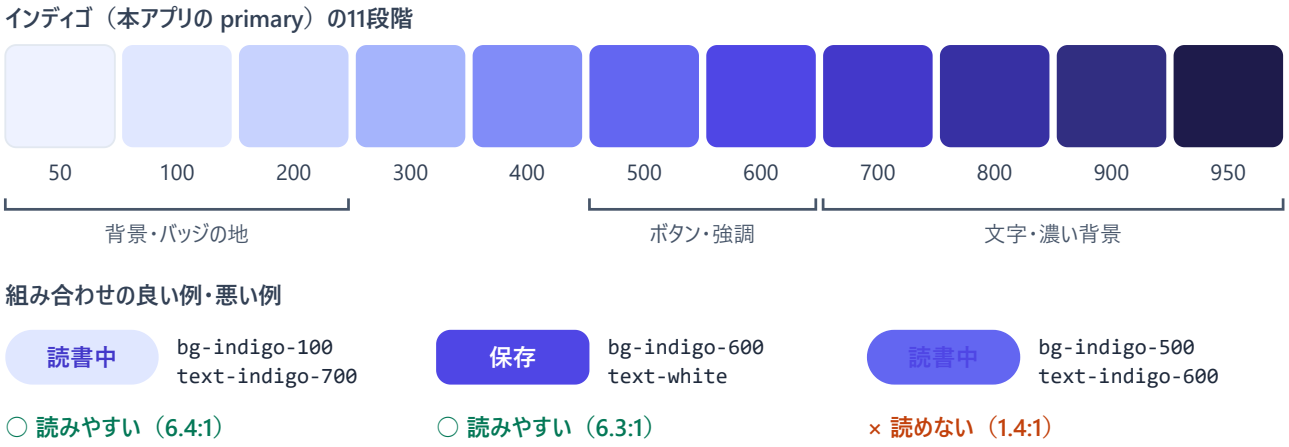


図9-7: 色の濃淡スケールと、背景・文字の組み合わせ方（かっこ内はコントラスト比。6.3節で解説）

右端の悪い例のように、**近い濃さどうし（500 と 600 など）を組み合わせると、文字が背景に溶けて読めなくなります。**「背景と文字は、数字を 400 以上離す」を目安にすると失敗しにくいです。

> **注意:** Tailwind v4 の標準の色は `oklch(...)` という書き方で定義されていて、v3 の16進数（`#3b82f6` など）とはごくわずかに色味が違います。この章に出てくる16進数は v3 の値ですが、見た目の差はほとんど分からない程度なので、気にせず読み進めて大丈夫です。

0.7 レスポンシブの書き方

「PCでは横並び、スマホでは縦並び」のように画面サイズで変えるには **接頭辞（prefix、プレフィックス：先頭につける目印）** を付けます。

コピー

```
<!-- ↓ Tailwind の典型的なレスポンシブ指定 -->
<!-- flex          : 中の子要素を flex レイアウトで並べる（これが無いと下の2つは効かない） -->
<!-- flex-col      : flex レイアウトで「縦並び」（column）にする -->
<!-- md:flex-row   : 「md」接頭辞は 768px 以上で適用。横並び（row）に切り替わる -->
<!-- ※ flex-col と md:flex-row の両方を書くことで、 -->
<!--    「小さい画面では縦、大きくなったら横」と動的に切り替わる -->
<div class="flex flex-col md:flex-row">
  <!-- スマホ: 縦並び（flex-col） -->
  <!-- 768px 以上（md）: 横並び（flex-row） -->
</div>
```

> **注意:** `flex-col` や `flex-row` は「flex レイアウトの**並ぶ向き**」を決めるだけのクラスです。並べ方そのもの（`display: flex`）を決める `flex` を付け忘れると、`md:flex-row` を書いても横並びになりません。必ず `flex flex-col md:flex-row` のようにセットで書きます。

このクラスを付けた箱の中に A・B・C の3つを入れると、画面の幅によって次のように並び方が変わります。

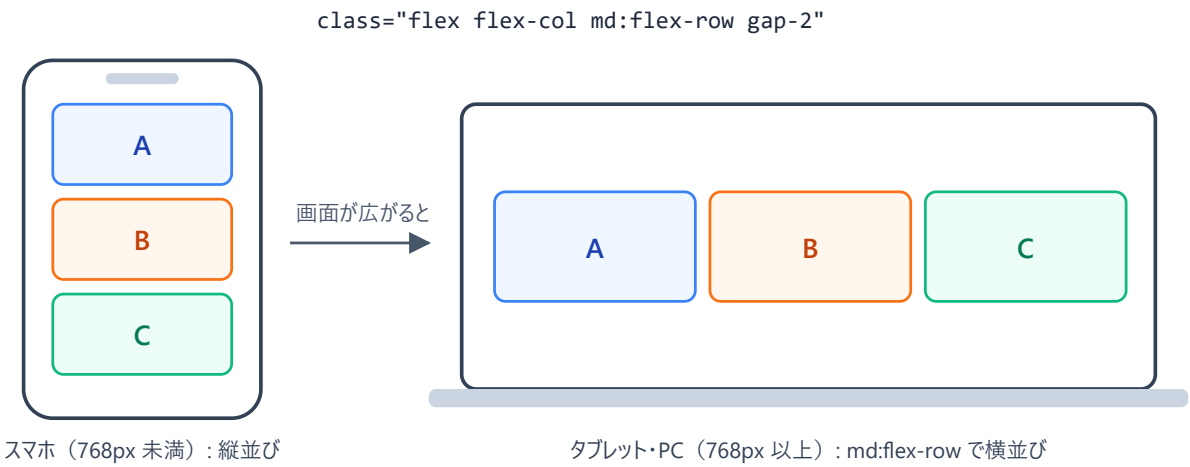


図9-8: 画面イメージ — 同じ HTML でも、画面の幅によって縦並び ⇔ 横並びが切り替わる

接頭辞	何以上で適用？	想定デバイス
`sm:`	640px 以上	大きめスマホ
`md:`	768px 以上	タブレット
`lg:`	1024px 以上	ノートPC
`xl:`	1280px 以上	デスクトップ

> **モバイルファーストの原則：** Tailwind では「接頭辞なし＝最小画面（モバイル）」「接頭辞あり＝それ以上の画面で上書き」というルールになっています。つまり最初にスマホ用のレイアウトを書き、画面が大きくなるにつれて `sm:` `md:` `lg:` で部分的に上書きしていきます。これを**モバイルファースト（mobile-first）** と呼びます。逆に「PCを基準に書いて、小さい画面用に縮める」やり方は古いスタイルです。
>
> **flex と grid の使い分けの目安：**
>
> - **flex（フレックスボックス）**：1次元（横一列、または縦一列）の並びに向く。ボタンを横に並べる、ヘッダーの中身を左右に配置する、など。
> - **grid（グリッド）**：2次元（行と列の格子）に向く。書籍カードを「4列×何行」の格子状に並べるなど。

---

1. Tailwind CSS 実践

1.1 globals.css のカスタマイズ

`app/globals.css` にアプリ全体で使うカスタムスタイルを定義します。**CSS カスタムプロパティ（CSS Custom Properties / CSS variables：`--name` のように定義し `var(--name)` で参照する変数機能）** を使い、ライトモードとダークモードの両方に対応できる設計にします。

> **注意（Tailwind のバージョンについて）**：この章のコードは、2026年10月時点で `create-next-app` が入れる **Tailwind CSS v4**（公式ドキュメントの最新は v4.3）の書き方です。ネットの記事や少し前に作ったプロジェクトでは **v3** の書き方（ `@tailwind base;` などの3行 ＋ `tailwind.config.ts` ）もよく見かけます。v3 との違いは **1.1.1 節**、v3 の設定ファイルは **1.3 節**（参考）にまとめました。どちらの版かは `package.json` の `"tailwindcss"` の数字（ `^4.…` か `^3.…` か）で確認できます。

このセクションで出てくる Tailwind 独自の命令（ディレクティブ）も先に整理しておきます。

- `@import "tailwindcss";` : Tailwind 本体を読み込む1行。これが無いと Tailwind のクラスは一切効かない。
- `@custom-variant dark (…);` : `dark:` を「いつ効かせるか」を決める命令。本アプリでは `<html class="dark">` のときに効かせる。
- `@theme { … }` : 色・影・角丸・アニメーションなどの値を登録し、`bg-primary-600` のような**クラスとして使えるようにする**ブロック。
- `@layer base { … }` : ベース層（最も優先度が低い）に、`body` や `h1` の初期スタイルを追加するブロック。
- `@layer components { … }` : 自作のコンポーネントクラス（ `.btn` など）を入れる層。
- `@utility 名前 { … }` : 自作のユーティリティクラス（ `glass` など）を作る命令。
- `@apply クラス名1 クラス名2` : CSS の中で Tailwind のクラスを展開する命令。長いクラス指定を1つにまとめるのに便利。

▼ **このコードがやること（先に日本語で）**：アプリ全体で使う「色・影・角丸・アニメーション」をまとめて定義する土台のCSSファイルを作ります。ポイントは、色などを `--color-primary-500` のような「CSS変数（名前を付けた値の入れ物）」にして `@theme` に登録することです。登録すると `bg-primary-500` のようなクラスとして使えるうえ、ダークモードのときは変数の中身を入れ替えるだけで画面全体の色が一気に切り替わります。v3 では `tailwind.config.ts` に分けて書いていた設定も、このファイル1枚にまとまっています。各行の細かい意味はコード内コメントで丁寧に説明しています。

コピー

```
/* app/globals.css
   ↑ このファイルはアプリ全体に適用される「グローバルCSS」。
     Next.js の App Router では app/layout.tsx で import "./globals.css" すると
     アプリのすべてのページに反映される。
     ※ Tailwind CSS v4（2026年の create-next-app が入れる版）の書き方。
       v3 の書き方との違いは 1.1.1 節と 1.3 節を参照。 */

/* @import "tailwindcss" は Tailwind CSS 本体を読み込む1行。
   ビルド時に、リセットCSS（base）と、bg-red-500・p-4 などのクラス（utilities）に展開される。
   この1行が無いと Tailwind のクラスは一切効かない。 */
@import "tailwindcss";

/* dark: の付いたクラスを「<html class="dark"> のとき」に効かせる宣言。
   これが無いと、v4 の dark: は OS の設定（ダーク/ライト）にしか反応しない。
   5.3節のテーマ切り替えボタンで、ユーザーが自分で選べるようにするために必要。 */
@custom-variant dark (&:where(.dark, .dark *));

/* ========================================
   1. テーマ（Tailwind のクラスになる値＝カラートークンなど）
   ======================================== */
/* @theme の中に書いた CSS 変数は、名前の前半（--color- など）に応じて Tailwind のクラスになる。
   例: --color-primary-600 → bg-primary-600 / text-primary-600 / border-primary-600 など
   同時に :root（≒ <html>）の CSS 変数としても出力されるので、
   ページのどこからでも var(--color-primary-600) で参照できる。
   このように共通の値を変数化したものを「デザイントークン（design tokens）」と呼ぶ。 */
@theme {
  /* プライマリカラー（インディゴ系：青と紫の中間のような色） */
  /* --color- で始まる名前は「CSSカスタムプロパティ（CSS変数）」。
     値（例：#eef2ff）は16進数カラーコード。先頭の # の後の2桁ずつが
     赤・緑・青の強さを 00〜FF で表す。 */
  /* 一番薄いインディゴ（背景バッジ等に使う） */
  --color-primary-50: #eef2ff;
  /* 薄め */
  --color-primary-100: #e0e7ff;
  --color-primary-200: #c7d2fe;
  --color-primary-300: #a5b4fc;
  --color-primary-400: #818cf8;
  /* 標準のプライマリ色 */
  --color-primary-500: #6366f1;
  /* ボタン背景でよく使う濃さ */
  --color-primary-600: #4f46e5;
  /* ホバー時の濃さ */
  --color-primary-700: #4338ca;
  --color-primary-800: #3730a3;
  --color-primary-900: #312e81;
  /* 一番濃いインディゴ（ダークモード背景等） */
  --color-primary-950: #1e1b4b;

  /* セカンダリカラー（エメラルド系：緑系の鮮やかな色） */
  --color-secondary-50: #ecfdf5;
  --color-secondary-100: #d1fae5;
  --color-secondary-200: #a7f3d0;
  --color-secondary-300: #6ee7b7;
  --color-secondary-400: #34d399;
  /* 標準のセカンダリ色 */
  --color-secondary-500: #10b981;
  --color-secondary-600: #059669;
  --color-secondary-700: #047857;
  --color-secondary-800: #065f46;
  --color-secondary-900: #064e3b;
  --color-secondary-950: #022c22;
```

```
/* 背景・テキスト（セマンティック＝意味ベースの命名） */
/* ここに書くのは「ライトモードの値」。ダークモードの値は下の .dark で上書きする */
/* ページ全体の背景色（白）→ bg-background */
--color-background: #ffffff;
/* 前景色＝メインの文字色（ほぼ黒）→ text-foreground */
--color-foreground: #0f172a;
/* muted = 弱めた色。補足テキスト用 → text-muted */
--color-muted: #64748b;
/* さらに薄い補足文字色 → text-muted-foreground */
--color-muted-foreground: #94a3b8;
/* 枠線の色 → border-border */
--color-border: #e2e8f0;
/* カード（書籍カードなど）の背景色 → bg-card */
--color-card: #ffffff;
/* カード内の文字色 → text-card-foreground */
--color-card-foreground: #0f172a;

/* 状態カラー（操作結果や状態を伝えるための色） */
/* 成功＝緑 */
--color-success: #10b981;
/* 警告＝黄/オレンジ */
--color-warning: #f59e0b;
/* エラー＝赤 */
--color-error: #ef4444;
/* 情報＝青 */
--color-info: #3b82f6;

/* シャドウ（box-shadow：影）→ shadow-sm / shadow-md / shadow-lg */
/* box-shadow の値は「横ずれ 縦ずれ ぼかし 広がり 色」の順に書く */
/* rgb(0 0 0 / 0.05) は黒で透明度5%という意味 */
--shadow-sm: 0 1px 2px 0 rgb(0 0 0 / 0.05);
/* 2つの影を重ねる */
--shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
--shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.1), 0 4px 6px -4px rgb(0 0 0 / 0.1);
/* 独自の影を新規追加 → shadow-card / shadow-card-hover */
--shadow-card: 0 2px 8px -2px rgb(0 0 0 / 0.08), 0 4px 12px -4px rgb(0 0 0 / 0.04);
--shadow-card-hover: 0 8px 24px -4px rgb(0 0 0 / 0.12), 0 4px 8px -4px rgb(0 0 0 / 0.08);

/* ボーダー半径（角丸の大きさ）→ rounded-sm / rounded-md / rounded-lg / rounded-xl */
/* rem は「ルート要素のフォントサイズを基準とする単位」。普通は 1rem = 16px */
/* = 6px */
--radius-sm: 0.375rem;
/* = 8px */
--radius-md: 0.5rem;
/* = 12px */
--radius-lg: 0.75rem;
/* = 16px */
--radius-xl: 1rem;

/* フォント → font-sans / font-mono */
/* 先頭から探して、利用可能な最初のフォントが使われる。
   Inter は日本語を含まないので、2番目に日本語フォントを置く */
--font-sans: Inter, "Noto Sans JP", ui-sans-serif, system-ui, -apple-system, sans-serif;
--font-mono: "JetBrains Mono", "Fira Code", ui-monospace, monospace;

/* 標準には無い超小サイズの文字 → text-2xs（10px / 行間14px） */
--text-2xs: 0.625rem;
--text-2xs--line-height: 0.875rem;

/* アニメーション → animate-fade-in など */
/* 書き方: "<キーフレーム名> <時間> <イージング> <繰り返し> <塗り潰し>" */
--animate-fade-in: fade-in 0.3s ease-out;
--animate-fade-out: fade-out 0.3s ease-out;
```

```
  --animate-slide-in-right: slide-in-right 0.3s ease-out;
  --animate-slide-in-up: slide-in-up 0.4s ease-out;
  --animate-scale-in: scale-in 0.2s ease-out;
  /* infinite: 無限ループ、linear: 等速 */
  --animate-shimmer: shimmer 2s infinite linear;
  --animate-toast-in: toast-slide-in 0.3s ease-out;
  /* forwards: 終了状態をキープ */
  --animate-toast-out: toast-slide-out 0.3s ease-in forwards;

  /* @keyframes はアニメーションの「始まりから終わりまでのコマ」を定義する。
     from { 開始状態 } to { 終了状態 } の形が基本。
     0% { ... } 100% { ... } のようにパーセント刻みで複数指定もできる。
     @theme の中に書くと、そのアニメーションを使ったときだけ CSS に出力される。 */

  /* fade-in: 少し下から、透明 → 不透明にフェードイン */
  @keyframes fade-in {
    from {
      /* 透明（見えない） */
      opacity: 0;
      /* 8px 下にずらした位置から始まる */
      transform: translateY(8px);
    }
    to {
      /* 不透明（見える） */
      opacity: 1;
      /* 元の位置へ */
      transform: translateY(0);
    }
  }

  /* fade-out: 上記の逆。表示状態 → 透明＋少し下に消える */
  @keyframes fade-out {
    from {
      opacity: 1;
      transform: translateY(0);
    }
    to {
      opacity: 0;
      transform: translateY(8px);
    }
  }

  /* 右からスライドイン（X軸＝横方向の動き） */
  @keyframes slide-in-right {
    from {
      opacity: 0;
      /* 16px 右にずれた位置 */
      transform: translateX(16px);
    }
    to {
      opacity: 1;
      /* 元の位置 */
      transform: translateX(0);
    }
  }

  /* 下からスライドアップ（カードの一斉登場などに使う） */
  @keyframes slide-in-up {
    from {
      opacity: 0;
      transform: translateY(16px);
    }
    to {
      opacity: 1;
```

```
      transform: translateY(0);
    }
  }

  /* scale-in: 少し小さい状態 → 通常サイズに拡大しながら出現 */
  @keyframes scale-in {
    from {
      opacity: 0;
      /* 95% の大きさ */
      transform: scale(0.95);
    }
    to {
      opacity: 1;
      /* 100%（等倍） */
      transform: scale(1);
    }
  }

  /* shimmer: ローディングスケルトンの光が左から右へ流れるアニメーション */
  @keyframes shimmer {
    0% {
      /* 背景画像を左外側に配置 */
      background-position: -200% 0;
    }
    100% {
      /* 右外側まで動かす */
      background-position: 200% 0;
    }
  }

  /* トーストが右から滑り込んでくる */
  @keyframes toast-slide-in {
    from {
      opacity: 0;
      /* 100% = 自身の幅ぶん右にいる状態 */
      transform: translateX(100%);
    }
    to {
      opacity: 1;
      transform: translateX(0);
    }
  }

  /* トーストが右に滑り出ていく（消える時） */
  @keyframes toast-slide-out {
    from {
      opacity: 1;
      transform: translateX(0);
    }
    to {
      opacity: 0;
      transform: translateX(100%);
    }
  }
}

/* ========================================
   2. ダークモード用の上書き
   ======================================== */
/* <html class="dark"> のように dark クラスが付いたとき、
   この中の変数だけが上書きされる。CSS変数の「値だけを差し替える」
   仕組みなので、各コンポーネントで dark:bg-... を一つずつ書かなくても
   ダークモードに対応できる。 */
.dark {
```

```
  /* ほぼ黒（slate-900）に */
  --color-background: #0f172a;
  /* 文字は逆に白系に */
  --color-foreground: #f8fafc;
  --color-muted: #94a3b8;
  --color-muted-foreground: #64748b;
  /* 暗い枠線 */
  --color-border: #334155;
  /* カード背景は背景より少し明るい暗色 */
  --color-card: #1e293b;
  --color-card-foreground: #f8fafc;
}

/* ======================================
   3. ベースレイヤー
   ====================================== */
/* @layer base は「層（layer）」のひとつ。
   ここに書くと、ユーティリティクラスより優先度が低くなるので
   個別の Tailwind クラスで上書きしやすい。 */
@layer base {
  /* HTML・Body のリセットと基本設定 */
  html {
    /* ページ内リンクのスクロールをなめらかに */
    scroll-behavior: smooth;
    /* Mac の Chrome/Safari でフォントを滑らかに表示 */
    -webkit-font-smoothing: antialiased;
    /* Mac の Firefox でフォントを滑らかに表示 */
    -moz-osx-font-smoothing: grayscale;
  }

  body {
    /* @apply は CSS の中で Tailwind のクラスを使うための書き方。 */
    /* bg-background : 背景色を --color-background に（ダークでは .dark の値になる） */
    /* text-foreground : 文字色を --color-foreground に */
    @apply bg-background text-foreground;
    /* OpenType の合字機能を有効化（綺麗な文字に） */
    font-feature-settings: "rlig" 1, "calt" 1;
    /* ダークモード切替時に背景・文字色を 0.3 秒かけて滑らかに変える */
    transition: background-color 0.3s ease, color 0.3s ease;
  }

  /* フォーカスリングのデフォルトスタイル */
  /* :focus-visible は「キーボードでフォーカスされたとき」のみ発火する擬似クラス。
     マウスクリック時には発火しないので、マウスユーザーに邪魔にならない。 */
  *:focus-visible {
    /* outline-2          : outline（外側の輪郭線）の太さ 2px */
    /* outline-offset-2   : outline を要素から 2px 離す */
    /* outline-primary-500: outline の色を primary-500 に */
    @apply outline-2 outline-offset-2 outline-primary-500;
  }

  /* 見出しのデフォルトスタイル */
  h1 {
    /* text-3xl       : 文字サイズ 1.875rem（30px） */
    /* font-bold      : 文字の太さを「bold（700）」に */
    /* tracking-tight : 文字間隔（letter-spacing）を狭く（-0.025em） */
    @apply text-3xl font-bold tracking-tight;
  }

  h2 {
    /* text-2xl       : 1.5rem（24px） */
    /* font-semibold  : 太字よりやや細い 600 */
    @apply text-2xl font-semibold tracking-tight;
  }
```

```
  h3 {
    /* text-xl        : 1.25rem (20px) */
    @apply text-xl font-semibold;
  }

  /* リンクのデフォルトスタイル */
  a {
    /* text-primary-600           : ライトモード時の文字色 */
    /* hover:text-primary-700     : ホバー時はより濃く */
    /* dark:text-primary-400      : ダークモード時はより明るく（dark: 接頭辞） */
    /* dark:hover:text-primary-300: ダーク × ホバーの組み合わせ */
    @apply text-primary-600 hover:text-primary-700 dark:text-primary-400 dark:hover:text-primary-300;
    /* 色変化を 0.15 秒かけて滑らかに */
    transition: color 0.15s ease;
  }
}

/* ======================================
   4. コンポーネントレイヤー
   ====================================== */
/* @layer components はベースより優先度が高く、ユーティリティより低い層。
   .btn のような再利用可能なクラスをここに定義する。
   ※ v4 では「自作クラスを @apply で取り込む（@apply btn）」ことはできないので、
     共通部分は「.btn, .btn-primary, ...」のようにセレクタを並べて1か所に書く。 */
@layer components {
  /* ボタンの共通スタイル（全ボタンの土台） */
  /* カンマで区切って並べたクラスすべてに、同じスタイルが当たる */
  .btn,
  .btn-primary,
  .btn-secondary,
  .btn-outline,
  .btn-danger,
  .btn-ghost {
    /* inline-flex                 : display: inline-flex（横並び＋インライン要素） */
    /* items-center                : 縦方向中央揃え（align-items: center） */
    /* justify-center              : 横方向中央揃え（justify-content: center） */
    /* rounded-lg                  : 角丸 0.75rem（上の --radius-lg） */
    /* px-4 py-2                   : 横16px、縦8px の内側余白 */
    /* text-sm                     : フォントサイズ 0.875rem (14px) */
    /* font-medium                 : 文字太さ 500 */
    /* transition-all duration-200 : 全プロパティを 200ms かけて変化 */
    /* ease-in-out                 : 加速→減速のカーブ */
    /* focus-visible:outline-hidden : キーボードフォーカス時のデフォルト輪郭線を消す */
    /* focus-visible:ring-2        : 代わりに 2px のリング（box-shadow） */
    /* focus-visible:ring-offset-2 : リングを要素から 2px 離す */
    /* disabled:pointer-events-none : 無効化時はクリックできない */
    /* disabled:opacity-50         : 無効化時は半透明（50%） */
    @apply inline-flex items-center justify-center
           rounded-lg px-4 py-2
           text-sm font-medium
           transition-all duration-200 ease-in-out
           focus-visible:outline-hidden focus-visible:ring-2
           focus-visible:ring-offset-2
           disabled:pointer-events-none disabled:opacity-50;
  }

  .btn-primary {
    /* 共通部分は上でまとめて指定済みなので、ここでは色だけを足す */
    /* bg-primary-600 text-white    : インディゴ背景に白文字 */
    /* hover:bg-primary-700         : ホバー時にやや濃く */
    /* active:bg-primary-800        : クリック中（押している瞬間）はさらに濃く */
    /* focus-visible:ring-primary-500 : フォーカスリングをインディゴで */
    @apply bg-primary-600 text-white
```

```
          hover:bg-primary-700
          active:bg-primary-800
          focus-visible:ring-primary-500;
  }

  .btn-secondary {
    /* セカンダリ（エメラルド系）のボタン。primary と同じ構造で色だけ違う */
    @apply bg-secondary-600 text-white
          hover:bg-secondary-700
          active:bg-secondary-800
          focus-visible:ring-secondary-500;
  }

  .btn-outline {
    /* アウトラインボタン：枠線だけのスタイル */
    /* border-2 border-primary-600 : 2px の枠線をプライマリ色で */
    /* text-primary-600            : 文字色もプライマリ */
    /* hover:bg-primary-50         : ホバー時にうっすら塗りつぶし */
    /* active:bg-primary-100       : 押下時はもう少し濃く */
    /* dark:border-primary-400 ... : ダークモード用は明るめのプライマリで */
    /* dark:hover:bg-primary-950   : ダークでのホバーは超暗いプライマリ */
    @apply border-2 border-primary-600 text-primary-600
          hover:bg-primary-50
          active:bg-primary-100
          dark:border-primary-400 dark:text-primary-400
          dark:hover:bg-primary-950
          focus-visible:ring-primary-500;
  }

  .btn-danger {
    /* 危険な操作（削除など）用の赤いボタン */
    @apply bg-red-600 text-white
          hover:bg-red-700
          active:bg-red-800
          focus-visible:ring-red-500;
  }

  .btn-ghost {
    /* ゴーストボタン：通常は背景なし、ホバー時だけ薄く塗る控えめなボタン */
    @apply text-gray-600 hover:bg-gray-100
          dark:text-gray-400 dark:hover:bg-gray-800
          focus-visible:ring-gray-500;
  }

  /* カードの共通スタイル（.card と .card-hover の両方に当てる） */
  .card,
  .card-hover {
    /* rounded-xl              : 角丸 1rem（上の --radius-xl） */
    /* border border-border    : 枠線の色を --color-border に */
    /* bg-card                 : 背景色を --color-card に */
    /* text-card-foreground    : 文字色を --color-card-foreground に */
    /* shadow-sm               : 小さめの影（上の --shadow-sm） */
    @apply rounded-xl border border-border
          bg-card text-card-foreground
          shadow-sm;
    /* 影と位置変化を 0.2 秒かけて滑らかに */
    transition: box-shadow 0.2s ease, transform 0.2s ease;
  }

  .card-hover {
    /* card に「ホバーで影を大きく＋少し上に浮く」を追加 */
    /* hover:shadow-lg     : ホバー時に大きな影 */
    /* hover:-translate-y-1: 上に -0.25rem (4px) 移動 */
    @apply hover:shadow-lg hover:-translate-y-1;
```

```
}

/* 入力フィールドの共通スタイル */
.input {
  /* w-full                              : 横幅100% */
  /* rounded-lg border border-border     : 角丸＋枠線 */
  /* bg-background                       : 背景色 */
  /* px-4 py-2.5                         : 内側余白（py-2.5 は 0.625rem = 10px） */
  /* text-sm                             : 14px */
  /* placeholder:text-muted-foreground   : placeholder（入力前の薄い案内文）の色 */
  /* focus:border-primary-500            : フォーカス時の枠線色 */
  /* focus:outline-hidden                : ブラウザ標準のアウトラインを消す */
  /* focus:ring-2                        : 代わりに2pxリング */
  /* focus:ring-primary-500/20           : リングの色＝primary-500 の透明度20% */
  /* disabled:cursor-not-allowed         : 無効化時はカーソルが「禁止マーク」に */
  /* disabled:opacity-50                 : 無効時は半透明 */
  /* dark:focus:border-primary-400       : ダーク × フォーカス時の枠線色 */
  @apply w-full rounded-lg border border-border
         bg-background
         px-4 py-2.5
         text-sm
         placeholder:text-muted-foreground
         focus:border-primary-500 focus:outline-hidden focus:ring-2
         focus:ring-primary-500/20
         disabled:cursor-not-allowed disabled:opacity-50
         dark:focus:border-primary-400 dark:focus:ring-primary-400/20;
  transition: border-color 0.15s ease, box-shadow 0.15s ease;
}

/* バッジ（badge：ラベルや状態を示す小さな印）の共通部分 */
.badge,
.badge-primary,
.badge-success,
.badge-warning,
.badge-error {
  /* inline-flex items-center : 横並びで中央揃え（中にアイコン入れる時のため） */
  /* rounded-full             : 角丸を最大（円形/カプセル形） */
  /* px-2.5 py-0.5            : 横10px、縦2px の余白 */
  /* text-xs                  : 12px */
  /* font-medium              : 太さ500 */
  @apply inline-flex items-center rounded-full
         px-2.5 py-0.5
         text-xs font-medium;
}

.badge-primary {
  /* 「読書中」など、注目度を上げたいバッジ */
  /* bg-primary-100 text-primary-700 : 薄い背景に濃い文字色 */
  /* dark:bg-primary-900 dark:text-primary-300 : ダークモードは反転 */
  @apply bg-primary-100 text-primary-700
         dark:bg-primary-900 dark:text-primary-300;
}

.badge-success {
  /* 「読了」など成功状態を表す緑バッジ */
  @apply bg-green-100 text-green-700
         dark:bg-green-900 dark:text-green-300;
}

.badge-warning {
  /* 「読みたい」「注意」など警告を表す黄バッジ */
  @apply bg-yellow-100 text-yellow-700
         dark:bg-yellow-900 dark:text-yellow-300;
}
```

```
  .badge-error {
    /* エラー状態を表す赤バッジ */
    @apply bg-red-100 text-red-700
           dark:bg-red-900 dark:text-red-300;
  }
}

/* ========================================
   5. 自作ユーティリティ（@utility）
   ======================================== */
/* @utility 名前 { ... } で、Tailwind に無い小さな便利クラスを自作できる。
   作ったクラスは p-4 などと同じ「ユーティリティ」として扱われ、
   hover: や md: などの接頭辞も付けられる。
   ※ 複数行の省略表示 line-clamp-1 / line-clamp-2 / line-clamp-3 は
     Tailwind に最初から入っているので、自作する必要はない。 */

/* container: 中央寄せ＋画面幅に応じた左右の余白
   （v4 の標準の container は「最大幅」だけなので、中央寄せと余白をここで足す） */
@utility container {
  /* 左右の margin を auto にして中央寄せ */
  margin-inline: auto;
  /* 全画面共通の最小余白 = 16px */
  padding-inline: 1rem;
  /* 640px 以上 = 32px */
  @variant sm {
    padding-inline: 2rem;
  }
  /* 1024px 以上 = 64px */
  @variant lg {
    padding-inline: 4rem;
  }
  /* 1536px 以上では、最大幅を標準の 1536px より少し狭く（読みやすさのため） */
  @variant 2xl {
    max-width: 1400px;
  }
}

/* スクロールバーのカスタマイズ */
@utility scrollbar-thin {
  /* Firefox 用: スクロールバーを細く */
  scrollbar-width: thin;
  /* つまみ色 トラック色 */
  scrollbar-color: var(--color-muted-foreground) transparent;

  /* &::-webkit-scrollbar は Chrome/Safari 用のスクロールバー擬似要素 */
  &::-webkit-scrollbar {
    /* 縦スクロールバーの幅 */
    width: 6px;
    /* 横スクロールバーの高さ */
    height: 6px;
  }

  &::-webkit-scrollbar-track {
    /* スクロールバーの「軌道（背景部分）」を透明に */
    background: transparent;
  }

  &::-webkit-scrollbar-thumb {
    /* つまみの色 */
    background-color: var(--color-muted-foreground);
    /* つまみを丸く */
    border-radius: 3px;
  }
```

```
}

/* グラスモーフィズム（glassmorphism：すりガラス風効果） */
@utility glass {
  /* bg-white/80      : 白の透明度80%背景（数字 / 数字 で透明度指定） */
  /* backdrop-blur-md : 背景をぼかす（半透明の向こう側がモザイク状に） */
  /* dark:bg-gray-900/80 : ダーク時は暗い半透明背景 */
  @apply bg-white/80 backdrop-blur-md dark:bg-gray-900/80;
}
```

▼ コードを1つずつ分解して解説

このCSSはとても長く見えますが、やっていることは「①色などの値に名前を付ける」「②その名前を使ってまとめてスタイルを定義する」の2つだけです。塊ごとに見ていきましょう。

解説1: Tailwind を読み込み、ダークモードの合図を決める（ `@import` / `@custom-variant` ）

コピー

```
@import "tailwindcss";

@custom-variant dark (&:where(.dark, .dark *));
```

- `@import "tailwindcss";` は Tailwind CSS 本体を読み込む1行です。ビルド時にこの1行が、ブラウザごとの違いをそろえるリセット CSS（base）と、`p-4`・`bg-red-500` などのクラス（utilities）に**展開**されます。
- **この1行が無いと Tailwind のクラスは一切効きません。**必ずファイルの先頭に書きます。
- `@custom-variant dark (…)` は「`dark:` の付いたクラスを**いつ**効かせるか」を決める宣言です。`&:where(.dark, .dark *)` は「`dark` クラスが付いた要素自身か、その中にある要素」という意味で、`<html class="dark">` にすればページ全体が当てはまります。書き忘れると、v4 の `dark:` は OS の設定にしか反応しません（5.1 節のつまずきポイント）。

> **用語: ディレクティブ（directive）** … CSSやツールに対する「特別な指示」を表す命令文のこと。`@` から始まるものが多い（ `@import` , `@theme` , `@layer` , `@apply` など）。

`@import "tailwindcss"` で読み込まれる CSS は、base（土台）・components（部品）・utilities（便利クラス）という**層（レイヤー）**に分けて並べられます。

3つの層は、**積み重なった棚**のような関係です。上の段ほど強く、同じ要素に別々の段からスタイルが当たってぶつかったときは、上の段のものが勝ちます。

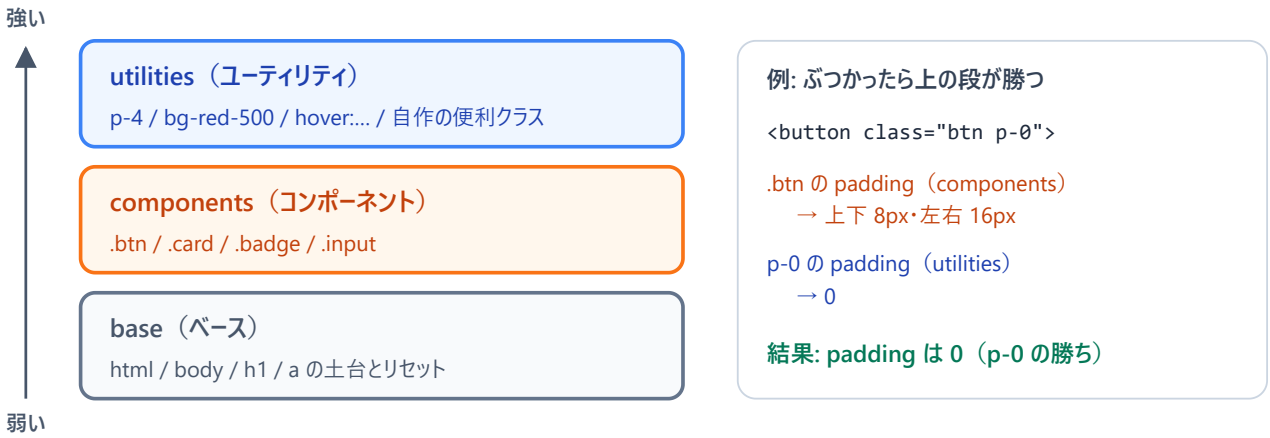


図9-9: CSS の3つの層（レイヤー）と強さの順番（実際には一番下に、色などの変数を置く theme の層もある）

この仕組みのおかげで、「`.btn` で共通の見た目を作っておき、特定のボタンだけ `p-0` や `w-full` などのユーティリティで**部分的に上書きする**」ことが安心してできます。

v3 では、この部分を `@tailwind base;` `@tailwind components;` `@tailwind utilities;` の3行で書き、ダークモードの方式は `tailwind.config.ts` の `darkMode: "class"` で指定していました（1.3 節）。古い記事でこの3行を見かけたら「v3 の書き方だな」と読み替えてください。

解説2: 色に名前を付けてクラスにする（`@theme` / カラートークン）

コピー

```
@theme {
  /* 一番薄いインディゴ（背景バッジ等に使う） */
  --color-primary-50: #eef2ff;
  /* ボタン背景でよく使う濃さ */
  --color-primary-600: #4f46e5;
  /* ページ全体の背景色（白）→ bg-background */
  --color-background: #ffffff;
}
```

- （上はコードの抜粋です）`@theme { … }` の中に書いた `--color-primary-600` のような **CSS変数（カスタムプロパティ）** は、Tailwind の**クラスとして使える**ようになります。名前が `--color-` で始まるので「色」と判断され、`bg-primary-600`（背景）・`text-primary-600`（文字）・`border-primary-600`（枠線）などが一度に使えるようになります。
- 同時に `:root`（ページの一番外側＝`<html>`）の CSS 変数としても出力されるので、`var(--color-primary-600)` と書けばページのどこからでも呼び出せます。本章のコンポーネントで使っている `bg-[var(--color-card)]` は、こちらの使い方です。
- 値の `#4f46e5` は色を表す16進数コードで、`#` の後ろ2桁ずつが「赤・緑・青」の強さを表します。色を直接書かず変数にしておくと、後で色を変えたいとき**1か所直すだけ**で全体に反映できます。

- 影（`--shadow-`）・角丸（`--radius-`）・フォント（`--font-`）・文字サイズ（`--text-`）も同じしくみで、**名前の前半で何のクラスになるかが決まります**（1.1.1 節の「名前空間」）。

> **用語: カラートークン（color tokens）／デザイントークン** … 「primary-600 はこの色」のように、デザインで使う値に意味のある名前を付けて一元管理したもの。チームでデザインの一貫性を保つための定番手法。

解説3: ダークモードで色を入れ替える（`.dark`）

コピー

```
.dark {
  /* ほぼ黒（slate-900）に */
  --color-background: #0f172a;
  /* 文字は逆に白系に */
  --color-foreground: #f8fafc;
  /* カード背景は背景より少し明るい暗色 */
  --color-card: #1e293b;
}
```

- `<html class="dark">` のように `dark` クラスが付いたときだけ、この中の**変数の値が上書き**されます。
- ポイントは「**新しいクラスを足すのではなく、同じ変数の中身だけを差し替える**」こと。`bg-background` などのクラスは中で `var(--color-background)` を使っているので、変数を黒に入れ替えるだけで画面全体がダークモードに変わります。
- 各コンポーネントで `dark:bg-...` を一つずつ書かなくて済むのが、CSS変数方式の最大の利点です。
- ただし影（`--shadow-sm` など）は、ここで上書きしても変わりません。v4 の `shadow-sm` などのクラスは、`@theme` の値がクラスの中に直接書き込まれるためです。ダークモードで影を変えたいときは `dark:shadow-lg` のように `dark:` を使います。

> **用語: ダークモード（dark mode）** … 背景を黒・濃灰にした暗い配色モード。夜間や暗所での目の負担を減らす目的で用意する。

解説4: ベースレイヤーで土台を整える（`@layer base`）

コピー

```
@layer base {
  body {
    @apply bg-background text-foreground;
    transition: background-color 0.3s ease, color 0.3s ease;
  }
}
```

- `@layer base`（レイヤー・ベース）は「**一番優先度の低い層**」にスタイルを置く指定です。低い層に置くと、後から個別の Tailwind クラスで簡単に上書きできます。
- `@apply`（アプライ）は「**CSSの中でTailwindクラスを使う**」ための命令です。`bg-background` は解説2で `@theme` に登録した `--color-background` を背景色にするクラスです。
- `transition: ... 0.3s ease` で、ダークモード切替時に背景と文字色が**0.3秒かけて滑らかに**変わるようにしています。

> **用語: レイヤー（layer）／@apply** … `@layer` はCSSの優先順位を整理するための「層」。`@apply` は長くなりがちなTailwindクラス指定をCSS側にまとめる書き方。

解説5: ボタンの共通スタイルを作る（`@layer components` の `.btn`）

コピー

```
@layer components {
  .btn,
  .btn-primary,
  .btn-secondary,
  .btn-outline,
  .btn-danger,
  .btn-ghost {
    @apply inline-flex items-center justify-center
           rounded-lg px-4 py-2
           text-sm font-medium
           transition-all duration-200 ease-in-out
           disabled:pointer-events-none disabled:opacity-50;
  }

  .btn-primary {
    @apply bg-primary-600 text-white
           hover:bg-primary-700;
  }
}
```

- （上はコードの抜粋です）`.btn, .btn-primary, …` のように**カンマで区切ってクラスを並べる**と、並べたすべてのクラスに同じスタイルが当たります。これが「全ボタン共通の土台」です。
- 中身は、`inline-flex items-center justify-center` で中身を中央寄せ、`rounded-lg` で角丸、`px-4 py-2` で内側余白、`transition-all duration-200` で変化を200msかけて滑らかにする指定です。
- `disabled:opacity-50` は「ボタンが無効化されたときは半透明にする」指定で、`disabled:` は状態に応じてスタイルを変える接頭辞です。
- `.btn-primary` には、土台に足りない**色だけ**（インディゴ背景＋白文字）を書いています。こうすると共通部分を何度も書かずに済みます。

- v3 では `.btn-primary { @apply btn …; }` のように**自作クラスを `@apply` で取り込む**書き方もできましたが、v4 の `@apply` で取り込めるのは Tailwind のクラス（と `@utility` で作ったクラス）だけです。そのため本章では「セレクタを並べる」方法を使っています。

> **用語: ホバー（hover）** … マウスカーソルを要素の上に乗せた状態。`hover:bg-primary-700` は「乗せたときだけ背景を濃くする」という指定。

完成した `.btn-primary`（本文のコード全体）は、操作の状態によって次の5通りの見た目に切り替わります。PDF や紙ではマウスを乗せられないので、それぞれの状態を並べて描きました。



図9-10: .btn-primary の状態ごとの見た目（色が少しずつ濃くなる・輪が出る・薄くなる、の違いに注目）

「押せる」「今押している」「キーボードでここを選んでいる」「今は押せない」が見た目だけで伝わることで、ユーザーは迷わずに操作できます。特に **Tab キーで選択したときの輪（フォーカスリング）** は、マウスを使わない人にとって「今どこにいるか」を知る唯一の手がかりなので、消さないようにしましょう。

解説6: カードと入力欄の共通スタイル（`.card` / `.input`）

コピー

```
.card,
.card-hover {
  @apply rounded-xl border border-border
         bg-card text-card-foreground
         shadow-sm;
  transition: box-shadow 0.2s ease, transform 0.2s ease;
}
.input {
  @apply w-full rounded-lg border border-border
         px-4 py-2.5 text-sm
         focus:border-primary-500 focus:outline-hidden focus:ring-2;
}
```

- `.card` は書籍カードなどの土台です。`border-border` は「枠線の色を `--color-border` に」、`bg-card` は「背景を `--color-card` に」という指定で、どれも解説2の変数経由なので**ダークモードでも自動で色が変わります。**

- `.input` は入力欄の共通スタイル。`w-full` で横幅いっぱい、`focus:outline-hidden` はブラウザ標準の輪郭線を消す指定（v3 の `outline-none` に当たる）、`focus:ring-2` は「クリックして入力中のとき、枠の周りに2pxの光るリングを出す」指定です。`focus:` は入力欄が選択された状態を表します。
- このように共通スタイルを `@layer components` にまとめておくと、HTML側では `<div class="card">` と書くだけで統一された見た目になります。

> **用語: フォーカス（focus）** … 入力欄やボタンが「いま操作対象として選ばれている」状態。キーボード操作のユーザーにとって、どこが選択中かを示すリング表示が重要になる。

解説7: 自作ユーティリティを作る（`@utility`）

コピー

```
@utility glass {
  @apply bg-white/80 backdrop-blur-md dark:bg-gray-900/80;
}
```

- `@utility 名前 { … }` は、Tailwind に無い便利クラスを**自作**する書き方です（名前の前に `.` は付けません）。作ったクラスは `p-4` などと同じユーティリティとして扱われ、`md:glass` のように接頭辞も付けられます。
- `glass` は「半透明の白＋向こう側をぼかす」で、すりガラスのように見せるクラスです。同じやり方で、`container`（中央寄せと画面幅に応じた余白）と `scrollbar-thin`（細いスクロールバー）も作っています。`@variant sm { … }` は「`sm:` の条件（640px 以上）のときだけ」という意味です。
- 長いタイトルを「…」で打ち切る `line-clamp-1` 〜 `line-clamp-3` は、Tailwind に**最初から入っている**ので自作は不要です（以前の版では自作していました）。Tailwind が作る CSS は次のとおりで、`-webkit-line-clamp` の数字が打ち切る行数です。

コピー

```
.line-clamp-2 {
  overflow: hidden;
  display: -webkit-box;
  -webkit-box-orient: vertical;
  -webkit-line-clamp: 2;
}
```

> **用語: ユーティリティクラス（utility class）** … `p-4`（余白16px）のように「1つの機能だけを持つ小さなクラス」。これらを組み合わせてデザインするのがTailwindの考え方。

解説8: 動きの素材を定義する（`--animate-*` と `@keyframes`）

コピー

```
@theme {
  --animate-fade-in: fade-in 0.3s ease-out;

  @keyframes fade-in {
    from {
      /* 透明（見えない） */
      opacity: 0;
      /* 8px 下にずらした位置から始まる */
      transform: translateY(8px);
    }
    to {
      /* 不透明（見える） */
      opacity: 1;
      /* 元の位置へ */
      transform: translateY(0);
    }
  }
}
```

- `@keyframes`（キーフレーム）は「**アニメーションの始まり（`from`）と終わり（`to`）のコマ**」を定義します。
- この `fade-in` は「透明で8px下にいる状態」から「不透明で元の位置」へ変化します。つまり**少し下からふわっと現れる**動きです。
- `--animate-fade-in: fade-in 0.3s ease-out;` は「`fade-in` のコマを 0.3 秒、最後ゆっくり（ease-out）で再生する」という登録で、これで `animate-fade-in` というクラスが使えるようになります。`@keyframes` を `@theme` の中に書いておくと、そのアニメーションを実際に使ったときだけ CSS に出力されます。
- `opacity`（透明度：0で透明、1で不透明）と `transform: translateY(...)`（縦方向の移動）を組み合わせるのがフェードイン演出の定番です。

> **用語: キーフレーム（keyframe）** … アニメーションの途中経過を指定する「コマ」。`from` / `to` のほか `0%` / `50%` / `100%` のようにパーセントで複数指定もできる。

1.1.1 v3 と v4 の違い（古い記事・古いプロジェクトを読むときのために）

1.1 節の `globals.css` は **v4** の書き方です。一方、ネットの記事や 2024年ごろまでに作られたプロジェクトでは、**v3** の書き方（`@tailwind` の3行 ＋ `tailwind.config.ts`）がまだよく見つかります。両者の違いを整理しておきましょう。いちばん大きな違いは、**設定が JavaScript のファイル（`tailwind.config.ts`）から CSS ファイルの中へ引っ越した**ことです。

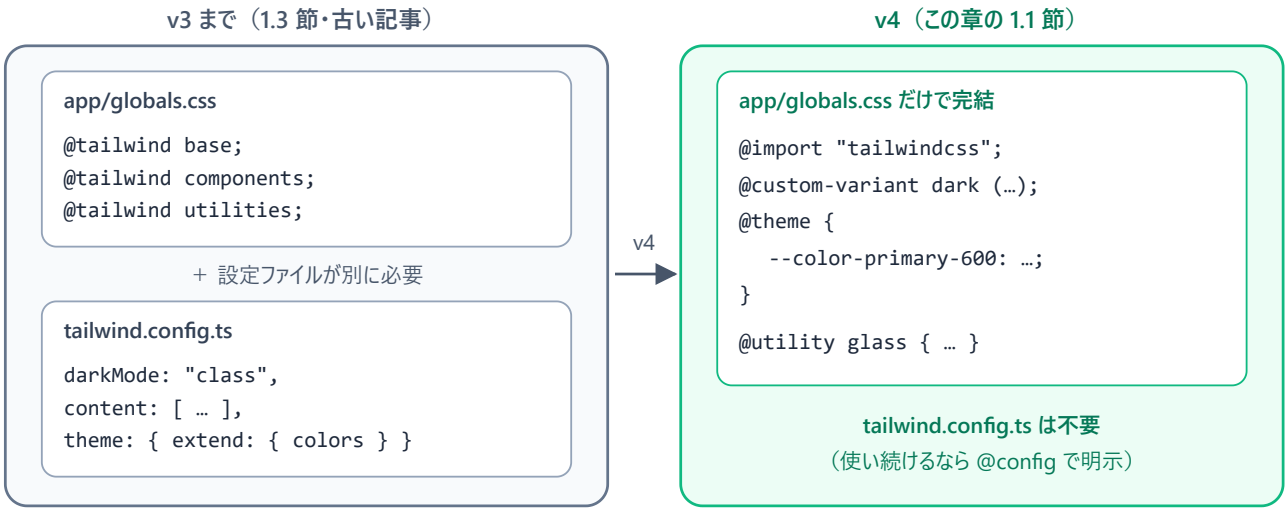


図9-11: v3 と v4 のファイル構成の違い（v4 では設定が CSS ファイルの中に集まる）

v3 と v4 の書き方の対応表です。

やりたいこと	v3（古い書き方）	v4（この章）
Tailwind を読み込む	`@tailwind base;` など3行	`@import "tailwindcss";` の1行
クラスを探すファイルの指定	`tailwind.config.ts` の `content`	指定不要（`.gitignore` に書かれたもの以外を自動で探す）。足りない場所だけ `@source` で追加
色・フォント・影などを追加	`theme.extend.colors` など	CSS の `@theme { --color-primary-600: #4f46e5; }`
CSS 変数を色クラスにする	`primary: { 600: "var(--…)" }`	`@theme inline { --color-background: var(--background); }`
アニメーションを追加	`theme.extend.keyframes` と `animation`	`@theme` の中に `--animate-fade-in: …;` と `@keyframes`
ダークモードを class で切替	`darkMode: "class"`	`@custom-variant dark (&:where(.dark, .dark *));`
自作クラス（`.btn` など）	`@layer components { .btn { … } }`（`@apply btn` で取り込める）	`@layer components` はそのまま使える（ただし `@apply btn` は不可）。接頭辞を付けたい小さなクラスは `@utility glass { … }`
`container` の中央寄せ・余白	`theme.container` の `center` / `padding`	`@utility container { margin-inline: auto; … }`
JS の設定ファイルを使い続ける	自動で読まれる	`@config "../tailwind.config.ts";` と書いたときだけ読まれる

ほかに、**名前や初期値が変わったクラス**もあります。古い記事のコードを v4 のプロジェクトで使うときは、次の点に気を付けてください。

v3	v4	メモ
`shadow-sm` / `shadow`	`shadow-xs` / `shadow-sm`	影・角丸・ぼかしの名前が1段ずつずれた
`rounded-sm` / `rounded`	`rounded-xs` / `rounded-sm`	同上
`outline-none`	`outline-hidden`	「輪郭線を見えなくする」意味のものは改名
`ring`（太さ3px）	`ring-3`	v4 の `ring` は 1px。この章の `ring-2` のように太さを書いていれば影響なし
`border`（色は gray-200）	`border`（色は文字色）	枠線の色は `border-gray-200` のように**必ず書く**

> **ヒント:** 既存の v3 プロジェクトは、公式の自動変換ツール `npx @tailwindcss/upgrade` で v4 の書き方へまとめて書き換えられます（Node.js 20 以上が必要）。必ず新しいブランチで試し、変更点を目で確認してから取り込みましょう。

参考までに、`create-next-app` が最初に作る `app/globals.css` は次の内容です（2026年10月時点の公式テンプレート）。上の表の `@import` と `@theme inline` がもう使われていることが分かります。1.1 節の `globals.css` は、このファイルを丸ごと置き換えたものです。

コピー

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #171717;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

@media (prefers-color-scheme: dark) {
  :root {
    --background: #0a0a0a;
    --foreground: #ededed;
  }
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}
```

v4 の globals.css を小さく作って動かす

> ▼ **このコードがやること（先に日本語で）**：1.1 節の `globals.css` は長いので、その中から代表的な4つ「ダークモードの切り替え方式」「切り替わる色の登録」「ブランド色とアニメーションの追加」「自作のボタンクラス」だけを取り出した**最小版**を作ります。練習用フォルダで CLI に通し、`bg-primary-600` や `btn-primary` などのクラスが本当に作られるかを確かめます。

コピー

```
/* app/globals.css（v4 版の最小構成） */

/* (1) Tailwind 本体を読み込む（v3 の @tailwind 3行の代わり） */
@import "tailwindcss";

/* (2) dark: を「<html class="dark"> のとき」に効かせる */
@custom-variant dark (&:where(.dark, .dark *));

/* (3) 切り替わる色は普通の CSS 変数で持つ */
:root {
  --background: #ffffff;
  --foreground: #0f172a;
}

.dark {
  --background: #0f172a;
  --foreground: #f8fafc;
}

/* (4) CSS 変数を Tailwind の色クラスとして登録する */
@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
}

/* (5) ブランド色とアニメーションを登録する */
@theme {
  --color-primary-100: #e0e7ff;
  --color-primary-600: #4f46e5;
  --color-primary-700: #4338ca;
  --animate-fade-in: fade-in 0.3s ease-out;

  @keyframes fade-in {
    from {
      opacity: 0;
      transform: translateY(8px);
    }
    to {
      opacity: 1;
      transform: translateY(0);
    }
  }
}

/* (6) 自作クラスは @utility で作る */
@utility btn-primary {
```

```
  @apply rounded-lg bg-primary-600 px-4 py-2 text-white hover:bg-primary-700;
}
```

コピー

```
// app/page.tsx（上の CSS で作ったクラスを使ってみる）
export default function Page() {
  return (
    <main className="bg-background p-8 text-foreground">
      <button className="btn-primary animate-fade-in">保存</button>
      <span className="bg-primary-100 dark:bg-primary-700">読書中</span>
    </main>
  );
}
```

コピー

```
# 練習用フォルダで、app/globals.css を入力にして out.css を作る
npx @tailwindcss/cli -i app/globals.css -o out.css
```

▼ **実行結果**（`out.css` の `@layer utilities` の部分。`page.tsx` で使ったクラスだけが作られている）

コピー

```
@layer utilities {
  .animate-fade-in {
    animation: var(--animate-fade-in);
  }
  .btn-primary {
    border-radius: var(--radius-lg);
    background-color: var(--color-primary-600);
    padding-inline: calc(var(--spacing) * 4);
    padding-block: calc(var(--spacing) * 2);
    color: var(--color-white);
    &:hover {
      @media (hover: hover) {
        background-color: var(--color-primary-700);
      }
    }
  }
  .bg-background {
    background-color: var(--background);
  }
  .bg-primary-100 {
    background-color: var(--color-primary-100);
  }
  .p-8 {
    padding: calc(var(--spacing) * 8);
  }
  .text-foreground {
    color: var(--foreground);
  }
  .dark\:bg-primary-700:where(.dark, .dark *) {
    background-color: var(--color-primary-700);
  }
}
```

▼ コードを1つずつ分解して解説

解説1: `@custom-variant dark` でダークモードの合図を決める

コピー

```
@custom-variant dark (&:where(.dark, .dark *));
```

- v4 の `dark:` は、何も書かないと **OS の設定（ダーク/ライト）に自動で連動**します。この章のように「ボタンで切り替えたい」ときは、この1行で「`dark` クラスが付いた要素とその中身のときに `dark:` を効かせる」と教えます（v3 の `darkMode: "class"` に当たります）。

- 実行結果の最後の `.dark\:bg-primary-700:where(.dark, .dark *)` が、この設定どおりに作られたクラスです。

解説2: 切り替わる色は `:root` / `.dark` + `@theme inline`

コピー

```
:root {
  --background: #ffffff;
}
.dark {
  --background: #0f172a;
}
@theme inline {
  --color-background: var(--background);
}
```

- `:root` と `.dark` で同じ名前の変数 `--background` に別の値を入れておき、`@theme inline` で「`--color-background` という色は、`var(--background)` を使ってね」と Tailwind に登録します。
- 名前が `--color-` で始まるので、`bg-background` や `text-background` などのクラスが使えるようになります。実行結果の `.bg-background { background-color: var(--background); }` のとおり、**クラスが直接 `var(--background)` を見る**ので、`.dark` で値が変わればクラスの色も変わります。
- `inline` を付けるのは「変数の中身（`var(--background)`）をそのままクラスに書き込んで」という意味です。切り替わる値を参照するときは `@theme inline` を使うのが公式のおすすめです。
- 1.1 節では別のやり方として、`@theme` に**ライトの値を直接**書き、`.dark` で同じ名前の変数を上書きしました。どちらも「変数の中身が入れ替わるとクラスの色が変わる」点は同じです。`create-next-app` の初期ファイルはこちらの `@theme inline` 方式なので、両方読めるようにしておきましょう。

解説3: `@theme` に書いた名前がクラス名になる

コピー

```
@theme {
  --color-primary-600: #4f46e5;
  --animate-fade-in: fade-in 0.3s ease-out;
}
```

- `@theme` の中の変数は、**名前の前半（名前空間）で何のクラスになるかが決まります。**`--color-primary-600` は `bg-primary-600`・`text-primary-600` などに、`--animate-fade-in` は `animate-fade-in` になります。
- `@keyframes` を `@theme` の中に書いておくと、そのアニメーションを使ったときだけ CSS に出力されます。
- v3 では `tailwind.config.ts` の `extend.colors` や `animation` に書いていた内容が、ここに引っ越したと考えると分かりやすいです。

> **用語: 名前空間（namespace）** … 変数名の先頭部分（ `--color-` 、 `--font-` 、 `--animate-` 、 `--breakpoint-` など）のこと。Tailwind v4 はこの部分を見て、どの種類のクラスを作るかを決める。

解説4: 自作クラスは `@utility` で作る

コピー

```
@utility btn-primary {
  @apply rounded-lg bg-primary-600 px-4 py-2 text-white hover:bg-primary-700;
}
```

- v3 では `@layer components { .btn-primary { … } }` と書いていた自作クラスを、v4 では `@utility 名前 { … }` と書きます（名前の前に `.` は付けません）。
- 中では `@apply` で既存のクラスをまとめられます。実行結果を見ると、`hover:bg-primary-700` もきちんと `&:hover` の中に展開されています。
- `@utility` で作ったクラスは普通のユーティリティと同じ扱いになるので、`hover:btn-primary` や `md:btn-primary` のように接頭辞も付けられます。
- 1.1 節のアプリでは、ボタンを `@layer components` に書きました。components の層のクラスは、あとから `p-0` などのユーティリティで**部分的に上書きできる「部品」**です（図9-9）。一方 `@utility` で作ったクラスは utilities の層に入ります。「部品（ボタン・カード）は `@layer components` 、小さな道具（ `glass` など）は `@utility` 」と使い分けるのがおすすめです。

> **つまずきポイント: v4 のプロジェクトで `tailwind.config.ts` に色を足しても効かない**
>
> ネットの記事（v3 時代のもの）を見て `tailwind.config.ts` を作り、`extend.colors` に `primary` を足したのに、`bg-primary-600` が真っ白のまま……。これは **v4 が `tailwind.config.ts` を自動では読まない**からです。

コピー

```
// tailwind.config.ts（v3 時代の記事どおりに作った設定ファイル）
import type { Config } from "tailwindcss";

const config: Config = {
  content: ["./app/**/*.{ts,tsx}"],
  theme: {
    extend: {
      colors: {
        primary: { 600: "#4f46e5" },
      },
    },
  },
};

export default config;
```

コピー

```
/* NG: app/globals.css が @import だけ → 上の設定ファイルは読まれない */
@import "tailwindcss";
```

コピー

```
/* OK その1: @config で設定ファイルの場所を教える（globals.css から見た相対パス） */
@import "tailwindcss";
@config "../tailwind.config.ts";
```

コピー

```
/* OK その2（おすすめ）: v4 の書き方で、色を CSS に直接書く */
@import "tailwindcss";

@theme {
  --color-primary-600: #4f46e5;
}
```

`<button className="bg-primary-600 text-white">保存</button>` というページを用意して、NG と OK その1 を CLI で実際に比べた結果が次です。

▼ **実行結果**（それぞれの出力の `@layer utilities` の部分）

コピー

```
---- NG（@import だけ）: bg-primary-600 が作られていない ----
@layer utilities {
  .text-white {
    color: var(--color-white);
  }
}

---- OK その1（@config あり）: bg-primary-600 が作られた ----
@layer utilities {
  .bg-primary-600 {
    background-color: #4f46e5;
  }
  .text-white {
    color: var(--color-white);
  }
}
```

> **注意:** `globals.css` が `src/app/globals.css` にある場合、`@config` のパスは `"../../tailwind.config.ts"` になります（`globals.css` の場所から数えるため）。また、`@config` で読み込めるのは色やフォントなどの設定で、v3 の `corePlugins`・`safelist`・`separator` は v4 では使えません。新しく書くなら「OK その2」の `@theme` がおすすめです。

1.2 カスタムカラーパレットの設定

アプリでは以下のカラーパレットを採用しています。「カラーパレット（color palette）」とは「アプリ全体で使う色の組み合わせ」のことです。色を決めて統一することで、デザインに一貫性が出ます。

用途	カラー名	ベースカラー	説明
メインアクション	`primary`	インディゴ (#6366f1)	ボタン、リンク、フォーカスリング
補助アクション	`secondary`	エメラルド (#10b981)	成功状態、補助ボタン
成功	`success`	グリーン (#10b981)	操作成功の通知
警告	`warning`	アンバー (#f59e0b)	注意喚起
エラー	`error`	レッド (#ef4444)	エラーメッセージ、削除ボタン
情報	`info`	ブルー (#3b82f6)	情報表示
テキスト	`foreground`	スレート (#0f172a)	メインテキスト
背景	`background`	ホワイト (#ffffff)	ページ背景
ミュート	`muted`	スレート (#64748b)	補足テキスト

カラーパレットの設計指針:

- **一貫性**: 同じ意味を持つ色は全画面で統一する
- **コントラスト比**: WCAG 2.1 AA 基準（4.5:1 以上）を満たす
- **ダークモード対応**: 各カラーに明暗の両バリエーションを用意する

1.3 （参考）v3 の tailwind.config.ts

> **注意**: この節は **v3 のプロジェクト向けの参考**です。v4（この章のアプリ）では、ここに書いてある設定はすべて 1.1 節の `globals.css`（`@custom-variant`・`@theme`・`@utility`）に書き済みなので、**このファイルは作りません。**v4 のプロジェクトに置いても自動では読まれません（1.1.1 節のつまずきポイント）。古い記事や v3 のプロジェクトで `tailwind.config.ts` を見かけたときに読めるよう、設定項目ごとの意味を解説しておきます。

それでは、v3 の設定ファイルを見ていきましょう（1.1 節の v4 版と見比べると、同じ値が並んでいることが分かります）。

> **▼ このコードがやること（先に日本語で）**: v3 のプロジェクトで Tailwind CSS の動作を決めていた設定ファイルの例です（v4 のこの章のアプリでは作りません）。「ダークモードの切り替え方法」「どのファイルからクラスを探すか」「自分で追加した色・影・アニメーション」などをここにまとめます。特に、先ほど作った CSS 変数を `var(--color-primary-500)` の形で Tailwind に登録することで、`bg-primary-500` のようなクラスが使えるようになります。設定項目ごとの意味はコメントを参照してください。

コピー

```
// tailwind.config.ts
// ↑ Tailwind CSS のすべての設定をまとめたファイル。
//   Tailwind はビルド時にこのファイルを読み、どのクラスを生成するかを決める。

// "tailwindcss" パッケージから Config 型を「型としてのみ」インポート
// （import type なら実行時には消える → ビルドサイズを節約）
import type { Config } from "tailwindcss";

// config: Config と書くことで、TypeScript に「これは Tailwind の設定」と伝えられ
// 補完やタイプチェックが効くようになる
const config: Config = {
  // ダークモードの切り替え方法
  // "class"  : <html class="dark"> のように手動でクラスを付けて切り替える方式
  // "media"  : OSのカラーモード設定（prefers-color-scheme）に自動連動する方式
  // 本アプリでは「ユーザーが切り替えボタンで選べる」必要があるので "class" を選択
  darkMode: "class",

  // Tailwind がクラスを抽出する対象ファイルのパス
  // ここに書かれたファイル内で実際に使われているクラスだけを最終CSSに含めることで
  // バンドルサイズを最小化する（PurgeCSS / JIT エンジンの仕組み）
  content: [
    // Pages Router 用（使っていなくてもOK）
    "./pages/**/*.{js,ts,jsx,tsx,mdx}",
    // 自作コンポーネント
    "./components/**/*.{js,ts,jsx,tsx,mdx}",
    // App Router 用
    "./app/**/*.{js,ts,jsx,tsx,mdx}",
    // ※ ** はワイルドカード（任意の深さのフォルダ）、{ } は複数拡張子の指定
  ],

  // theme: Tailwind のデザインシステム本体。
  // ここに書いたものが Tailwind のクラスとして利用可能になる。
  theme: {
    // ========================================
    // コンテナの設定
    // ========================================
    // .container クラスの挙動をカスタマイズ。
    // <div class="container"> と書いたときの最大幅・余白・中央寄せの設定。
    container: {
      // コンテナを左右中央寄せ（margin: 0 auto 相当）
      center: true,
      // 画面サイズごとの左右パディング
      padding: {
        // 全画面共通の最小余白 = 16px
        DEFAULT: "1rem",
        // 640px 以上 = 32px
        sm: "2rem",
        // 1024px 以上 = 64px
        lg: "4rem",
        // 1280px 以上 = 80px
```

```
        xl: "5rem",
        // 1536px 以上 = 96px
        "2xl": "6rem",
      },
      // 各ブレークポイントでのコンテナ最大幅
      screens: {
        sm: "640px",
        md: "768px",
        lg: "1024px",
        xl: "1280px",
        // デフォルトの1536pxより少し狭く（読みやすさのため）
        "2xl": "1400px",
      },
    },

    // extend: 既存のテーマを「上書き」せず「追加」する。
    // theme: { colors: {...} } と書くと標準色が全部消えるので、extend を使うのが基本。
    extend: {
      // ========================================
      // カスタムカラーパレット
      // ========================================
      colors: {
        // プライマリカラー（インディゴ系）
        // CSS変数（globals.css の :root で定義した --color-primary-XX）を経由することで
        // ダークモード時に変数の値だけ差し替えれば自動で色が変わる
        primary: {
          // bg-primary-50 のように使える
          50: "var(--color-primary-50)",
          100: "var(--color-primary-100)",
          200: "var(--color-primary-200)",
          300: "var(--color-primary-300)",
          400: "var(--color-primary-400)",
          500: "var(--color-primary-500)",
          600: "var(--color-primary-600)",
          700: "var(--color-primary-700)",
          800: "var(--color-primary-800)",
          900: "var(--color-primary-900)",
          950: "var(--color-primary-950)",
        },
        // セカンダリカラー（エメラルド系）
        secondary: {
          50: "var(--color-secondary-50)",
          100: "var(--color-secondary-100)",
          200: "var(--color-secondary-200)",
          300: "var(--color-secondary-300)",
          400: "var(--color-secondary-400)",
          500: "var(--color-secondary-500)",
          600: "var(--color-secondary-600)",
          700: "var(--color-secondary-700)",
          800: "var(--color-secondary-800)",
          900: "var(--color-secondary-900)",
          950: "var(--color-secondary-950)",
        },
```

```
      // セマンティックカラー（用途ベースの命名）
      // bg-background で使える
      background: "var(--color-background)",
      // text-foreground で使える
      foreground: "var(--color-foreground)",
      muted: {
        // bg-muted（DEFAULT は階層なしの指定で参照される）
        DEFAULT: "var(--color-muted)",
        // text-muted-foreground
        foreground: "var(--color-muted-foreground)",
      },
      border: "var(--color-border)",
      card: {
        DEFAULT: "var(--color-card)",
        foreground: "var(--color-card-foreground)",
      },
      success: "var(--color-success)",
      warning: "var(--color-warning)",
      error: "var(--color-error)",
      info: "var(--color-info)",
    },

    // ========================================
    // カスタムフォント
    // ========================================
    // font-sans / font-mono クラスで使われるフォントの「フォールバック順」
    // 先頭から探して、利用可能な最初のフォントが使われる
    fontFamily: {
      sans: [
        // 第一候補: 欧文用の美しいフォント
        "Inter",
        // 日本語フォント（Inter は日本語が含まれないので必須）
        "Noto Sans JP",
        // OS標準のサンセリフ（macOS / Windows）
        "ui-sans-serif",
        // システム標準
        "system-ui",
        // iOS / macOS の San Francisco
        "-apple-system",
        // 最終フォールバック（必ず使える総称名）
        "sans-serif",
      ],
      // 等幅フォント（コード表示用）
      mono: [
        // プログラミング用の美しい等幅フォント
        "JetBrains Mono",
        // リガチャ（合字）対応の等幅フォント
        "Fira Code",
        "ui-monospace",
        "monospace",
      ],
    },
```

```
      // ========================================
      // カスタムフォントサイズ
      // ========================================
      // text-2xs クラスを新規追加（標準には無い超小サイズ）
      // 値の形式: [フォントサイズ, { lineHeight: 行間 }]
      fontSize: {
        // 10px / 行間14px
        "2xs": ["0.625rem", { lineHeight: "0.875rem" }],
      },

      // ========================================
      // カスタムシャドウ
      // ========================================
      boxShadow: {
        // shadow-sm が CSS 変数を参照するように
        sm: "var(--shadow-sm)",
        md: "var(--shadow-md)",
        lg: "var(--shadow-lg)",
        // 独自の影を新規追加
        // カード用
        card: "0 2px 8px -2px rgb(0 0 0 / 0.08), 0 4px 12px -4px rgb(0 0 0 / 0.04)",
        "card-hover":
          // ホバー時用
          "0 8px 24px -4px rgb(0 0 0 / 0.12), 0 4px 8px -4px rgb(0 0 0 / 0.08)",
      },

      // ========================================
      // カスタムボーダー半径
      // ========================================
      // rounded-sm / rounded-md などの値をCSS変数経由に置き換え
      borderRadius: {
        sm: "var(--radius-sm)",
        md: "var(--radius-md)",
        lg: "var(--radius-lg)",
        xl: "var(--radius-xl)",
      },

      // ========================================
      // カスタムアニメーション
      // ========================================
      // keyframes: 動きの定義（CSS の @keyframes 相当）
      keyframes: {
        "fade-in": {
          // 透明＋8px下
          from: { opacity: "0", transform: "translateY(8px)" },
          // 不透明＋元位置
          to: { opacity: "1", transform: "translateY(0)" },
        },
        "fade-out": {
          from: { opacity: "1", transform: "translateY(0)" },
          to: { opacity: "0", transform: "translateY(8px)" },
        },
        "slide-in-right": {
```

```
      // 16px 右からスライド
      from: { opacity: "0", transform: "translateX(16px)" },
      to: { opacity: "1", transform: "translateX(0)" },
    },
    "slide-in-up": {
      // 16px 下からスライド
      from: { opacity: "0", transform: "translateY(16px)" },
      to: { opacity: "1", transform: "translateY(0)" },
    },
    "scale-in": {
      // 95% から
      from: { opacity: "0", transform: "scale(0.95)" },
      // 100% へ拡大
      to: { opacity: "1", transform: "scale(1)" },
    },
    shimmer: {
      "0%": { backgroundPosition: "-200% 0" },
      "100%": { backgroundPosition: "200% 0" },
    },
    "toast-slide-in": {
      from: { opacity: "0", transform: "translateX(100%)" },
      to: { opacity: "1", transform: "translateX(0)" },
    },
    "toast-slide-out": {
      from: { opacity: "1", transform: "translateX(0)" },
      to: { opacity: "0", transform: "translateX(100%)" },
    },
  },
  // animation: keyframes を「いつ・どんな速度で」再生するかを定義
  // 形式: "<keyframes名> <時間> <イージング> <繰り返し> <塗り潰し>"
  animation: {
    // animate-fade-in クラスとして使える
    "fade-in": "fade-in 0.3s ease-out",
    "fade-out": "fade-out 0.3s ease-out",
    "slide-in-right": "slide-in-right 0.3s ease-out",
    "slide-in-up": "slide-in-up 0.4s ease-out",
    "scale-in": "scale-in 0.2s ease-out",
    // infinite: 無限ループ、linear: 等速
    shimmer: "shimmer 2s infinite linear",
    "toast-in": "toast-slide-in 0.3s ease-out",
    // forwards: 終了状態をキープ
    "toast-out": "toast-slide-out 0.3s ease-in forwards",
    // animate-spin（標準）の上書き
    spin: "spin 1s linear infinite",
  },

  // ========================================
  // カスタムスペーシング
  // ========================================
  // 標準にない余白/サイズの値を追加。w-18, h-88, p-128 のように使える
  spacing: {
    // 72px
    "18": "4.5rem",
```

```
        // 352px
        "88": "22rem",
        // 512px
        "128": "32rem",
      },

      // ======================================
      // カスタムトランジション
      // ======================================
      // duration-250 / duration-350 を使えるようにする
      transitionDuration: {
        "250": "250ms",
        "350": "350ms",
      },
    },
  },

  // plugins: Tailwind の機能を拡張するプラグインを並べる配列
  // 例: @tailwindcss/forms、@tailwindcss/typography など
  // 今回は使わないので空配列
  plugins: [],
};

// ESM 形式でデフォルトエクスポート（Next.js が require できるようにする）
export default config;
```

設定のポイント解説:

- `darkMode: "class"` -- `<html class="dark">` でダークモードを切り替える方式。`media` にするとシステム設定に連動する
- `container` -- レスポンシブなコンテナ幅を画面サイズごとにカスタマイズ
- `colors` -- CSS カスタムプロパティ経由でカラーを定義することで、ダークモード切り替えが CSS 変数の上書きだけで済む
- `keyframes` / `animation` -- globals.css の `@keyframes` と合わせて、Tailwind のクラスとしてアニメーションを呼び出せるようにする

▼ コードを1つずつ分解して解説

この設定ファイルは「Tailwind に対する取扱説明書」です。塊ごとに、どんな設定をしているかを見ていきましょう。

解説1: ダークモードの方式を選ぶ（`darkMode`）

コピー

```
darkMode: "class",
```

- `darkMode` は「ダークモードをどう切り替えるか」を決める設定です。

- `"class"` を選ぶと、`<html class="dark">` のように**クラスを手動で付け外し**して切り替えます。ユーザーが切替ボタンで選べるようにしたいので、この方式にしています。
- もう一つの `"media"` を選ぶと、OSの設定（ダーク/ライト）に自動で連動しますが、ユーザーが自分で選べなくなります。

> **用語: ダークモードの class 方式** … `<html>` に `dark` クラスが付いているかどうかでテーマを切り替える方式。JSからクラスを付け外しして制御する。

解説2: クラスを探す対象ファイルを指定（`content`）

コピー

```
content: [
  // Pages Router 用（使っていなくてもOK）
  "./pages/**/*.{js,ts,jsx,tsx,mdx}",
  // 自作コンポーネント
  "./components/**/*.{js,ts,jsx,tsx,mdx}",
  // App Router 用
  "./app/**/*.{js,ts,jsx,tsx,mdx}",
],
```

- `content` は「**どのファイルからTailwindクラスを探すか**」のリストです。
- Tailwind はここに書かれたファイルを読み、**実際に使われているクラスだけ**を最終的なCSSに含めます。使っていないクラスを除くことで、出力サイズを小さくできます。
- `**` は「どんな深さのフォルダでも」、`{js,ts,jsx,tsx,mdx}` は「これらの拡張子すべて」という意味の記号です。

> **用語: バンドルサイズ／パージ（purge）** … 最終的にユーザーへ配信されるCSS/JSの容量のこと。未使用クラスを削る（パージする）と軽くなり、表示が速くなる。

解説3: CSS変数を Tailwind の色として登録（`extend.colors`）

コピー

```
extend: {
  colors: {
    primary: {
      500: "var(--color-primary-500)",
      600: "var(--color-primary-600)",
      // ...
    },
  },
},
```

- `extend`（エクステンド＝拡張）は「標準のテーマを**消さずに追加する**」ための場所です。`extend` を使わずに書くと標準の色などが全部消えてしまうので注意します。
- `primary` の各段階を `var(--color-primary-500)` のように **globals.css のCSS変数に紐付け**ています。こうすると `bg-primary-500` のようなクラスが使えるようになります。
- さらに変数経由にしておくことで、ダークモード時は変数の値を入れ替えるだけで `bg-primary-500` の色も自動で変わります。

> **用語: extend** … Tailwind の設定で「標準設定を上書きせず、追加だけする」キーワード。色・余白・影などを安全に足せる。

解説4: アニメーションをクラス化する（`keyframes` / `animation`）

コピー

```
keyframes: {
  "fade-in": {
    from: { opacity: "0", transform: "translateY(8px)" },
    to: { opacity: "1", transform: "translateY(0)" },
  },
},
animation: {
  "fade-in": "fade-in 0.3s ease-out",
  shimmer: "shimmer 2s infinite linear",
},
```

- `keyframes` は「動きのコマ」、`animation` は「そのコマを**どんな速度・回数で再生するか**」をまとめた設定です。
- `animation` に登録すると、`animate-fade-in` や `animate-shimmer` のように**クラス名としてアニメーションを呼び出せる**ようになります。
- `"fade-in 0.3s ease-out"` は「fade-in を0.3秒、ease-out（最後ゆっくり）で再生」、`"shimmer 2s infinite linear"` は「2秒・無限ループ・等速」という意味です。

> **用語: イージング（easing）** … アニメーションの速度変化のカーブ。`ease-out` は登場向き（パッと出てフワッと止まる）、`linear` は等速、`infinite` は無限ループの指定。

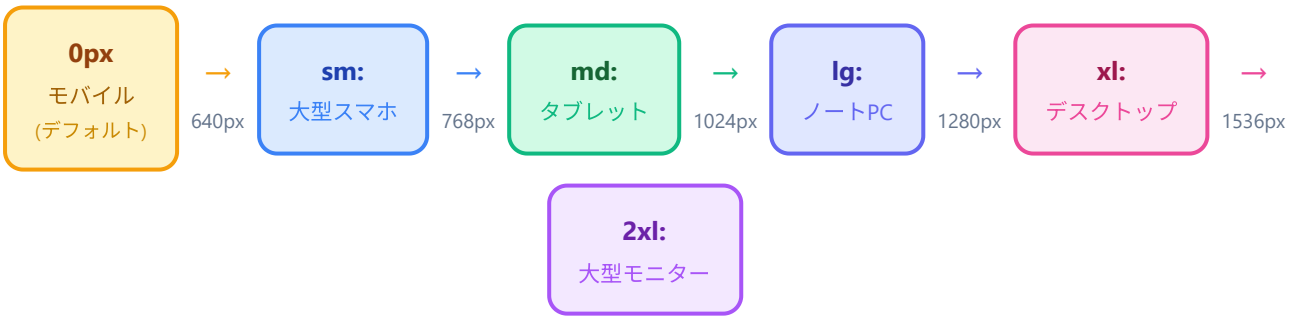
---

2. レスポンシブデザイン

2.1 Tailwind のブレークポイント解説

Tailwind CSS はモバイルファースト設計です。何もプレフィックスを付けないスタイルがモバイル（最小画面）に適用され、`sm:` `md:` などのプレフィックスを付けるとそのブレークポイント**以上**の画面幅で適用されます。

プレフィックス	最小幅	CSS 相当	想定デバイス	用途
(なし)	0px	`@media (min-width: 0px)`	小型スマートフォン	デフォルト。全画面で適用
`sm:`	640px	`@media (min-width: 640px)`	大型スマートフォン（横持ち）	テキストサイズ微調整、パディング拡大
`md:`	768px	`@media (min-width: 768px)`	タブレット	2カラムレイアウト、サイドバー表示
`lg:`	1024px	`@media (min-width: 1024px)`	ノートPC	3カラムレイアウト、ナビゲーション展開
`xl:`	1280px	`@media (min-width: 1280px)`	デスクトップ	4カラムレイアウト、広いマージン
`2xl:`	1536px	`@media (min-width: 1536px)`	大型モニター	最大幅レイアウト



重要な考え方: モバイルファースト

コピー

```
/* 悪い例: デスクトップから書いて、小さい画面を上書き */
/* Tailwind は「min-width」基準（その幅以上で適用）なので、 */
/* sm: より md: のほうが優先される。小さい画面向けに後から指定しても効かない。 */
className="grid-cols-4 md:grid-cols-2 sm:grid-cols-1"  ← 動かない

/* 良い例: モバイルから書いて、大きい画面を上書き */
/* デフォルト（接頭辞なし）= スマホ向け / md: = タブレット向け / lg: = PC向け の順 */
/* 画面が大きくなるほど右側の指定が優先される */
className="grid-cols-1 md:grid-cols-2 lg:grid-cols-4"  ← 正しい
```

モバイルファースト原則の理由： 1. モバイルユーザーが多数派の時代であること。2. CSSのカスケード（後優先）の仕組みと相性が良いこと。「最小画面 → 上書き」が自然な流れになる。3. パフォーマンス的にも、モバイルでは多くのスタイルがスキップできて軽くなる。

「なぜ小さい画面から書くと上手くいくのか」を、画面の幅を**物差し**にして描くと次のようになります。それぞれのクラスは「その幅**から右側ずっと**」効き続け、重なったところでは**下の行（より大きい画面向け）が勝ちます**。

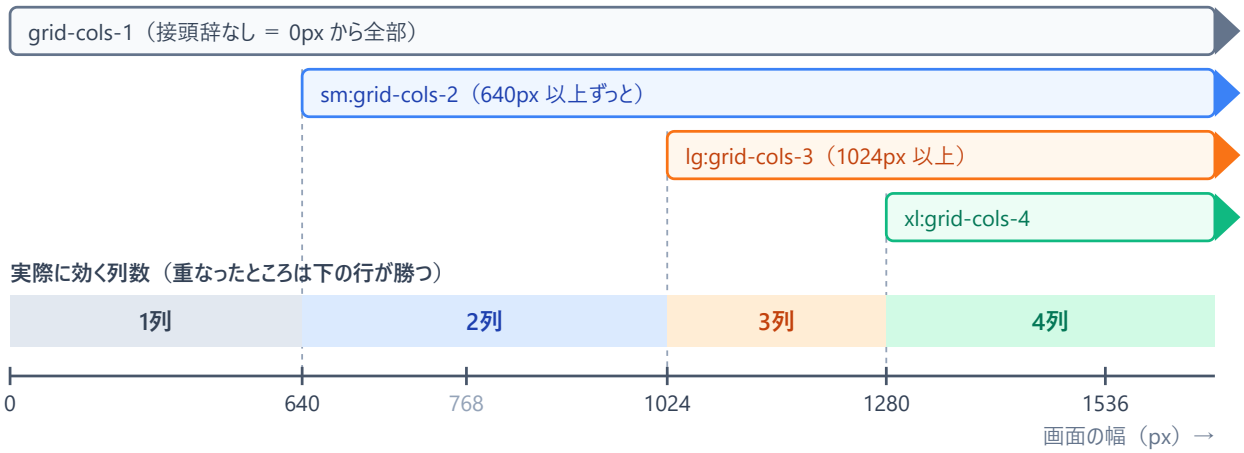


図9-12: モバイルファーストの物差し（BookGrid の指定 grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4）

試してみよう: 画面幅ごとに「勝つクラス」を計算する

▼ このコードがやること（先に日本語で）： `grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4` のようなクラスの並びを受け取り、「画面の幅が○px のとき、最終的にどのクラスが効くか」を計算して表示します。ブラウザが CSS に対して行っている判断（図9-12）を、小さなプログラムでまねしてみるのが目的です。上の「悪い例」（大きい画面から書いた指定）も並べて、なぜ動かないのかを数字で確かめます。

コピー

```
// breakpoints.ts
// Tailwind の初期ブレークポイント（この幅「以上」で効く）
const breakpoints: Record<string, number> = {
  sm: 640,
  md: 768,
  lg: 1024,
  xl: 1280,
};

// className の中から「その画面幅で最終的に効くクラス」を探す関数
function activeClass(className: string, width: number): string {
  let winner = "";
  let winnerMin = -1;
  // スペースで区切って、クラスを1つずつ調べる
  for (const token of className.split(" ")) {
    // "lg:grid-cols-3" → prefix = "lg", name = "grid-cols-3"
    const [prefix, name] = token.includes(":")
      ? token.split(":")
      : ["", token];
    // 接頭辞なしは 0px 以上＝いつでも効く
    const minWidth = prefix === "" ? 0 : breakpoints[prefix];
    // 条件を満たすもののうち「より大きい画面向け」が勝つ
    //  （Tailwind は sm → md → lg → xl の順に CSS を出力するため）
    if (width >= minWidth && minWidth > winnerMin) {
      winner = name;
      winnerMin = minWidth;
    }
  }
  return winner;
}

// OK: 本章の BookGrid と同じ「小さい画面から順に足す」書き方
const good = "grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 xl:grid-cols-4";
// NG: 「大きい画面から書いて、小さい画面で減らそう」とした書き方
const bad = "grid-cols-4 md:grid-cols-2 sm:grid-cols-1";

for (const width of [375, 640, 900, 1440]) {
  const label = String(width).padStart(4, " ");
  console.log(
    `${label}px  OK→ ${activeClass(good, width).padEnd(11)}  NG→ ${activeClass(bad, width)}`,
  );
}
```

▼ **実行結果**（`node breakpoints.ts` で実行）

コピー

```
 375px  OK→ grid-cols-1  NG→ grid-cols-4
 640px  OK→ grid-cols-2  NG→ grid-cols-1
 900px  OK→ grid-cols-2  NG→ grid-cols-2
1440px  OK→ grid-cols-4  NG→ grid-cols-2
```

▼ コードを1つずつ分解して解説

解説1: ブレークポイントの表を作る

コピー

```
const breakpoints: Record<string, number> = {
  sm: 640,
  md: 768,
  lg: 1024,
  xl: 1280,
};
```

- 2.1 節の表と同じ「接頭辞 → 何px 以上で効くか」の対応表です。`Record<string, number>` は「キーが文字列で、値が数値のオブジェクト」という型です。
- `breakpoints["lg"]` のように書くと `1024` が取り出せます。

解説2: クラスを「接頭辞」と「名前」に分ける

コピー

```
const [prefix, name] = token.includes(":")
  ? token.split(":")
  : ["", token];
```

- `token.includes(":")` で「`:` が入っているか」を調べます。入っていれば `split(":")` で `"lg:grid-cols-3"` を `["lg", "grid-cols-3"]` に分けます。
- 入っていなければ接頭辞なしなので、`["", token]` として扱います。左側の `const [prefix, name]` は、配列の1つ目と2つ目に名前を付けて取り出す**分割代入**です。

解説3: 「より大きい画面向け」が勝つ

コピー

```
if (width >= minWidth && minWidth > winnerMin) {
  winner = name;
  winnerMin = minWidth;
}
```

- `width >= minWidth` は「今の画面幅で、このクラスの条件を満たしているか」。`minWidth > winnerMin` は「今までの候補より、大きい画面向けのクラスか」です。両方を満たすと勝者が入れ替わります。
- Tailwind は CSS を「接頭辞なし → `sm:` → `md:` → `lg:` → `xl:` 」の順に書き出すので、条件を満たすクラスが複数あると**後ろ（大きい画面向け）が勝ちます**。このプログラムはその順番をまねしています。
- 実行結果の NG 列を見てください。375px のスマホで **4列**（`grid-cols-4` しか条件を満たさない）、1440px の PC で **2列**（`md:grid-cols-2` が `sm:grid-cols-1` より強い）になっています。大きい画面から書くと、狙いと正反対の結果になるのです。

> **ヒント:** Tailwind v4 が作る CSS では、`sm:` は `@media (width >= 40rem)` という書き方になります（40rem = 640px）。2.1 節の表にある `@media (min-width: 640px)` と意味は同じで、新しい書き方になっただけです。

もう1つ、接頭辞の読み方でとても多い勘違いがあります。

> **つまずきポイント: `sm:` は「スマホ用」ではなく「640px 以上用」**
>
> `sm` は small の略なので「スマホのときだけ」と読みたくなりますが、正しくは「**640px 以上の画面で**」という意味です。スマホ（多くは 640px 未満）に効かせたいスタイルは、接頭辞を**付けずに**書きます。

コピー

```
<!-- NG: 「スマホ（sm）では隠す」つもりで sm:hidden と書いた -->
<!-- → 実際は「640px 以上で隠す」なので、スマホでは見え、PC で消える -->
<nav class="sm:hidden">メニュー</nav>
```

コピー

```
<!-- OK: まず全画面で隠し（hidden）、640px 以上で表示（sm:flex）に戻す -->
<!-- → スマホでは隠れ、PC では横並びで表示される -->
<nav class="hidden sm:flex">メニュー</nav>
```



図9-13: 画面イメージ — sm:hidden（左）は狙いと正反対になる。スマホだけ隠すなら hidden sm:flex（右）

覚え方は「**接頭辞なし＝スマホを含む全部、`sm:` ＝ちょっと広い画面から上**」です。逆に「スマホのときだけ表示したい（ハンバーガーボタンなど）」場合は、`md:hidden`（768px 以上で隠す）のように書きます。6.1 節のヘッダーでも、PC 用のナビは `hidden md:flex`、ハンバーガーボタンは `md:hidden` という組み合わせを使っています。

2.2 書籍カードのグリッドをレスポンシブ対応

▼ **このコードがやること（先に日本語で）**：書籍カードを「画面サイズに応じて列数が変わる格子（グリッド）」に並べるコンポーネントを作ります。カギは Tailwind の「レスポンシブ接頭辞」で、`grid-cols-1`（スマホは1列）に `sm:grid-cols-2`（少し広い画面は2列）などを並べるだけで、面倒なメディアクエリを書かずに列数を切り替えられます。なお書籍が0冊のときは専用の空状態を表示する分岐も入っています。

コピー

```
// ============================================================================
// ファイルパス: components/BookGrid.tsx
// 役割      : 書籍カードを「画面サイズに応じて列数が変わる格子状」に並べる
// ----------------------------------------------------------------------------
// Tailwind CSS のレスポンシブ接頭辞を使うことで、メディアクエリを書かずに
// 「PCでは4列、タブレットでは3列、スマホでは1列」のような切り替えができる。
// ============================================================================

"use client";

import Link from "next/link";
import { Book } from "@/types/book";
import { BookCard } from "./BookCard";
import { EmptyState } from "./EmptyState";

type BookGridProps = {
  // 表示する書籍配列
  books: Book[];
};

export function BookGrid({ books }: BookGridProps) {
  // (1) 0冊なら専用の空状態コンポーネントを出して終わる（早期リターン）
  if (books.length === 0) {
    return <EmptyState />;
  }

  // (2) 1冊以上なら格子レイアウトで描画
  return (
    <div
      // ▼ Tailwind クラスの読み解き ▼
      //   grid           : display: grid（CSSグリッドレイアウト）
      //   grid-cols-1    : 列数 = 1（モバイル既定）
      //   gap-4          : マス目同士の隙間 16px
      //   sm:grid-cols-2 : 640px以上で 2 列に切替
      //   sm:gap-5       : 640px以上で 隙間 20px
      //   lg:grid-cols-3 : 1024px以上で 3 列
      //   lg:gap-6       : 1024px以上で 隙間 24px
      //   xl:grid-cols-4 : 1280px以上で 4 列
      // 「sm:」「lg:」などはメディアクエリを意味する接頭辞。
      className="
        grid
        grid-cols-1
        gap-4
        sm:grid-cols-2
        sm:gap-5
        lg:grid-cols-3
        lg:gap-6
        xl:grid-cols-4
```

```
      "
    >
      {/*
        (3) 配列を map で1件ずつ <Link> + <BookCard> に展開する
             ・key={book.id}     : Reactのリスト識別キー（必須）
             ・href={...}        : クリックで詳細ページへ
             ・className="group" : 子要素の "group-hover:..." を有効化する目印
             ・animationDelay    : i番目のカードを i*50ms 遅らせて順次フェードイン
      */}
      {books.map((book, index) => (
        <Link
          key={book.id}
          href={`/books/${book.id}`}
          className="group block"
          style={{
            animationDelay: `${index * 50}ms`,
          }}
        >
          <BookCard book={book} />
        </Link>
      ))}
    </div>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: 0冊のときは早期リターン

コピー

```
if (books.length === 0) {
  return <EmptyState />;
}
```

- `books.length === 0` は「書籍の配列が空（0冊）か」を判定しています。`.length` は配列の要素数です。
- 0冊なら、グリッドを描かずに `<EmptyState />`（「まだ書籍がありません」の案内）を返して**関数をここで終わらせます**。
- この「条件を満たしたら途中で `return` して抜ける」書き方を**早期リターン**と呼びます。先に例外ケースを片付けると、後の本処理がすっきりします。

> **用語: 早期リターン（early return）** … 条件を満たしたときに関数の途中で `return` して処理を打ち切る書き方。ネスト（入れ子）が浅くなり読みやすくなる。

解説2: レスポンシブな格子レイアウト（grid 系クラス）

コピー

```
className="
  grid
  grid-cols-1
  gap-4
  sm:grid-cols-2
  sm:gap-5
  lg:grid-cols-3
  lg:gap-6
  xl:grid-cols-4
"
```

- `grid` は「CSSグリッド（格子）レイアウトにする」指定です。`grid-cols-1` は列数を1にする、`gap-4` はマス目同士の隙間を16pxにする指定です。
- `sm:` `lg:` `xl:` は**レスポンシブ接頭辞**で、その画面幅以上のときだけ適用されます。`sm:grid-cols-2`（640px以上で2列）→ `lg:grid-cols-3`（1024px以上で3列）→ `xl:grid-cols-4`（1280px以上で4列）と段階的に増えます。
- 接頭辞なしの `grid-cols-1` が一番小さい画面（スマホ）向け。画面が広がるほど右側の指定で上書きされます（モバイルファースト）。

> **用語: レスポンシブ接頭辞（responsive prefix）** … `sm:` `md:` `lg:` のように、画面幅に応じてスタイルを切り替えるためにクラスの先頭に付ける目印。メディアクエリを手書きせずに済む。

解説3: 各カードを順番にフェードインさせる（`animationDelay`）

コピー

```
{books.map((book, index) => (
  <Link
    key={book.id}
    href={`/books/${book.id}`}
    className="group block"
    style={{
      animationDelay: `${index * 50}ms`,
    }}
  >
    <BookCard book={book} />
  </Link>
))}
```

- `books.map((book, index) => ...)` で配列の各書籍を `<Link>` + `<BookCard>` に変換しています。`index` は0から始まる「何番目か」の番号です。

- `key={book.id}` はReactがリスト要素を見分けるための必須の目印。データ固有のIDを使うのが鉄則です。
- `className="group"` は「子要素の `group-hover:...` を効かせるための目印」。`animationDelay: ${index * 50}ms` は「i番目のカードを `i×50` ミリ秒遅らせて登場させる」指定で、カードが**ぱらぱらと順番に**現れる演出になります。

> **用語: animationDelay（アニメーション遅延）** … アニメーション開始までの待ち時間。各要素に少しずつ違う遅延を与えると、一斉ではなく順番に動く「スタッガード（staggered）」な演出になる。

> **▼ このコードがやること（先に日本語で）**：書籍1冊分の見た目（タイトル・著者・状態バッジなど）を表示する「カード」コンポーネントを作ります。先ほど globals.css で用意した `.card` や `.badge` といった共通スタイルのクラスを当てて、見た目を整えるのがポイントです。このカードが先ほどのグリッドの中に1セルずつ並びます。各部分の役割はコメントで説明しています。

なぞってコピーすると「…」が別の文字になり、長い行も分かれます。正しくは → コピー

```
// components/BookCard.tsx
// ↑ 書籍1冊分の見た目を表示するカードコンポーネント。
//   一覧画面（BookGrid）の中で1セルずつ並ぶ部品。

// "use client" は Next.js App Router で「これはクライアントコンポーネント」と宣言するディレクティブ
// useState / useEffect / onClick などを使うなら必要
"use client";

// 書籍データの型を import（@/ はプロジェクトルートを指すパスエイリアス）
import { Book } from "@/types/book";

// このコンポーネントが受け取る props（外から渡される値）の型を定義
type BookCardProps = {
  // Book 型の書籍データを 1 件
  book: Book;
};

// export function: 名前付きエクスポート（他ファイルから { BookCard } で取り出す）
// 引数の { book } は分割代入：props の book プロパティを直接取り出している
export function BookCard({ book }: BookCardProps) {
  // 読了状況（status）に応じてバッジのスタイルを切り替えるヘルパー関数
  // ※ 関数のなかでJSXを return して、 {statusBadge()} で呼び出す
  const statusBadge = () => {
    switch (book.status) {
      case "reading":
        // badge-primary はインディゴ色のバッジ（globals.css で定義）
        return <span className="badge-primary">読書中</span>;
      case "completed":
        // badge-success は緑色のバッジ
        return <span className="badge-success">読了</span>;
      case "want_to_read":
        // badge-warning は黄色のバッジ
        return <span className="badge-warning">読みたい</span>;
      default:
        // 該当ステータスがなければ何も表示しない
        return null;
    }
  };

  return (
    // <article> は「独立した記事的なコンテンツ」を表すHTML5 タグ
    // 書籍1冊分の情報のかたまりとして意味的に正しい
    <article
      className="
        card-hover
        animate-fade-in
        flex flex-col
        overflow-hidden
        p-0
      "
      /* ▼ クラスの読み解き ▼
         card-hover     : .card（カードのベース）にホバー時の浮き上がりを追加
         animate-fade-in: 表示時に下からフェードインするアニメーション
         flex flex-col  : flex レイアウトで子要素を縦に並べる
         overflow-hidden: はみ出た要素を非表示（画像のはみ出し対策）
         p-0            : padding を 0 に（card のデフォルトpadding解除）
      */
    >
      {/* 書籍のサムネイルエリア */}
      <div
        className="
          relative
```

```
          aspect-[3/4]
          w-full
          overflow-hidden
          bg-gradient-to-br from-primary-100 to-primary-200
          dark:from-primary-900 dark:to-primary-800
        "
        /* ▼ クラスの読み解き ▼
           relative      : position: relative。子要素の absolute 配置の基準にする
           aspect-[3/4]  : 縦横比 3:4（本の縦長プロポーション）。任意値構文 [ ]
           w-full        : 幅100%
           overflow-hidden: はみ出し非表示（hover で画像がズームしてもカードからはみ出ない）
           bg-gradient-to-br: 背景を「左上 → 右下」のグラデーションに（br = bottom-right）
           from-primary-100 to-primary-200 : 開始色と終了色
           dark:from-primary-900 dark:to-primary-800: ダークモード時のグラデーション
        */
      >
        {/* book.thumbnailUrl があれば画像を表示、なければプレースホルダーを表示する分岐 */}
        {/* JSX 内の {} は JavaScript 式を埋め込む構文。三項演算子（条件 ? A : B）で出し分け */}
        {book.thumbnailUrl ? (
          <img
            // 画像URL
            src={book.thumbnailUrl}
            // 代替テキスト（読み上げ・画像非表示時用）
            alt={`「${book.title}」の表紙`}
            className="
              h-full w-full
              object-cover
              transition-transform duration-300
              group-hover:scale-105
            "
            /* ▼ クラスの読み解き ▼
               h-full w-full          : 親要素いっぱいに広げる
               object-cover           : 縦横比を保ちながら親要素を完全に覆う（はみ出る部分はカット）
               transition-transform   : transform プロパティに transition を適用
               duration-300           : 300ms かけて変化
               group-hover:scale-105  : 親要素のうち className="group" を持つもののホバー時に
                                        この要素を 105% に拡大。BookGrid 側の <Link> に group を付けている
            */
            // 画面外の画像は遅延読み込み（パフォーマンス向上）
            loading="lazy"
          />
        ) : (
          /* プレースホルダー（画像がない場合の代替表示） */
          <div
            className="
              flex h-full w-full
              flex-col items-center justify-center
              gap-2
              p-4
              text-primary-400
              dark:text-primary-600
            "
            /* ▼ クラスの読み解き ▼
               flex h-full w-full           : flex レイアウトで親全体に広げる
               flex-col                     : 縦並び
               items-center justify-center  : 縦横とも中央寄せ
               gap-2                        : 子要素間に 0.5rem（8px）の隙間
               p-4                          : 内側余白 1rem（16px）
               text-primary-400             : アイコン色（薄めのインディゴ）
               dark:text-primary-600        : ダーク時の色
            */
          >
            {/* SVG（Scalable Vector Graphics：拡大しても綺麗な画像形式）でアイコンを描く */}
            <svg
              // 48x48 サイズ
```

```
              className="h-12 w-12"
              // 塗りつぶしなし
              fill="none"
              // SVG の内部座標系（24x24 のキャンバス）
              viewBox="0 0 24 24"
              // 線の色は親要素の文字色（text-primary-400）を継承
              stroke="currentColor"
              // 線の太さ
              strokeWidth={1.5}
              // スクリーンリーダーに無視させる（装飾アイコンのため）
              aria-hidden="true"
            >
              <path
                // 線の端を丸く
                strokeLinecap="round"
                // 線の折れ角を丸く
                strokeLinejoin="round"
                d="M12 6.042A8.967 8.967 0 006 3.75c-1.052 0-2.062.18-3 .512v14.25A8.987 8.987 0 016
18c2.305 0 4.408.867 6 2.292m0-14.25a8.966 8.966 0 016-2.292c1.052 0 2.062.18 3 .512v14.25A8.987 8.987 0
0018 18a8.967 8.967 0 00-6 2.292m0-14.25v14.25"
                /* ↑ d 属性は SVG のパス（描画命令）。M=移動、A=円弧、C=曲線など */
              />
            </svg>
            <span className="text-xs font-medium">No Image</span>
            {/*  text-xs: 12px / font-medium: 500 の太さ */}
          </div>
        )}

        {/* ステータスバッジ（カード右上に絶対配置） */}
        <div className="absolute right-2 top-2">
          {/* absolute        : position: absolute。親 .relative を基準に配置
              right-2 top-2  : 右から 0.5rem、上から 0.5rem の位置 */}
          {statusBadge()}
        </div>
      </div>

      {/* 書籍情報エリア */}
      <div className="flex flex-1 flex-col gap-1.5 p-4">
        {/* ▼ クラスの読み解き ▼
            flex flex-1 : flex の子で、残り領域を全部使う（伸縮自在）
            flex-col    : 子要素を縦並びに
            gap-1.5     : 子要素間に 0.375rem（6px）の隙間
            p-4         : 内側余白 1rem（16px）*/}

        {/* タイトル */}
        <h3
          className="
            line-clamp-2
            text-sm font-semibold
            leading-tight
            text-[var(--color-foreground)]
            transition-colors duration-200
            group-hover:text-primary-600
            dark:group-hover:text-primary-400
            sm:text-base
          "
          /* ▼ クラスの読み解き ▼
             line-clamp-2                    : 2 行を超えたら省略「…」（Tailwind 標準のクラス）
             text-sm font-semibold           : 14px の太字
             leading-tight                   : 行間を狭め（line-height: 1.25）
             text-[var(--color-foreground)]  : 文字色をCSS変数で指定（ダーク対応のため）
             transition-colors duration-200  : 色変化を 200ms かけて滑らかに
             group-hover:text-primary-600    : 親（.group）ホバー時にインディゴへ変色
             dark:group-hover:text-primary-400: ダーク × ホバー時の色
             sm:text-base                    : 640px 以上で 16px に拡大
```

```
          */
        >
          {book.title}
        </h3>

        {/* 著者 */}
        <p
          className="
            line-clamp-1
            text-xs text-[var(--color-muted)]
            sm:text-sm
          "
          /* line-clamp-1: 1行で省略
             text-xs (12px) → sm:text-sm (14px に拡大)
             text-[var(--color-muted)]: 補足テキスト用の薄めの色 */
        >
          {book.author}
        </p>

        {/* 評価（星表示） */}
        {/* 条件付きレンダリング: rating が定義済みで 0 より大きい時のみ描画 */}
        {/* && は短絡評価。左が true なら右の JSX を返す、false なら何も返さない */}
        {book.rating !== undefined && book.rating > 0 && (
          <div className="mt-auto flex items-center gap-1 pt-2">
            {/* mt-auto: 上方向の margin を auto に → flex 子要素を一番下に押し下げる
                flex items-center: 横並びで縦中央
                gap-1: 子要素間 0.25rem (4px)
                pt-2: 上パディング 0.5rem (8px) */}

            {/* [1,2,3,4,5] の配列を map で星アイコン5つにループ展開 */}
            {[1, 2, 3, 4, 5].map((star) => (
              <svg
                // React リストの一意キー（必須）
                key={star}
                /* テンプレートリテラル ` ` の中で ${} を使い動的にクラスを切替 */
                /* star が現在の評価以下なら金色（塗る）、それより大きいなら灰色（空き） */
                className={`h-3.5 w-3.5 ${
                  // ! は「null/undefined でない」と保証する非 null アサーション
                  star <= book.rating!
                    ? "text-yellow-400"
                    : "text-gray-300 dark:text-gray-600"
                }`}
                // 塗り色＝親要素の文字色
                fill="currentColor"
                viewBox="0 0 20 20"
                // スクリーンリーダーから隠す（後ろの数値で代替）
                aria-hidden="true"
              >
                {/* 星型のパス */}
                <path d="M9.049 2.927c.3-.921 1.603-.921 1.902 0l1.07 3.292a1 1 0 00.95.69h3.462c.969 0
1.371 1.24.588 1.81l-2.8 2.034a1 1 0 00-.364 1.118l1.07 3.292c.3.921-.755 1.688-1.54 1.118l-2.8-2.034a1 1 0
00-1.175 0l-2.8 2.034c-.784.57-1.838-.197-1.539-1.118l1.07-3.292a1 1 0 00-.364-1.118L2.98
8.72c-.783-.57-.38-1.81.588-1.81h3.461a1 1 0 00.951-.69l1.07-3.292z" />
              </svg>
            ))}
            {/* 評価の数値（4.0 など）も併記 */}
            <span className="ml-1 text-xs text-[var(--color-muted)]">
              {/* ml-1: 左マージン 0.25rem (4px) */}
              {book.rating}
            </span>
          </div>
        )}
      </div>
    </article>
```

```
  );
}
```

▼ コードを1つずつ分解して解説

解説1: ステータスに応じてバッジを出し分ける（`statusBadge`）

コピー

```
const statusBadge = () => {
  switch (book.status) {
    case "reading":
      return <span className="badge-primary">読書中</span>;
    case "completed":
      return <span className="badge-success">読了</span>;
    case "want_to_read":
      return <span className="badge-warning">読みたい</span>;
    default:
      return null;
  }
};
```

- `statusBadge` は「**JSXを返す小さな関数**」です。`{statusBadge()}` のように呼び出すと、状態に合ったバッジが描画されます。
- `switch (book.status)` で読了状況を分岐し、`badge-primary`（青）・`badge-success`（緑）・`badge-warning`（黄）という globals.css 定義のクラスを当て分けています。
- `default: return null` は「どれにも当てはまらなければ何も表示しない」という意味。Reactでは `null` を返すと何も描画されません。

> **用語: switch 文** … 1つの値を複数の `case` と照合して分岐する制御構文。状態（status）ごとの出し分けに向く。

解説2: 画像があるか無いかで表示を切り替える（三項演算子）

コピー

```
{book.thumbnailUrl ? (
  <img
    src={book.thumbnailUrl}
    alt={`「${book.title}」の表紙`}
    className="h-full w-full object-cover ... group-hover:scale-105"
    loading="lazy"
  />
) : (
  /* プレースホルダー（画像がない場合の代替表示） */
  <div className="flex h-full w-full ...">
    {/* No Image アイコン */}
  </div>
)}
```

- `book.thumbnailUrl ? (画像) : (代替表示)` は**三項演算子**で、「URLがあれば画像を、なければ『No Image』の代替を表示」と切り替えています。
- `object-cover` は「縦横比を保ったまま枠を覆い、はみ出る部分はカット」する指定。`group-hover:scale-105` は親（`.group`）にマウスを乗せたとき画像を105%に拡大します。
- `loading="lazy"` は「画面外の画像は後回しで読み込む」指定で、ページ表示を軽くします。`alt` は画像が出ない時や読み上げソフト用の代替文です。

> **用語: 三項演算子（`条件 ? A : B`）／object-cover** … 三項演算子は条件で値を出し分ける式。`object-cover` は画像を切り抜いて枠いっぱいに表示するCSS。

解説3: 評価の星を5つ描く（配列の map）

コピー

```
{book.rating !== undefined && book.rating > 0 && (
  <div className="mt-auto flex items-center gap-1 pt-2">
    {[1, 2, 3, 4, 5].map((star) => (
      <svg
        key={star}
        className={`h-3.5 w-3.5 ${
          star <= book.rating!
            ? "text-yellow-400"
            : "text-gray-300 dark:text-gray-600"
        }`}
        // ...
      >
        <path d="..." />
      </svg>
    ))}
  </div>
)}
```

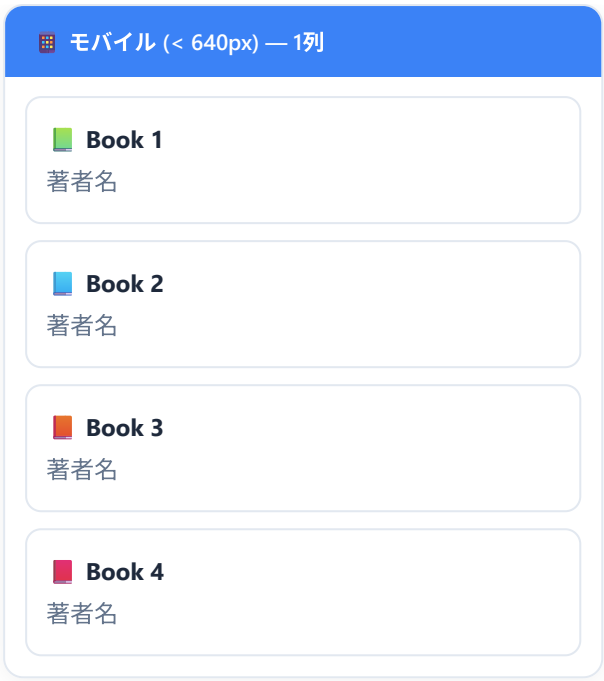
- 先頭の `book.rating !== undefined && book.rating > 0 &&` は「評価が設定されていて0より大きいときだけ」星を描く条件です（`&&` は左が成立したときだけ右を実行）。
- `[1, 2, 3, 4, 5].map((star) => ...)` で**星アイコンを5個**作ります。`star` は1〜5の番号です。
- `className={`... ${star <= book.rating! ? "text-yellow-400" : "text-gray-300 ..."}`}` で、「その星の番号が評価以下なら金色、超えていれば灰色」と色を切り替えます。`book.rating!` の `!` は「ここでは null/undefined ではない」とTypeScriptに保証する記号です。

> **用語: 非nullアサーション（`!`）／mt-auto** … 値の後ろの `!` は「絶対にnull/undefinedでない」とコンパイラに伝える印。`mt-auto` は上の余白を自動で最大化し、要素を下端に押し下げるTailwindクラス。

2.3 モバイル/タブレット/デスクトップでの表示の違い

各画面サイズでのグリッドレイアウトの変化を以下にまとめます。

モバイル（~639px）：`grid-cols-1`



- カードが1列で縦に並ぶ
- タイトルは `text-sm`（小さめ）
- パディングは最小限
- タッチ操作しやすい大きなタップ領域

タブレット（640px~1023px）：`sm:grid-cols-2`



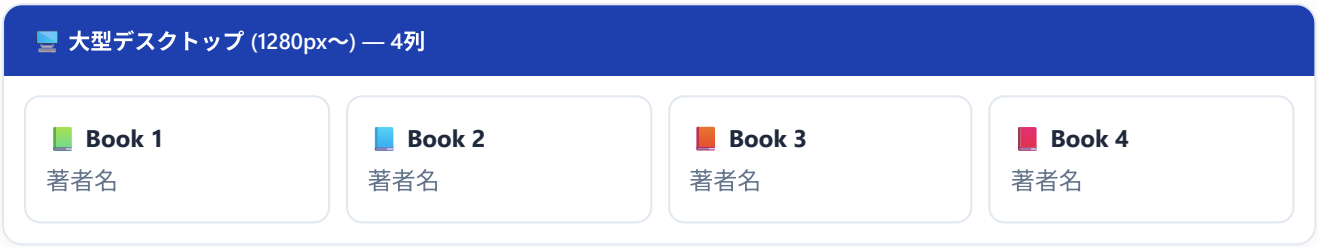
- 2列のグリッド
- タイトルは `sm:text-base`（通常サイズ）
- ギャップが少し広がる（`sm:gap-5`）

デスクトップ（1024px~1279px）：`lg:grid-cols-3`



- 3列のグリッド
- ギャップがさらに広がる（`lg:gap-6`）

大型デスクトップ（1280px~）：`xl:grid-cols-4`



- 4列のグリッド
- 情報の一覧性が最大化

---

3. UI コンポーネントの改善

3.1 トースト通知の実装

操作結果（成功/エラー）をユーザーに伝えるトースト通知コンポーネントを作成します。

React の **Context API（コンテキスト・エーピーアイ：親から孫・ひ孫…と深い階層に値を渡すための仕組み）** を使い、アプリのどこからでもトーストを呼び出せるようにします。

> **Context API のイメージ：** 通常 React は props で値を「親 → 子」と一段ずつ渡しますが、深い階層では「props のバケツリレー」が辛くなります。Context はその回避策で、「アプリのある範囲に値を放送して、その範囲内の誰でも `useContext` で受信できる」仕組みです。

3.1.1 トーストの型定義

> **▼ このコードがやること（先に日本語で）**：トースト通知（画面の隅にひょいと出る短いメッセージ）で扱うデータの「型」を先に定義します。トーストは「成功・エラー・警告・情報」の4種類だけに限定し、それ以外の文字列を間違って書けないようにします（タイポ防止）。型を別ファイルに切り出しておくと、このあと作る通知の仕組みとボタンの間でデータの形を共有しやすくなります。

コピー

```
// types/toast.ts
// ↑ トースト通知に使う型をまとめて定義するファイル。
//   型を別ファイルに切り出すと、Context・コンポーネント間で共有しやすい。

// ToastType: トーストの「種類」を表す文字列リテラル型のユニオン（| で連結した複数候補）
// この4つ以外の文字列は代入できなくなる（タイポ防止）
export type ToastType = "success" | "error" | "warning" | "info";

// Toast: 1件のトーストデータを表す型（オブジェクト型）
export type Toast = {
  // 一意な識別子（削除時に使う）
  id: string;
  // 上で定義した4種のどれか
  type: ToastType;
  // 通知のタイトル（必須）
  title: string;
  // 補足メッセージ。? は「省略可能（undefined を許容）」の意味
  message?: string;
  // 表示時間（ミリ秒）。省略時はデフォルト 5000ms = 5秒
  duration?: number;
};
```

3.1.2 トースト Context

> **▼ このコードがやること（先に日本語で）**：アプリのどこからでもトースト通知を出せるようにする「中央の仕組み」を作ります。React の Context（コンテキスト）という機能を使うと、深い階層のコンポーネントにもデータや関数を「バケツリレー」せずに直接届けられます。ここでは「トーストを追加・削除する関数」を Context に入れ、`useToast()` という短い呼び出しで使えるようにするのが狙いです。

コピー

```
// ============================================================================
// ファイルパス: src/contexts/ToastContext.tsx
// 役割　　: アプリ全体で「トースト通知（一時的なメッセージ）」を扱う仕組み
// ----------------------------------------------------------------------------
// React の Context API を使うと、深い階層のコンポーネントにも
// プロパティをバケツリレーせずに値や関数を共有できる。
// ここでは「トーストを追加・削除する関数」を Context にして、
// どのコンポーネントからでも showToast(...) のように呼べるようにする。
// ============================================================================

// "use client" が必要な理由:
//   useState / useContext / setTimeout を使う＝ブラウザ側で動く必要がある。
"use client";

// React の Hook と型を取り込む
//   createContext: Context オブジェクトを作る関数
//   useContext   : Context の値を読む Hook
//   useState     : 状態管理 Hook
//   useCallback  : 関数を「依存変更時のみ再生成」するメモ化 Hook
//   ReactNode    : 「Reactで子要素として使えるあらゆる値」の型
import {
  createContext,
  useContext,
  useState,
  useCallback,
  type ReactNode,
} from "react";
import { Toast, ToastType } from "@/types/toast";

// ----------------------------------------------------------------------------
// (1) Context が提供する「価値」の型を定義
// ----------------------------------------------------------------------------
// この型に書かれた値・関数を、Context.Provider 経由で配布する。
type ToastContextType = {
  // 現在表示中のトースト配列
  toasts: Toast[];
  // 任意のトーストを追加
  addToast: (
    type: ToastType,
    title: string,
    message?: string,
    duration?: number
  ) => void;
  // 指定IDを削除
  removeToast: (id: string) => void;
  // ↓ よく使う4種のショートカット
  success: (title: string, message?: string) => void;
  error: (title: string, message?: string) => void;
```

```
  warning: (title: string, message?: string) => void;
  info: (title: string, message?: string) => void;
};

// ---------------------------------------------------------------------------
// (2) Context オブジェクトを作る
// ---------------------------------------------------------------------------
// 初期値は undefined（後で Provider が値を提供）。
// 取り出すときに undefined チェックを入れて安全にする（後述の useToast 参照）。
const ToastContext = createContext<ToastContextType | undefined>(undefined);

// ---------------------------------------------------------------------------
// (3) 一意なIDを作る簡易関数
// ---------------------------------------------------------------------------
// Date.now()（ミリ秒）と単調増加カウンタを組み合わせて衝突しないIDを作る。
// 大規模アプリでは uuid ライブラリを使うほうが安心。
let toastIdCounter = 0;
function generateToastId(): string {
  toastIdCounter += 1;
  return `toast-${Date.now()}-${toastIdCounter}`;
}

// ---------------------------------------------------------------------------
// (4) Provider コンポーネント
// ---------------------------------------------------------------------------
// アプリのルート付近で <ToastProvider> でラップすると、その内側の全ての
// コンポーネントが useToast() で値を受け取れる。
export function ToastProvider({ children }: { children: ReactNode }) {
  // (4-1) 表示中のトースト配列を state で保持
  const [toasts, setToasts] = useState<Toast[]>([]);

  // (4-2) 指定IDのトーストを削除する関数
  //       useCallback で関数の同一性を保つことで、依存配列に入れた箇所で
  //       不必要な再レンダリングが起きるのを防ぐ。
  const removeToast = useCallback((id: string) => {
    // 配列を「削除対象以外を残す」フィルタで更新
    setToasts((prev) => prev.filter((toast) => toast.id !== id));
  }, []);

  // (4-3) トーストを追加する関数
  //       duration 経過後に setTimeout で自動で削除する。
  const addToast = useCallback(
    (
      type: ToastType,
      title: string,
      message?: string,
      // 既定で5秒で消える
      duration: number = 5000
    ) => {
      const id = generateToastId();
```

```
      const newToast: Toast = { id, type, title, message, duration };

      // 既存の配列に新トーストを末尾追加（破壊しないよう新配列を作る）
      setToasts((prev) => [...prev, newToast]);

      // duration が 0 より大きいときだけ自動削除のタイマーをセット
      if (duration > 0) {
        setTimeout(() => {
          removeToast(id);
        }, duration);
      }
    },
    // removeToast が変わったら再生成
    [removeToast]
  );

  // ショートカットメソッド
  const success = useCallback(
    (title: string, message?: string) => addToast("success", title, message),
    [addToast]
  );

  const error = useCallback(
    (title: string, message?: string) =>
      // エラーは長めに表示
      addToast("error", title, message, 8000),
    [addToast]
  );

  const warning = useCallback(
    (title: string, message?: string) => addToast("warning", title, message),
    [addToast]
  );

  const info = useCallback(
    (title: string, message?: string) => addToast("info", title, message),
    [addToast]
  );

  return (
    <ToastContext.Provider
      value={{ toasts, addToast, removeToast, success, error, warning, info }}
    >
      {children}
    </ToastContext.Provider>
  );
}

// ---------------------------------------------------------------------------
// (5) Context から値を取り出すカスタム Hook
```

```
// ---------------------------------------------------------------------------
// この関数を使えば、コンポーネントから `const toast = useToast()` のように
// 呼び出すだけで Context の値を受け取れる。
// 関数名は use で始める必要がある（React の Hook ルール）。
export function useToast(): ToastContextType {
  // useContext で Context オブジェクトから値を取得
  const context = useContext(ToastContext);
  // Provider 内で使われなかった場合は、エラーで気付かせる
  //  （取り出した値が undefined のままだと型エラーの温床になる）
  if (context === undefined) {
    throw new Error("useToast must be used within a ToastProvider");
  }
  return context;
}
```

▼ コードを1つずつ分解して解説

解説1: Context の「形」を型で決める（`ToastContextType`）

コピー

```
type ToastContextType = {
  toasts: Toast[];
  addToast: (type: ToastType, title: string, message?: string, duration?: number) => void;
  removeToast: (id: string) => void;
  success: (title: string, message?: string) => void;
  // error / warning / info も同様
};
```

- これは「Context を通じて配る中身の一覧」を表す型です。`toasts`（表示中の配列）と、追加・削除・各種ショートカット関数が含まれます。
- `addToast: (...) => void` のように書くと「**こういう引数を取り、戻り値の無い関数**」という型になります。`message?` の `?` は省略可能の意味です。
- 先に型を決めておくと、使う側で `toast.success(...)` のように呼ぶとき、VS Codeが補完や間違いチェックをしてくれます。

> **用語: Context（コンテキスト）** … propsのバケツリレーをせずに、ある範囲のどのコンポーネントへも直接値を届けるReactの仕組み。

解説2: Context オブジェクトを作る（`createContext`）

コピー

```
const ToastContext = createContext<ToastContextType | undefined>(undefined);
```

- `createContext` は「値を配るための入れ物（Context）」を作る関数です。
- 初期値を `undefined` にしているのは、「Providerで包まずに使ったら気づけるようにする」ためです（後述の `useToast` で `undefined` を検出してエラーを出します）。
- 型 `ToastContextType | undefined` は「中身は ToastContextType か、まだ無い（undefined）かのどちらか」という意味です。

> **用語: createContext** … Contextの入れ物を生成するReact関数。`.Provider` で値を流し込み、`useContext` で受け取る。

解説3: トーストを追加して自動で消す（`addToast`）

コピー

```
const addToast = useCallback(
  (type, title, message, duration = 5000) => {
    const id = generateToastId();
    const newToast: Toast = { id, type, title, message, duration };
    setToasts((prev) => [...prev, newToast]);
    if (duration > 0) {
      setTimeout(() => { removeToast(id); }, duration);
    }
  },
  [removeToast]
);
```

- `duration = 5000` は「指定が無ければ5000ミリ秒（5秒）」というデフォルト値です。
- `setToasts((prev) => [...prev, newToast])` は「**今ある配列をコピーして末尾に新トーストを足した新しい配列**」を作ってセットします。`...prev` で元を壊さずに追加するのがReactのお作法です。
- `setTimeout(() => removeToast(id), duration)` で、`duration` 経過後に**自動でそのトーストを削除**します。`useCallback` で包むのは、関数を毎回作り直さず使い回す（メモ化）ためです。

> **用語: useCallback（メモ化）／setTimeout** … `useCallback` は依存が変わらない限り同じ関数を保つReact Hook。`setTimeout` は指定ミリ秒後に1回だけ処理を実行するブラウザ関数。

解説4: Provider で値を配る（`ToastContext.Provider`）

コピー

```
return (
  <ToastContext.Provider
    value={{ toasts, addToast, removeToast, success, error, warning, info }}
  >
    {children}
  </ToastContext.Provider>
);
```

- `<ToastContext.Provider value={...}>` で囲むと、その**内側にある全コンポーネント**が `value` の中身を受け取れます。
- `value` には解説1の型に対応する一式（配列と関数群）を渡しています。
- `{children}` は「このProviderで包まれた中身」を表し、アプリ全体をここに入れることで、どこからでもトーストを呼べるようになります。

> **用語: Provider（プロバイダー）** … Contextの値を「ここから下の範囲に配る」役割のコンポーネント。`value` に配りたいデータを渡す。

解説5: 安全に値を取り出すカスタムフック（`useToast`）

コピー

```
export function useToast(): ToastContextType {
  const context = useContext(ToastContext);
  if (context === undefined) {
    throw new Error("useToast must be used within a ToastProvider");
  }
  return context;
}
```

- `useContext(ToastContext)` で Context の現在値を取り出します。
- `if (context === undefined)` は「Providerの外で誤って使った」状況の検出です。その場合 `throw new Error(...)` で**分かりやすいエラーを出して**早期に気づかせます。
- 関数名を `use` で始めるのはReactの「Hookは use で始める」というルールに従うためです。これで使う側は `const toast = useToast()` の1行で済みます。

> **用語: カスタムフック（custom hook）** … `use` で始まる自作の関数で、Hookのロジックを再利用しやすくまとめたもの。`useContext` などの組み込みHookを内部で使える。

3.1.3 トーストコンポーネント

▼ **このコードがやること（先に日本語で）**：トースト通知の「実際の見た目」を作るコンポーネントです。先ほどの Context から「今表示すべきトーストの一覧」を受け取り、種類（成功・エラーなど）に応じた色やアイコンを付けて画面の隅に並べます。一定時間が経つと自動で消える点と、右からスライドして登場するアニメーションがポイントです。各処理の詳細はコメントを参照してください。

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
// components/Toast.tsx
// ↑ トーストの実際の見た目（UIコンポーネント）を定義するファイル。
//   ToastContext からトースト配列を受け取って画面に描画する。

// クライアントコンポーネント宣言（state を使うため）
"use client";

// React Hooks
import { useState, useEffect, useCallback } from "react";
// 型を別名 ToastType で取り込む（同名のローカル定義と区別するため as でリネーム）
import { Toast as ToastType } from "@/types/toast";
// Context Hook
import { useToast } from "@/contexts/ToastContext";

// 各トーストタイプのアイコンとスタイルをまとめた設定オブジェクト
// 4種類（success/error/warning/info）ごとにアイコンと色を変える
// このように設定を1か所にまとめておくと、新しい種類を追加しやすい（DRY原則）
const toastConfig = {
  // ── 成功（チェックマークアイコン+緑色） ──
  success: {
    icon: (
      <svg
        /* 20x20 サイズ */
        className="h-5 w-5"
        fill="none"
        viewBox="0 0 24 24"
        stroke="currentColor"
        strokeWidth={2}
        aria-hidden="true"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          /* チェックマーク + 円形のパス */
          d="M9 12.75L11.25 15 15 9.75M21 12a9 9 0 11-18 0 9 9 0 0118 0z"
        />
      </svg>
    ),
    /* 各色は緑系の組み合わせ。ライトモードは薄背景+濃文字、ダークモードは逆 */
    containerClass:
      "border-green-200 bg-green-50 dark:border-green-800 dark:bg-green-950",
    iconClass: "text-green-600 dark:text-green-400",
    titleClass: "text-green-800 dark:text-green-200",
    messageClass: "text-green-700 dark:text-green-300",
    /* プログレスバーは中間色 */
    progressClass: "bg-green-500",
  },
  // ── エラー（!アイコン+赤色） ──
  error: {
    icon: (
      <svg
        className="h-5 w-5"
        fill="none"
        viewBox="0 0 24 24"
        stroke="currentColor"
        strokeWidth={2}
        aria-hidden="true"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          /* 円の中に「!」を描くパス */
```

```
          d="M12 9v3.75m9-.75a9 9 0 11-18 0 9 9 0 0118 0zm-9 3.75h.008v.008H12v-.008z"
        />
      </svg>
    ),
    containerClass:
      "border-red-200 bg-red-50 dark:border-red-800 dark:bg-red-950",
    iconClass: "text-red-600 dark:text-red-400",
    titleClass: "text-red-800 dark:text-red-200",
    messageClass: "text-red-700 dark:text-red-300",
    progressClass: "bg-red-500",
  },
  // ── 警告（三角!アイコン+黄色） ──
  warning: {
    icon: (
      <svg
        className="h-5 w-5"
        fill="none"
        viewBox="0 0 24 24"
        stroke="currentColor"
        strokeWidth={2}
        aria-hidden="true"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          /* 三角形の中に「!」（道路標識のような形） */
          d="M12 9v3.75m-9.303 3.376c-.866 1.5.217 3.374 1.948 3.374h14.71c1.73 0 2.813-1.874 1.948-
3.374L13.949 3.378c-.866-1.5-3.032-1.5-3.898 0L2.697 16.126zM12 15.75h.007v.008H12v-.008z"
        />
      </svg>
    ),
    containerClass:
      "border-yellow-200 bg-yellow-50 dark:border-yellow-800 dark:bg-yellow-950",
    iconClass: "text-yellow-600 dark:text-yellow-400",
    titleClass: "text-yellow-800 dark:text-yellow-200",
    messageClass: "text-yellow-700 dark:text-yellow-300",
    progressClass: "bg-yellow-500",
  },
  // ── 情報（iアイコン+青色） ──
  info: {
    icon: (
      <svg
        className="h-5 w-5"
        fill="none"
        viewBox="0 0 24 24"
        stroke="currentColor"
        strokeWidth={2}
        aria-hidden="true"
      >
        <path
          strokeLinecap="round"
          strokeLinejoin="round"
          /* 円の中に「i」（インフォメーション） */
          d="M11.25 11.25l.041-.02a.75.75 0 011.063.852l-.708 2.836a.75.75 0 001.063.853l.041-.021M21 12a9
9 0 11-18 0 9 9 0 0118 0zm-9-3.75h.008v.008H12V8.25z"
        />
      </svg>
    ),
    containerClass:
      "border-blue-200 bg-blue-50 dark:border-blue-800 dark:bg-blue-950",
    iconClass: "text-blue-600 dark:text-blue-400",
    titleClass: "text-blue-800 dark:text-blue-200",
    messageClass: "text-blue-700 dark:text-blue-300",
    progressClass: "bg-blue-500",
  },
```

```
};

// ---------------------------------------------------------------------------
// 個別のトーストアイテム（1件分の見た目）
// ---------------------------------------------------------------------------
function ToastItem({ toast }: { toast: ToastType }) {
  // Context から「削除関数」を取り出す
  const { removeToast } = useToast();
  // isExiting: 閉じるアニメーション中かどうか（true で出ていくアニメに切替）
  const [isExiting, setIsExiting] = useState(false);
  // 種類に応じた設定（アイコン・色）を取得
  const config = toastConfig[toast.type];

  // 閉じるボタンクリック時のハンドラ
  // useCallback で関数をメモ化（依存が変わらない限り同じ関数を使い回す）
  const handleClose = useCallback(() => {
    // アニメーション開始
    setIsExiting(true);
    // 300ms（アニメーション時間）待ってから state から削除
    setTimeout(() => {
      removeToast(toast.id);
    }, 300);
  }, [removeToast, toast.id]);

  // プログレスバー用のアニメーション時間（指定がなければ5秒）
  // || 演算子: 左が falsy（undefined や 0）なら右を使う
  const duration = toast.duration || 5000;

  return (
    <div
      // ARIA ロール: 警告メッセージとして扱う
      role="alert"
      // スクリーンリーダーに即読み上げさせる
      aria-live="assertive"
      // 内容変更時に全体を読み直す（部分だけ読まない）
      aria-atomic="true"
      className={`
        relative
        flex w-full max-w-sm items-start gap-3
        overflow-hidden
        rounded-lg border
        p-4
        shadow-lg
        ${config.containerClass}
        ${isExiting ? "animate-toast-out" : "animate-toast-in"}
      `}
      /* ▼ クラスの読み解き ▼
         relative                : 子要素の absolute（プログレスバー）の基準
         flex w-full max-w-sm    : flex 横並び、幅100%、最大幅 24rem（384px）
         items-start             : 縦方向の上揃え（アイコンとテキストを上揃え）
         gap-3                   : 子要素間 0.75rem（12px）
         overflow-hidden         : プログレスバーがはみ出ないように
         rounded-lg border       : 角丸 + 枠線
         p-4                     : 内側余白 1rem（16px）
         shadow-lg               : 大きめの影
         ${config.containerClass}: 種類ごとの色設定（緑/赤/黄/青）
         三項演算子で「閉じる時 → toast-out / 開く時 → toast-in」アニメ切替
      */
    >
      {/* アイコン */}
      <div className={`flex-shrink-0 ${config.iconClass}`}>
        {/* flex-shrink-0: flex 子要素が縮まないようにする（テキストが長くてもアイコンサイズ維持） */}
        {config.icon}
      </div>
```

```
      {/* テキスト内容 */}
      <div className="flex-1 min-w-0">
        {/* flex-1: 余り領域を全て使う
            min-w-0: flex 子要素がデフォルトで min-width: auto になり改行されない問題を回避 */}
        <p className={`text-sm font-semibold ${config.titleClass}`}>
          {toast.title}
        </p>
        {/* message があるときだけ補足行を表示 */}
        {toast.message && (
          <p className={`mt-1 text-sm ${config.messageClass}`}>
            {/* mt-1: 上マージン 0.25rem (4px) */}
            {toast.message}
          </p>
        )}
      </div>

      {/* 閉じるボタン */}
      <button
        // フォーム内に置かれても submit にならないよう明示
        type="button"
        onClick={handleClose}
        className={`
          flex-shrink-0
          rounded-md p-1
          opacity-70
          transition-opacity
          hover:opacity-100
          focus:outline-hidden focus:ring-2 focus:ring-offset-2
          ${config.iconClass}
        `}
        /* opacity-70 → hover:opacity-100: 普段は薄め、ホバーでくっきり */
        // スクリーンリーダー用のボタン説明
        aria-label="通知を閉じる"
      >
        <svg
          // 16x16 の小さい × アイコン
          className="h-4 w-4"
          fill="none"
          viewBox="0 0 24 24"
          stroke="currentColor"
          strokeWidth={2}
          aria-hidden="true"
        >
          <path
            strokeLinecap="round"
            strokeLinejoin="round"
            // 「×」を描く2本の斜め線
            d="M6 18L18 6M6 6l12 12"
          />
        </svg>
      </button>

      {/* プログレスバー（残り時間表示） */}
      <div className="absolute bottom-0 left-0 right-0 h-1">
        {/* absolute bottom-0 left-0 right-0: 親の下端いっぱいに横一杯
            h-1: 高さ 0.25rem (4px) */}
        <div
          className={`h-full ${config.progressClass} opacity-30`}
          style={{
            // インラインアニメーション。CSS の animation プロパティを直接指定
            // shrinkWidth は別途 @keyframes で定義（幅を100%→0%に縮める）
            animation: `shrinkWidth ${duration}ms linear forwards`,
          }}
        />
      </div>
```

```
    </div>
  );
}

// ---------------------------------------------------------------------------
// トーストコンテナ（画面右上に固定配置するラッパー）
// ---------------------------------------------------------------------------
// レイアウトのルートに1つ置いて、全トーストをここに集約する
export function ToastContainer() {
  // 表示中のトースト配列を取得
  const { toasts } = useToast();

  // 0件なら null を返して描画しない（DOMにも残さない）
  if (toasts.length === 0) return null;

  return (
    <div
      aria-label="通知"
      className="
        fixed
        right-4
        top-4
        z-50
        flex
        flex-col
        gap-3
        sm:right-6
        sm:top-6
      "
      /* ▼ クラスの読み解き ▼
         fixed              : 画面に固定（スクロールしても位置不変）
         right-4 top-4      : 右上から 16px 離れた位置
         z-50               : z-index: 50（他の要素より前面に）
         flex flex-col      : 縦並び
         gap-3              : トースト間 12px の隙間
         sm:right-6 sm:top-6: 640px 以上では 24px に広げる
      */
    >
      {/* トースト配列を1件ずつ ToastItem に展開 */}
      {toasts.map((toast) => (
        <ToastItem key={toast.id} toast={toast} />
      ))}
    </div>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: 種類ごとのアイコン・色を1か所にまとめる（`toastConfig`）

コピー

```
const toastConfig = {
  success: {
    icon: ( /* チェックマークのSVG */ ),
    containerClass: "border-green-200 bg-green-50 dark:border-green-800 dark:bg-green-950",
    iconClass: "text-green-600 dark:text-green-400",
    // ...
  },
  // error / warning / info も同じ構造
};
```

- `toastConfig` は「4種類（成功・エラー・警告・情報）それぞれの**アイコンと色設定**」をまとめたオブジェクトです。
- 後で `toastConfig[toast.type]` と書くだけで、その種類に合った設定一式を取り出せます。
- このように設定を1か所に集めておくと、**新しい種類を足すのも色を直すのも簡単**になります（同じ記述を繰り返さないDRY原則）。

> **用語: DRY原則（Don't Repeat Yourself）** … 同じ記述を何度も繰り返さず、1か所にまとめる設計の考え方。修正漏れやバグを減らせる。

解説2: 閉じるアニメーション付きで削除する（`handleClose`）

コピー

```
const [isExiting, setIsExiting] = useState(false);
const handleClose = useCallback(() => {
  setIsExiting(true);
  setTimeout(() => {
    removeToast(toast.id);
  }, 300);
}, [removeToast, toast.id]);
```

- `isExiting` は「いま閉じるアニメーション中か」を表す state です。`true` になると `animate-toast-out`（右へ滑り出る）に切り替わります。
- `handleClose` はまず `setIsExiting(true)` でアニメを始め、**300ミリ秒（アニメ時間）待ってから** `removeToast` で実際にデータから消します。
- 先に消すとアニメが見えないので、「アニメ→その後に削除」の順番が大事です。

> **用語: アニメーション中フラグ** … `isExiting` のように「いま演出の最中か」を保持する真偽値state。これでクラスを切り替えて出入りの動きを制御する。

解説3: 読み上げソフトへ通知する（`role` / `aria-live`）

コピー

```
<div
  role="alert"
  aria-live="assertive"
  aria-atomic="true"
  className={`... ${config.containerClass} ${isExiting ? "animate-toast-out" : "animate-toast-in"}`}
>
```

- `role="alert"` は「これは警告メッセージだ」とスクリーンリーダーに伝える指定です。
- `aria-live="assertive"` は「内容が出たら**すぐ読み上げて**」という指示。トーストは見逃せない情報なので即時読み上げにしています。
- `aria-atomic="true"` は「一部だけでなく**全文をまとめて読み直す**」指定です。末尾の三項演算子で、状態に応じて入場/退場アニメのクラスを切り替えています。

> **用語: aria-live** … 動的に出る内容を読み上げソフトにどう伝えるかを指定するARIA属性。`assertive`（即時）と `polite`（手が空いたら）がある。

解説4: 残り時間を見せるプログレスバー

コピー

```
<div className="absolute bottom-0 left-0 right-0 h-1">
  <div
    className={`h-full ${config.progressClass} opacity-30`}
    style={{
      animation: `shrinkWidth ${duration}ms linear forwards`,
    }}
  />
</div>
```

- 外側の `div` は `absolute bottom-0 left-0 right-0` でトーストの**下端いっぱい**に置かれ、`h-1`（高さ4px）の細い帯になります。
- 内側の `div` に `animation: shrinkWidth ${duration}ms linear forwards` を直接指定し、「**表示時間をかけて幅を100%→0%に縮める**」演出をしています。これで「あと何秒で消えるか」が視覚的に分かります。
- `linear`（等速）で時間どおりに、`forwards`（終了状態を維持）で縮みきった状態を保ちます。

> **用語: プログレスバー（progress bar）／forwards** … 進捗や残り時間を帯の長さで示すUI。`forwards` はアニメ終了後も最後のコマの見た目を保つ指定。

解説5: 画面右上に全トーストを集約する（`ToastContainer`）

コピー

```
export function ToastContainer() {
  const { toasts } = useToast();
  if (toasts.length === 0) return null;
  return (
    <div className="fixed right-4 top-4 z-50 flex flex-col gap-3 sm:right-6 sm:top-6">
      {toasts.map((toast) => (
        <ToastItem key={toast.id} toast={toast} />
      ))}
    </div>
  );
}
```

- `useToast()` で現在のトースト配列を受け取り、0件なら `return null` で何も描きません。
- `fixed right-4 top-4` で**画面の右上に固定**（スクロールしても動かない）、`z-50` で他要素より前面、`flex flex-col gap-3` で縦に間隔を空けて並べます。
- 配列を `map` で1件ずつ `<ToastItem>` に展開します。このコンテナをレイアウトに1つ置くだけで、全トーストがここに集まります。

> **用語: fixed / z-index（z-50）** … `fixed` は画面に固定する配置。`z-50` は重なり順（z-index）を50にする指定で、数字が大きいほど手前に表示される。

使用例:

> **▼ このコードがやること（先に日本語で）**: ここまでで作ったトーストの仕組みを、実際にどう呼び出すかの例です。削除ボタンを押したら書籍を消し、結果に応じて「成功（緑）」か「エラー（赤）」のトーストを出します。ポイントは `useToast()` を1行呼ぶだけで `toast.success(...)` のように通知を出せること。通信の成否は `res.ok`（HTTP の応答が正常か）で判定しています。

コピー

```
// 任意のコンポーネントから呼び出す例
"use client";

// Context にアクセスするカスタムフック
import { useToast } from "@/contexts/ToastContext";

// 削除ボタンコンポーネント。受け取った bookId の書籍を削除する
export function BookDeleteButton({ bookId }: { bookId: string }) {
  // useToast() でトースト操作のためのオブジェクトを取得
  // toast.success / toast.error などのメソッドが使える
  const toast = useToast();

  // 非同期削除処理。async を付けると関数内で await が使える
  const handleDelete = async () => {
    try {
      // fetch で DELETE リクエスト送信。バッククォート ` ` の中で ${} で値を埋め込み可能
      const res = await fetch(`/api/books/${bookId}`, { method: "DELETE" });
      // res.ok は HTTP ステータスが 200-299 なら true
      if (!res.ok) throw new Error("削除に失敗しました");
      // 成功時のトースト（緑色）を表示
      toast.success("削除完了", "書籍が正常に削除されました。");
    } catch (err) {
      // エラー時のトースト（赤色）を表示
      toast.error("エラー", "書籍の削除に失敗しました。もう一度お試しください。");
    }
  };

  return (
    // btn-danger は globals.css で定義した赤い削除ボタン
    <button onClick={handleDelete} className="btn-danger">
      削除
    </button>
  );
}
```

3.2 ページネーションコンポーネント

ページネーション（pagination：複数ページ分割表示） は、データが多い時に「1 2 3 … 10」のようにページを切り替えるUI です。一度に全件読み込むよりも、サーバーへの負荷とユーザーの待ち時間を減らせます。

> **▼ このコードがやること（先に日本語で）**：「1 2 3 … 10」のようにページを切り替えるボタン群（ページネーション）を作ります。ポイントは、ページ数がとても多いときに全部のボタンを並べず、現在ページの前後と最初・最後だけを残して途中を「…」で省略する計算をしている点です。前へ/次へボタンや、先頭・末尾ページでボタンを無効化する処理も含みます。詳しい計算はコメントを参照してください。

コピー

```
// components/Pagination.tsx
// ↑ ページネーション（複数ページを切り替えるナビゲーション）コンポーネント。
//   1   2   3 ... 10  のような表示を作る。

// クライアントコンポーネント（useSearchParams を使う）
"use client";

// Next.js 用リンクコンポーネント（高速遷移）
import Link from "next/link";
// URL のクエリパラメータ取得 Hook
import { useSearchParams } from "next/navigation";

// コンポーネントが受け取る props 型
type PaginationProps = {
  // 現在表示中のページ番号（1 始まり）
  currentPage: number;
  // 全体のページ数
  totalPages: number;
  // ベースとなるURLパス（例: 一覧ページなら "/"）
  basePath: string;
};

export function Pagination({
  currentPage,
  totalPages,
  basePath,
}: PaginationProps) {
  // 現在の URL のクエリパラメータ（?key=val&...）を取得
  const searchParams = useSearchParams();

  // ページ番号から URL を生成するヘルパー関数
  // 既存の検索条件（?search=xxx 等）を保ちつつ page だけ書き換える
  const createPageUrl = (page: number): string => {
    // URLSearchParams: クエリ文字列を扱う標準 API
    // 既存のパラメータを複製してから page を上書き
    const params = new URLSearchParams(searchParams.toString());
    // 数値を文字列にして set
    params.set("page", String(page));
    // 例: "/?page=2&search=react"
    return `${basePath}?${params.toString()}`;
  };

  // 表示するページ番号のリストを計算する関数
  // 例: currentPage=5, totalPages=10 → [1, "ellipsis", 4, 5, 6, "ellipsis", 10]
  // 返却型: number または "ellipsis"（省略記号）の配列
  const getPageNumbers = (): (number | "ellipsis")[] => {
    const pages: (number | "ellipsis")[] = [];
    // 表示するページ番号の最大数（省略記号除く）
    const maxVisible = 5;

    if (totalPages <= maxVisible + 2) {
      // 全ページ数が少ない場合（7ページ以下なら全部表示）
```

```
      for (let i = 1; i <= totalPages; i++) {
        pages.push(i);
      }
    } else {
      // 多い場合は「先頭 ... 真ん中 ... 末尾」形式
      // 常に最初のページを表示
      pages.push(1);

      // 現在のページが最初の方（1〜3）にある場合
      // → [1, 2, 3, 4, ..., totalPages]
      if (currentPage <= 3) {
        pages.push(2, 3, 4);
        pages.push("ellipsis");
      }
      // 現在のページが最後の方（totalPages-2 以降）にある場合
      // → [1, ..., totalPages-3, totalPages-2, totalPages-1, totalPages]
      else if (currentPage >= totalPages - 2) {
        pages.push("ellipsis");
        pages.push(totalPages - 3, totalPages - 2, totalPages - 1);
      }
      // 現在のページが中間にある場合
      // → [1, ..., current-1, current, current+1, ..., totalPages]
      else {
        pages.push("ellipsis");
        pages.push(currentPage - 1, currentPage, currentPage + 1);
        pages.push("ellipsis");
      }

      // 常に最後のページを表示
      pages.push(totalPages);
    }

    return pages;
  };

  // ページが1ページしかない場合は表示しない（早期リターン）
  if (totalPages <= 1) return null;

  // ページ番号リストを計算
  const pageNumbers = getPageNumbers();
  // 最初のページか
  const isFirstPage = currentPage === 1;
  // 最後のページか
  const isLastPage = currentPage === totalPages;

  // ----- スタイル定数（複数箇所で使い回すために変数化）-----
  // 共通のページ番号ボタンスタイル
  const basePageClass = `
    inline-flex h-10 w-10 items-center justify-center
    rounded-lg text-sm font-medium
    transition-all duration-200
    focus-visible:outline-hidden focus-visible:ring-2
    focus-visible:ring-primary-500 focus-visible:ring-offset-2
  `;
```

```
/* ▼ クラスの読み解き ▼
   inline-flex h-10 w-10        : 40x40 の正方形ボタン
   items-center justify-center  : 縦横中央に数字を配置
   rounded-lg                   : 角丸 8px
   text-sm font-medium          : 14px 太さ500
   transition-all duration-200  : 全プロパティ 200ms で滑らかに変化
   focus-visible:*              : キーボードフォーカス時のリング
*/

// 現在のページ（アクティブ）用スタイル
const activePageClass = `
  ${basePageClass}
  bg-primary-600 text-white shadow-md
  dark:bg-primary-500
`;
/* インディゴ背景＋白文字＋影。現在地が視覚的に分かるように */

// 他のページ番号用スタイル
const inactivePageClass = `
  ${basePageClass}
  text-gray-600 hover:bg-gray-100
  dark:text-gray-400 dark:hover:bg-gray-800
`;
/* 普段は灰色、ホバーで薄い灰色背景 */

// 「前へ」「次へ」が無効化されているとき用
const disabledNavClass = `
  inline-flex h-10 items-center justify-center
  rounded-lg px-3 text-sm font-medium
  text-gray-300 cursor-not-allowed
  dark:text-gray-600
`;
/* text-gray-300       : とても薄い灰色（押せない感じ） */
/* cursor-not-allowed : マウスカーソルが禁止マークに */

// 「前へ」「次へ」が有効なとき用
const enabledNavClass = `
  inline-flex h-10 items-center justify-center
  rounded-lg px-3 text-sm font-medium
  text-gray-600 hover:bg-gray-100
  transition-all duration-200
  dark:text-gray-400 dark:hover:bg-gray-800
  focus-visible:outline-hidden focus-visible:ring-2
  focus-visible:ring-primary-500 focus-visible:ring-offset-2
`;

return (
  // <nav> 要素はナビゲーションを表すセマンティック要素
  <nav
    role="navigation"
    // スクリーンリーダー向けの説明
    aria-label="ページネーション"
    className="flex items-center justify-center gap-1 py-8"
    /* flex items-center justify-center: 中央寄せ
```

```
        gap-1                           : 子要素間 4px
        py-8                            : 上下パディング 2rem (32px) */
    >
      {/* 前のページボタン */}
      {/* 最初のページなら無効化された span を、それ以外なら Link を表示 */}
      {isFirstPage ? (
        <span className={disabledNavClass} aria-disabled="true">
          <svg
            /* mr-1: 右マージン 4px */
            className="mr-1 h-4 w-4"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              /* 左向きの「<」アイコン */
              d="M15.75 19.5L8.25 12l7.5-7.5"
            />
          </svg>
          前へ
        </span>
      ) : (
        // Link は Next.js のクライアントサイドナビゲーション用
        <Link
          // 一つ前のページURL
          href={createPageUrl(currentPage - 1)}
          className={enabledNavClass}
          aria-label="前のページへ"
        >
          <svg
            className="mr-1 h-4 w-4"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M15.75 19.5L8.25 12l7.5-7.5"
            />
          </svg>
          前へ
        </Link>
      )}

      {/* ページ番号リスト */}
      <div className="flex items-center gap-1">
        {/* pageNumbers 配列を1件ずつ展開 */}
```

```
        {pageNumbers.map((page, index) => {
          // 要素が省略記号 "ellipsis" の場合 → "..." を表示
          if (page === "ellipsis") {
            return (
              <span
                // 同じキーが重複しないよう index を含める
                key={`ellipsis-${index}`}
                className="inline-flex h-10 w-10 items-center justify-center text-gray-400"
                // 省略記号は装飾なのでスクリーンリーダーには無視させる
                aria-hidden="true"
              >
                ...
              </span>
            );
          }

          // 数値の場合 → 現在のページか他のページかで表示分岐
          const isActive = page === currentPage;

          return isActive ? (
            // アクティブ（現在）ページは span（リンクではない）
            <span
              key={page}
              className={activePageClass}
              // ARIA 属性: 「現在のページ」と伝える
              aria-current="page"
              aria-label={`${page}ページ目（現在のページ）`}
            >
              {page}
            </span>
          ) : (
            // 他のページは Link で遷移可能に
            <Link
              key={page}
              href={createPageUrl(page)}
              className={inactivePageClass}
              aria-label={`${page}ページ目へ`}
            >
              {page}
            </Link>
          );
        })}
      </div>

      {/* 次のページボタン（最後のページなら無効化） */}
      {isLastPage ? (
        <span className={disabledNavClass} aria-disabled="true">
          次へ
          <svg
            /* ml-1: 左マージン 4px */
            className="ml-1 h-4 w-4"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
```

```
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              /* 右向きの「>」アイコン */
              d="M8.25 4.5l7.5 7.5-7.5 7.5"
            />
          </svg>
        </span>
      ) : (
        <Link
          // 次のページURL
          href={createPageUrl(currentPage + 1)}
          className={enabledNavClass}
          aria-label="次のページへ"
        >
          次へ
          <svg
            className="ml-1 h-4 w-4"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M8.25 4.5l7.5 7.5-7.5 7.5"
            />
          </svg>
        </Link>
      )}
    </nav>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: 検索条件を保ったままページURLを作る（`createPageUrl`）

コピー

```
const createPageUrl = (page: number): string => {
  const params = new URLSearchParams(searchParams.toString());
  params.set("page", String(page));
  return `${basePath}?${params.toString()}`;
};
```

- `URLSearchParams` は「`?key=val&...` というクエリ文字列を扱う標準の道具」です。今のURLのクエリを複製してから操作します。
- `params.set("page", String(page))` で `page` の値だけを上書きします。`String(page)` は数値を文字列に変換しています（クエリは文字列のため）。
- これにより `?search=react` のような**既存の検索条件を消さずに**、ページ番号だけ差し替えたURL（例: `/?page=2&search=react`）を作れます。

> **用語: クエリパラメータ／URLSearchParams** … URLの `?` 以降の `key=value` 部分のこと。`URLSearchParams` はそれを安全に読み書きするブラウザ標準API。

解説2: 表示するページ番号を計算する（`getPageNumbers`）

コピー

```
const getPageNumbers = (): (number | "ellipsis")[] => {
  const pages: (number | "ellipsis")[] = [];
  const maxVisible = 5;
  if (totalPages <= maxVisible + 2) {
    for (let i = 1; i <= totalPages; i++) pages.push(i);
  } else {
    pages.push(1);
    if (currentPage <= 3) { pages.push(2, 3, 4); pages.push("ellipsis"); }
    else if (currentPage >= totalPages - 2) { pages.push("ellipsis"); pages.push(totalPages - 3, totalPages - 2, totalPages - 1); }
    else { pages.push("ellipsis"); pages.push(currentPage - 1, currentPage, currentPage + 1); pages.push("ellipsis"); }
    pages.push(totalPages);
  }
  return pages;
};
```

- この関数は「画面に並べるページ番号の配列」を作ります。要素は**数値**か、省略記号を表す文字列 `"ellipsis"` のどちらかです。
- ページ数が少ない（7ページ以下）なら全部表示。多いときは「**先頭 … 真ん中 … 末尾**」の形にして、間を `"ellipsis"`（…）でまとめます。
- 現在ページが前寄り・後ろ寄り・中間のどこにあるかで、表示する番号の組み合わせを変えています。例: 5ページ目/全10なら `[1, "ellipsis", 4, 5, 6, "ellipsis", 10]`。

> **用語: ユニオン型（`number | "ellipsis"`）** … 「数値、または文字列 ellipsis のどちらか」のように複数の型を許す型。配列に番号と省略記号を混在させるのに使う。

解説3: 先頭・末尾でボタンを無効化する

コピー

```
if (totalPages <= 1) return null;
const isFirstPage = currentPage === 1;
const isLastPage = currentPage === totalPages;
```

コピー

```
{isFirstPage ? (
  <span className={disabledNavClass} aria-disabled="true"> ... 前へ </span>
) : (
  <Link href={createPageUrl(currentPage - 1)} className={enabledNavClass} aria-label="前のページへ"> ... 前へ </Link>
)}
```

- `if (totalPages <= 1) return null` は「1ページしかないなら、そもそもページ送りを表示しない」早期リターンです。
- `isFirstPage` / `isLastPage` で「今が最初/最後のページか」を判定します。
- 最初のページでは「前へ」を**リンクではなく無効化された** `<span>`（`aria-disabled="true"`）にして、押せないことを見た目と読み上げソフトの両方に伝えます。それ以外は `<Link>` で前ページへ遷移できます。

> **用語: aria-disabled** … 「この要素は今は操作できない」ことをスクリーンリーダーに伝えるARIA属性。見た目の `cursor-not-allowed` と合わせて使う。

解説4: 現在ページと他ページを描き分ける（map 内の分岐）

コピー

```
{pageNumbers.map((page, index) => {
  if (page === "ellipsis") {
    return <span key={`ellipsis-${index}`} ... aria-hidden="true">...</span>;
  }
  const isActive = page === currentPage;
  return isActive ? (
    <span key={page} className={activePageClass} aria-current="page" ...>{page}</span>
  ) : (
    <Link key={page} href={createPageUrl(page)} className={inactivePageClass} ...>{page}</Link>
  );
})}
```

- 配列を `map` で展開し、要素が `"ellipsis"` なら「…」を表示します。`aria-hidden="true"` で読み上げソフトには無視させます（装飾のため）。
- 数値の場合は `isActive`（現在ページか）で分岐。現在ページは**リンクにせず** `<span>` にし、`aria-current="page"` で「これが現在地」と伝えます。
- それ以外のページは `<Link>` にして、クリックでそのページへ遷移できるようにします。

> **用語: aria-current="page"** … 同種のリンク群の中で「これが今いるページ」であることを読み上げソフトに示すARIA属性。

3.3 空状態（データがない場合）の改善

空状態（empty state：データが0件・該当なし・初回利用などの「何もない」画面） は、ただ真っ白にせず、ユーザーに次の行動を促す案内を出すのが現代的なUXのお作法です。

> **▼ このコードがやること（先に日本語で）**：データが0件のときに、画面を真っ白にせず「まだ何もありません」とやさしく案内するコンポーネントを作ります。ポイントは、イラストや説明文だけでなく「最初の1冊を登録する」ボタンを置いて、ユーザーが次に何をすればよいかを示すことです。これは使い心地（UX）を良くするための定番テクニックです。

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは →

```
// components/EmptyState.tsx
// ↑ データが0件のときに「まだ何もありません」と分かりやすく案内するコンポーネント。
//   ただ空白にするより、ユーザーに次の行動を促すボタンを置くのが UX 的に親切。

"use client";

import Link from "next/link";

// props 型。? が付いているプロパティはすべて省略可能
type EmptyStateProps = {
  // タイトル
  title?: string;
  // 補足説明文
  message?: string;
  // ボタンの文字
  actionLabel?: string;
  // ボタンのリンク先
  actionHref?: string;
  // 3種類から選択
  icon?: "book" | "search" | "error";
};

// 引数のデフォルト値を分割代入で指定。
// 呼び出し側が省略してもこれらの値が使われる
export function EmptyState({
  title = "書籍が見つかりません",
  message = "まだ書籍が登録されていません。最初の一冊を登録してみましょう。",
  actionLabel = "書籍を登録する",
  actionHref = "/books/new",
  icon = "book",
}: EmptyStateProps) {
  // アイコンの描画
  const renderIcon = () => {
    switch (icon) {
      case "book":
        return (
          <svg
            className="h-16 w-16"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={1}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M12 6.042A8.967 8.967 0 006 3.75c-1.052 0-2.062.18-3 .512v14.25A8.987 8.987 0 016 18c2.305
0 4.408.867 6 2.292m0-14.25a8.966 8.966 0 016-2.292c1.052 0 2.062.18 3 .512v14.25A8.987 8.987 0 0018
18a8.967 8.967 0 00-6 2.292m0-14.25v14.25"
            />
          </svg>
        );
      case "search":
        return (
          <svg
            className="h-16 w-16"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={1}
            aria-hidden="true"
```

```
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M21 21l-5.197-5.197m0 0A7.5 7.5 0 105.196 5.196a7.5 7.5 0 0010.607 10.607z"
            />
          </svg>
        );
      case "error":
        return (
          <svg
            className="h-16 w-16"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={1}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M12 9v3.75m9-.75a9 9 0 11-18 0 9 9 0 0118 0zm-9 3.75h.008v.008H12v-.008z"
            />
          </svg>
        );
    }
  };

  return (
    <div
      className="
        animate-fade-in
        flex flex-col items-center justify-center
        rounded-xl
        border-2 border-dashed border-gray-200
        bg-gray-50/50
        px-6 py-16
        text-center
        dark:border-gray-700
        dark:bg-gray-800/50
      "
      /* ▼ クラスの読み解き ▼
         animate-fade-in              : フェードイン
         flex flex-col items-center   : 縦並びで中央寄せ
         justify-center               : 縦方向も中央
         rounded-xl                   : 大きめの角丸
         border-2 border-dashed       : 2px の点線枠線（dashed:破線）
         bg-gray-50/50                : 灰色の透明度50%（うっすら）
         px-6 py-16                   : 横24px、縦64px の余白
         text-center                  : テキスト中央揃え
         dark:* で各色のダーク版指定
      */
    >
      {/* アイコン */}
      <div
        className="
          mb-4
          rounded-full
          bg-gray-100
          p-4
          text-gray-400
          dark:bg-gray-700
          dark:text-gray-500
        "
        /* mb-4         : 下マージン 1rem (16px)
```

```
            rounded-full : 完全な円形（円形バッジの容器）
            bg-gray-100  : 薄い灰色背景
            p-4          : padding 16px
            text-gray-400: アイコン色 */
        >
          {renderIcon()}
        </div>

        {/* タイトル */}
        <h3
          className="
            mb-2
            text-lg font-semibold
            text-gray-900
            dark:text-gray-100
          "
          /* mb-2: 下16px / text-lg: 18px / font-semibold: 600 */
        >
          {title}
        </h3>

        {/* メッセージ */}
        <p
          className="
            mb-6
            max-w-sm
            text-sm text-gray-500
            dark:text-gray-400
          "
          /* mb-6      : 下24px
             max-w-sm  : 最大幅 24rem (384px)。長い文章でも適度に折り返す
             text-sm   : 14px
             text-gray-500 : 補足色 */
        >
          {message}
        </p>

        {/* アクションボタン（actionHref があれば表示） */}
        {actionHref && (
          <Link href={actionHref} className="btn-primary gap-2">
            {/* btn-primary: globals.css のプライマリボタン
                gap-2     : flex 子要素間 8px（アイコンと文字の間） */}
            <svg
              className="h-4 w-4"
              fill="none"
              viewBox="0 0 24 24"
              stroke="currentColor"
              strokeWidth={2}
              aria-hidden="true"
            >
              <path
                strokeLinecap="round"
                strokeLinejoin="round"
                /* 「+」プラスマーク */
                d="M12 4.5v15m7.5-7.5h-15"
              />
            </svg>
            {actionLabel}
          </Link>
        )}
      </div>
    );
}
```

▼ コードを1つずつ分解して解説

解説1: 省略可能なpropsにデフォルト値を持たせる

コピー

```
type EmptyStateProps = {
  title?: string;
  message?: string;
  actionLabel?: string;
  actionHref?: string;
  icon?: "book" | "search" | "error";
};

export function EmptyState({
  title = "書籍が見つかりません",
  message = "まだ書籍が登録されていません。最初の一冊を登録してみましょう。",
  actionLabel = "書籍を登録する",
  actionHref = "/books/new",
  icon = "book",
}: EmptyStateProps) {
```

- 型の各プロパティに付いた `?` は「**省略してもよい**」という意味です。すべて任意なので、`<EmptyState />` だけでも使えます。
- 分割代入の `title = "..."` の形は「**呼び出し側が渡さなかったときに使うデフォルト値**」です。指定があればそちらが優先されます。
- `icon?: "book" | "search" | "error"` は3種類のうちのどれか、というユニオン型。これ以外の文字列は書けません。

> **用語: デフォルト引数（default parameter）** … 引数が渡されなかったとき自動で使われる初期値。`title = "..."` のように分割代入と組み合わせて書ける。

解説2: iconの値に応じてSVGを返す（ `renderIcon` ）

コピー

```
const renderIcon = () => {
  switch (icon) {
    case "book":
      return ( <svg ...>{/* 本のアイコン */}</svg> );
    case "search":
      return ( <svg ...>{/* 虫眼鏡のアイコン */}</svg> );
    case "error":
      return ( <svg ...>{/* エラーのアイコン */}</svg> );
  }
};
```

- `renderIcon` は `icon` の値（"book"/"search"/"error"）で **表示するSVGアイコンを出し分ける** 関数です。
- `{renderIcon()}` のように本体側で呼び出すと、選ばれたアイコンが描画されます。
- 用途に合わせて「検索結果0件なら虫眼鏡」「エラーなら警告アイコン」のように、同じコンポーネントで見た目を変えられます。

> **用語: SVG（Scalable Vector Graphics）** … 拡大しても劣化しないベクター形式の画像。アイコンをコードで描けるためWebでよく使われる。

解説3: 行動を促すボタンを条件付きで出す（CTA）

コピー

```
{actionHref && (
  <Link href={actionHref} className="btn-primary gap-2">
    <svg ...>{/* 「+」プラスマーク */}</svg>
    {actionLabel}
  </Link>
)}
```

- `{actionHref && (...)}` は「**リンク先が指定されているときだけ**ボタンを表示する」条件付きレンダリングです（ `&&` の短絡評価）。
- `<Link>` に `btn-primary`（globals.css のプライマリボタン）を当て、`gap-2` でアイコンと文字の間に8pxの隙間を作っています。
- 空状態でただ案内するだけでなく「最初の1冊を登録する」など**次の行動への入口**を置くのが、現代的なUXの定石です。

> **用語: CTA（Call To Action）** … ユーザーに次の行動を促すボタンやリンク。「登録する」「始める」など、画面上で最も目立たせたい要素。

3.4 ローディングスケルトン

スケルトン（skeleton：骨組み） は、データ取得中に「これからこういう形の中身が出ます」と予告するグレーの仮表示です。スピナー（くるくる回るアイコン）よりも「コンテンツの骨組み」を見せたほうが、待ち時間の体感が短くなることが知られています。

> **▼ このコードがやること（先に日本語で）**：データ読み込み中に表示する「骨組み（スケルトン）」コンポーネントを作ります。実際のカードと同じ大きさのグレーの仮表示を並べ、光が左右に流れるアニメーション（shimmer）を付けることで「いま読み込み中です」と自然に伝えます。くるくる回るスピナーより待ち時間が短く感じられるのがねらいです。

コピー

```
// components/BookCardSkeleton.tsx
// ↑ ローディング中（データ読込中）に表示する「骨組み」コンポーネント。
//   実際のコンテンツと同じ大きさのグレーのプレースホルダーを表示することで
//   「もうすぐ何かが表示される」ことをユーザーに伝える。

// 1件分のスケルトン
export function BookCardSkeleton() {
  return (
    <div className="card overflow-hidden">
      {/* サムネイルスケルトン（書籍画像の代わり） */}
      <div
        className="
          aspect-[3/4] w-full
          bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200
          bg-[length:200%_100%]
          animate-shimmer
          dark:from-gray-700 dark:via-gray-600 dark:to-gray-700
        "
        /* ▼ クラスの読み解き ▼
           aspect-[3/4] w-full               : 縦横比 3:4、幅100%
           bg-gradient-to-r ... from/via/to  : 左→右の3色グラデーション（中央だけ明るい）
           bg-[length:200%_100%]             : 背景画像のサイズを横200%に
                                               → 余分な部分が画面外にあり、shimmer で動かして光が流れる演出
           animate-shimmer                   : globals.css の @theme で定義したアニメーションを再生
        */
      />

      {/* テキストスケルトン */}
      <div className="flex flex-col gap-2 p-4">
        {/* タイトル行 */}
        <div
          className="
            h-4 w-3/4 rounded
            bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200
            bg-[length:200%_100%]
            animate-shimmer
            dark:from-gray-700 dark:via-gray-600 dark:to-gray-700
          "
          /* h-4: 16px / w-3/4: 幅75% / rounded: 角丸 */
        />
        {/* 著者行（少し遅らせて動かす） */}
        <div
          className="
            h-3 w-1/2 rounded
            bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200
            bg-[length:200%_100%]
            animate-shimmer
            dark:from-gray-700 dark:via-gray-600 dark:to-gray-700
          "
          /* 0.1秒遅れて開始（複数行が同じタイミングだと不自然なため） */
          style={{ animationDelay: "0.1s" }}
        />
        {/* 評価行 */}
        <div
          className="
            mt-2 h-3 w-1/3 rounded
            bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200
            bg-[length:200%_100%]
            animate-shimmer
            dark:from-gray-700 dark:via-gray-600 dark:to-gray-700
          "
          /* さらに遅らせる */
          style={{ animationDelay: "0.2s" }}
        />
      </div>
    </div>
```

```
  );
}

// グリッド全体のスケルトン（複数のスケルトンを並べる）
// count はデフォルト 8、? は省略可能を意味する
export function BookGridSkeleton({ count = 8 }: { count?: number }) {
  return (
    <div
      className="
        grid grid-cols-1 gap-4
        sm:grid-cols-2 sm:gap-5
        lg:grid-cols-3 lg:gap-6
        xl:grid-cols-4
      "
      /* BookGrid と同じレスポンシブグリッド設定にすることで、
         ローディング中も実際の表示と同じ配置になる */
    >
      {/* Array.from({ length: count }) で count 個の空配列を作り、map で展開する慣用句 */}
      {/* _ は使わない引数の慣例的名前（i だけ使う） */}
      {Array.from({ length: count }).map((_, i) => (
        <BookCardSkeleton key={i} />
      ))}
    </div>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: 光が流れるシマー効果（gradient + shimmer）

コピー

```
<div
  className="
    aspect-[3/4] w-full
    bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200
    bg-[length:200%_100%]
    animate-shimmer
    dark:from-gray-700 dark:via-gray-600 dark:to-gray-700
  "
/>
```

- `aspect-[3/4] w-full` で「実際の書籍画像と同じ縦長の枠」を作ります。`[ ]` はTailwindに無い任意の値を直接書く構文です。
- `bg-gradient-to-r from-gray-200 via-gray-100 to-gray-200` は「左→右の3色グラデーション（中央だけ少し明るい）」。これが**流れる光の素**になります。
- `bg-[length:200%_100%]` で背景を横2倍に引き伸ばして余りを画面外に置き、`animate-shimmer` でそれを左右に動かすことで「**キラッと光が流れる**」演出になります。

> **用語: スケルトン（skeleton）／シマー（shimmer）** … スケルトンは読込中に出すグレーの仮表示。シマーはそこに光を流して「読込中」を自然に伝えるアニメーション。

解説2: 行ごとに開始をずらす（`animationDelay`）

コピー

```
<div className="h-3 w-1/2 rounded ... animate-shimmer ..." style={{ animationDelay: "0.1s" }} />
<div className="mt-2 h-3 w-1/3 rounded ... animate-shimmer ..." style={{ animationDelay: "0.2s" }} />
```

- タイトル行・著者行・評価行と、テキストの仮表示を `h-4` / `h-3`、`w-3/4` / `w-1/2` / `w-1/3` のように**高さと幅を変えて**並べ、実際のレイアウトに似せています。
- `animationDelay: "0.1s"` / `"0.2s"` で各行のアニメ開始を少しずつ遅らせています。
- 全行が同じタイミングで光ると不自然なので、**ずらすことで自然な動き**に見せています。

> **用語: animationDelay（アニメーション遅延）** … アニメ開始までの待ち時間。要素ごとに変えると、一斉でなく順々に動く生き生きとした演出になる。

解説3: 必要な数だけスケルトンを並べる（`Array.from`）

コピー

```
export function BookGridSkeleton({ count = 8 }: { count?: number }) {
  return (
    <div className="grid grid-cols-1 gap-4 sm:grid-cols-2 sm:gap-5 lg:grid-cols-3 lg:gap-6 xl:grid-cols-4">
      {Array.from({ length: count }).map((_, i) => (
        <BookCardSkeleton key={i} />
      ))}
    </div>
  );
}
```

- `count = 8` はデフォルトで8個のスケルトンを並べる指定（省略可能）。
- `Array.from({ length: count })` は「**要素が count 個の配列を作る**」慣用句で、その `.map` で `<BookCardSkeleton>` を必要数だけ生成します。`_` は「使わない引数」の慣例的な名前です。
- グリッドのクラスを `BookGrid` と**同じレスポンシブ設定**にしているので、読込中も本物と同じ配置で表示されます。

> **用語: Array.from({ length: n })** … 長さ n の配列を作る定番の書き方。`.map` と組み合わせて「同じ要素を n 個並べる」のに使う。

4. アニメーション

4.1 Tailwind のトランジションクラス

Tailwind CSS にはトランジション用のユーティリティクラスが組み込まれています。

「トランジション（transition）」と「アニメーション（animation）」の違いを整理しておきます。

- **トランジション**: ある状態 A から状態 B に変化するときの「途中の動き」を滑らかにする。例: `hover` で色を変える時に 0.2 秒かける。
- **アニメーション**: キーフレーム（`@keyframes`）を使って、独自の動きを定義する。例: ローディングスピナーの回転、フェードイン演出。

クラス	説明	CSS 出力
`transition`	一般的なプロパティにトランジション適用	`transition-property: color, background-color, border-color, ...`
`transition-all`	全プロパティにトランジション適用	`transition-property: all`
`transition-colors`	色関連のみ	`transition-property: color, background-color, ...`
`transition-opacity`	透明度のみ	`transition-property: opacity`
`transition-transform`	変形のみ	`transition-property: transform`
`duration-150`	150ms	`transition-duration: 150ms`
`duration-200`	200ms	`transition-duration: 200ms`
`duration-300`	300ms	`transition-duration: 300ms`
`duration-500`	500ms	`transition-duration: 500ms`
`ease-in`	加速カーブ（最初遅く・最後速い）	`transition-timing-function: cubic-bezier(0.4, 0, 1, 1)`
`ease-out`	減速カーブ（最初速く・最後ゆっくり）	`transition-timing-function: cubic-bezier(0, 0, 0.2, 1)`
`ease-in-out`	加速→減速カーブ	`transition-timing-function: cubic-bezier(0.4, 0, 0.2, 1)`

> **イージング（easing）の選び方の目安：** - 要素が登場する時 → `ease-out`（パッと出てフワッと止まる、自然な動き） - 要素が消える時 → `ease-in`（じわっと加速して消える） - 行き来する時（トグル等）→ `ease-in-out`

4つのイージングを「時間（横）」と「どこまで進んだか（縦）」のグラフにすると、違いが一目で分かります。線の傾きが急なところほど、速く動いています。

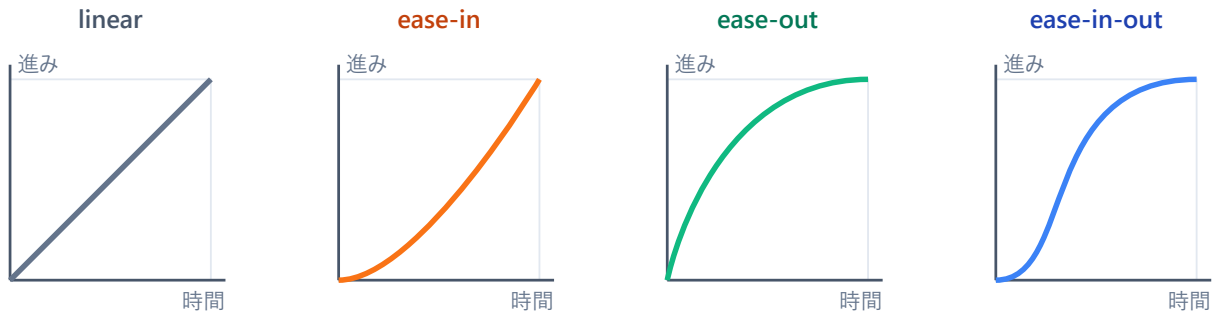


図9-14: イージングの違い（線が急なところほど速く動く。上の表の cubic-bezier の数値をそのまま描いたもの）

つまずきポイント: `transition` を `hover:` の側に付けてしまう

`transition` は「変化をなめらかにする」設定なので、**変化が起きる前から**要素に付いている必要があります。`hover:transition` と書くと、マウスを乗せたときはなめらかに変わりますが、離した瞬間に `transition` も一緒に外れるので、**元に戻る動きがパッと一瞬**になってしまいます。

コピー

```
<!-- NG: transition を hover: の中に入れた -->
<!-- → 乗せたときはなめらかだが、離したときにパッと戻ってしまう -->
<button class="bg-blue-500 hover:bg-blue-700 hover:transition hover:duration-300">保存</button>
```

コピー

```
<!-- OK: transition は「いつでも」付けておき、変える色だけを hover: にする -->
<!-- → 乗せたときも離したときも、300ms かけてなめらかに変わる -->
<button class="bg-blue-500 transition duration-300 hover:bg-blue-700">保存</button>
```

4.2 カードホバーエフェクト

ホバーエフェクト（hover effect：マウスを乗せた時に発動する視覚効果） は、ユーザーに「ここはクリックできる」「ここに注目すべき」と伝える重要な合図になります。やりすぎると煩わしいので、微妙な変化（影が深くなる・少し浮く・色が変わる）を組み合わせるのがコツです。

▼ このコードがやること（先に日本語で）：カードにマウスを乗せた時の「ふわっと浮かぶ・影が深くなる」などの演出（ホバーエフェクト）を付ける例です。ポイントは `transition`（変化を時間をかけて滑らかにつなぐ）と組み合わせて、急にパッと変わらず気持ちよく動かすこと。やりすぎると煩わしいので、影・浮き上がり・色変化を控えめに重ねるのがコツです。

コピー

```
// components/BookCard.tsx 内のホバーエフェクト部分（上のシンプル版に加えて、より凝った演出を追加した例）

// カード全体のホバー（上に浮かぶ + 影が深くなる）
<article
  className="
    card
    overflow-hidden
    transition-all duration-300 ease-out
    hover:-translate-y-1.5
    hover:shadow-card-hover
  "
  /* ▼ クラスの読み解き ▼
     card                       : globals.css のカード基本スタイル
     overflow-hidden            : 中身がはみ出さない（画像ズーム時のため）
     transition-all duration-300: 全プロパティを 300ms かけて変化
     ease-out                   : 最初速く・最後ゆっくり（自然な減速カーブ）
     hover:-translate-y-1.5     : ホバーで上に 6px 移動（浮き上がる）
     hover:shadow-card-hover    : ホバーで大きな影に
  */
>
  {/* サムネイル画像のホバー（ズームイン） */}
  <div className="relative aspect-[3/4] overflow-hidden">
    <img
      src={book.thumbnailUrl}
      alt={book.title}
      className="
        h-full w-full object-cover
        transition-transform duration-500 ease-out
        group-hover:scale-110
      "
      /* group-hover:scale-110 : 親（.group）のホバー時に画像を 110% にズーム */
    />

    {/* オーバーレイ（ホバー時にフェードイン） */}
    <div
      className="
        absolute inset-0
        bg-gradient-to-t from-black/60 via-transparent to-transparent
        opacity-0
        transition-opacity duration-300
        group-hover:opacity-100
      "
      /* ▼ クラスの読み解き ▼
         absolute inset-0          : 親いっぱいに広げる（top:0 right:0 bottom:0 left:0）
         bg-gradient-to-t          : 下→上方向のグラデーション
         from-black/60 ... transparent: 下側は黒60%、上側は透明
         opacity-0                 : 通常は透明（非表示）
         group-hover:opacity-100   : 親ホバー時に表示
      */
    />

    {/* 「詳細を見る」テキスト（ホバー時に下からスライドイン） */}
    <div
      className="
        absolute bottom-0 left-0 right-0
        p-4
        text-white
        translate-y-4 opacity-0
        transition-all duration-300
        group-hover:translate-y-0 group-hover:opacity-100
```

```
        "
        /* translate-y-4 opacity-0       : 通常は 16px 下にずれて透明（見えない）
           group-hover:translate-y-0 ... : ホバー時に元位置 + 不透明（スライドアップで登場）
        */
      >
        <span className="text-sm font-medium">詳細を見る →</span>
      </div>
    </div>
  </article>
```

4.3 ページ遷移時のアニメーション

Next.js App Router でのページ遷移時にフェードイン・スライドインのアニメーションを付けます。

> **▼ このコードがやること（先に日本語で）**：ページを移動したときに、新しい画面が「ふわっ」とフェードインしながら少し上に上がって現れる演出を付けるラッパー部品を作ります。仕組みのカギは、URL（パス）が変わったら一瞬だけ「透明・少し下」の状態を描いてから「不透明・元の位置」へ切り替えることで、アニメーションを毎回確実に発動させる点です。`requestAnimationFrame` の役割はコメントで説明しています。

コピー

```
// components/PageTransition.tsx
// ↑ ページ遷移時に「ふわっ」とフェードイン＋上昇するアニメーションを付けるラッパー。
//   <PageTransition>{children}</PageTransition> のように使う。

"use client";

// 現在の URL パス取得 Hook
import { usePathname } from "next/navigation";
// React の基本 Hook と型
import { useEffect, useState, type ReactNode } from "react";

// type ReactNode: JSXとして描画可能なあらゆる値（文字列・JSX・配列など）の型
type PageTransitionProps = {
  children: ReactNode;
};

export function PageTransition({ children }: PageTransitionProps) {
  // 現在のパス（/books など）
  const pathname = usePathname();
  // 表示状態（アニメ用フラグ）
  const [isVisible, setIsVisible] = useState(false);
  // 表示中の子要素
  const [displayChildren, setDisplayChildren] = useState(children);

  useEffect(() => {
    // pathname または children が変わったときに実行される

    // パスが変わるたびにアニメーションをリセット
    // 一旦非表示状態に
    setIsVisible(false);
    // 子要素を更新
    setDisplayChildren(children);

    // requestAnimationFrame: ブラウザの次の描画タイミングで実行されるコールバック
    // → ブラウザが「非表示状態」を一度描画してから、すぐに「表示」に切り替えるため
    //   アニメーションが確実に発火する
    const timer = requestAnimationFrame(() => {
      setIsVisible(true);
    });

    // クリーンアップ: 次の useEffect 実行前 or アンマウント時にタイマーをキャンセル
    return () => cancelAnimationFrame(timer);
  // この配列の値が変わるたびに再実行
  }, [pathname, children]);

  return (
    <div
      className={`
        transition-all duration-300 ease-out
```

```
        ${
          isVisible
            // 表示中 → 元位置 / 不透明
            ? "translate-y-0 opacity-100"
            // 非表示 → 8px下 / 透明
            : "translate-y-2 opacity-0"
        }
      `}
    >
      {displayChildren}
    </div>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: パスと表示中の子要素を state で持つ

コピー

```
const pathname = usePathname();
const [isVisible, setIsVisible] = useState(false);
const [displayChildren, setDisplayChildren] = useState(children);
```

- `usePathname()` は「**現在のURLパス**（`/` や `/books/new` など）」を取得するNext.jsのHookです。ページが変わるとこの値が変わります。
- `isVisible` は「いま表示状態（不透明・元位置）か」を表すアニメ用フラグ。最初は `false`（透明・少し下）です。
- `displayChildren` は「**今画面に出している中身**」。新しいページに切り替わる瞬間を制御するために、子要素を直接描かず一度stateに入れています。

> **用語: usePathname** … 現在のURLのパス部分を返すNext.jsのHook。これが変わったらページ遷移が起きたと判断できる。

解説2: 遷移ごとにアニメをやり直す（`useEffect` + `requestAnimationFrame`）

コピー

```
useEffect(() => {
  setIsVisible(false);
  setDisplayChildren(children);
  const timer = requestAnimationFrame(() => {
    setIsVisible(true);
  });
  return () => cancelAnimationFrame(timer);
}, [pathname, children]);
```

- `useEffect` の依存配列 `[pathname, children]` により、**ページ（パス）や中身が変わるたび**にこの中が実行されます。
- まず `setIsVisible(false)` で「透明・少し下」に戻し、`requestAnimationFrame(...)` で**次の描画タイミング**に `setIsVisible(true)` を予約します。一度「非表示」を描いてから「表示」に切り替えるので、アニメが毎回確実に発火します。
- `return () => cancelAnimationFrame(timer)` は後片付け（クリーンアップ）で、次の実行前や画面から消える時に予約を取り消します。

> **用語: requestAnimationFrame** … 「ブラウザの次の描画の直前に1回実行して」と予約するブラウザAPI。状態を確実に1コマ描かせてからアニメを始めたいときに使う。

解説3: フラグで見た目を切り替える（transition クラス）

コピー

```
<div
  className={`
    transition-all duration-300 ease-out
    ${
      isVisible
        ? "translate-y-0 opacity-100"
        : "translate-y-2 opacity-0"
    }
  `}
>
  {displayChildren}
</div>
```

- `transition-all duration-300 ease-out` は「全プロパティを300msかけて、最後ゆっくり（ease-out）で変化させる」指定です。

- `isVisible` が `true` なら `translate-y-0 opacity-100`（元位置・不透明）、`false` なら `translate-y-2 opacity-0`（8px下・透明）。この切り替わりに `transition` がかかるので**ふわっと上昇しながら現れます**。
- 中身は `children` ではなく `displayChildren` を描き、解説2のタイミング制御と組み合わせています。

> **用語: transition（トランジション）** … 状態Aから状態Bへ変わるときの途中を滑らかにつなぐCSS機能。`duration` で時間、`ease-out` で速度カーブを指定する。

> **▼ このコードがやること（先に日本語で）**：先ほど作った `PageTransition`（ページ遷移アニメーション）を、アプリ全体で使うために「ルートレイアウト」に組み込む例です。ルートレイアウトは全ページ共通の `<html><body>` の枠組みで、ここでページ本文を `PageTransition` で包むと、すべてのページ移動に自動でアニメーションが付きます。トーストの仕組みもここで全体に適用します。

コピー

```
// app/layout.tsx での使用例
// ↑ Next.js App Router のルートレイアウト。
//   全ページ共通の <html><body> 構造と、ヘッダー・フッターなどを定義する。

import { PageTransition } from "@/components/PageTransition";
import { ToastProvider } from "@/contexts/ToastContext";
import { ToastContainer } from "@/components/Toast";
import { Header } from "@/components/Header";
import { Footer } from "@/components/Footer";
// グローバル CSS は必ずここで import
import "./globals.css";

export default function RootLayout({
  // 各ページの内容がここに入る
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    // suppressHydrationWarning: ダークモード切替時のSSR/CSR差異警告を抑制
    <html lang="ja" suppressHydrationWarning>
      {/* flex min-h-screen flex-col:
            縦並びで最低でも画面高さ100%を占める → フッターを下端に固定できる */}
      <body className="flex min-h-screen flex-col">
        {/* ToastProvider: 内側の全コンポーネントから useToast を使えるようにする */}
        <ToastProvider>
          <Header />
          {/* main: メインコンテンツのセマンティック要素
              container: globals.css の @utility container で中央寄せ＋レスポンシブな余白
              flex-1: 余り領域を全て使う（ヘッダーとフッターの間を埋める）
              py-8: 上下 32px の余白 */}
          <main className="container flex-1 py-8">
            <PageTransition>{children}</PageTransition>
          </main>
          <Footer />
          {/* ToastContainer はトーストを画面右上に固定表示する別ツリー */}
          <ToastContainer />
        </ToastProvider>
      </body>
    </html>
  );
}
```

4.4 ボタンのアニメーション

▼ **このコードがやること（先に日本語で）**：押した瞬間に少し縮む・処理中はスピナーを出す・色違い（バリアント）を選べる、といった機能を備えた「使い回せるボタン」を作ります。ポイントは、`variant`（primary や danger など）を props で受け取り、対応する見た目のクラスを切り替える設計です。ローディング中はボタンを無効化して二重クリックを防ぎます。

コピー

```
// components/AnimatedButton.tsx
// ↑ 押した時のスケール変化、ローディング表示、バリアント切り替えを備えた汎用ボタン。

"use client";

// ButtonHTMLAttributes: <button> 要素が持つ全ての属性（onClick, type, name など）の型
// type を付けて import すると「型のみ」インポート（実行時に消える）
import { type ButtonHTMLAttributes, type ReactNode } from "react";

// props 型: <button> の全属性に独自プロパティを追加する（インターセクション型 &）
type AnimatedButtonProps = ButtonHTMLAttributes<HTMLButtonElement> & {
  // ボタンの種類
  variant?: "primary" | "secondary" | "danger" | "ghost";
  // ボタンの大きさ
  size?: "sm" | "md" | "lg";
  // 処理中フラグ（trueでスピナー表示）
  isLoading?: boolean;
  // ボタン内のラベル
  children: ReactNode;
};

export function AnimatedButton({
  // 既定はプライマリ
  variant = "primary",
  // 既定は中サイズ
  size = "md",
  // 既定は処理していない
  isLoading = false,
  children,
  // 追加クラスを受け取れるように（既定は空）
  className = "",
  // 標準の disabled も受け取る
  disabled,
  // 残りの<button>属性をまとめて受け取る（rest演算子）
  ...props
}: AnimatedButtonProps) {
  // バリアントごとのスタイル（オブジェクトで管理→ variantStyles[variant] で参照）
  const variantStyles = {
    primary: `
      bg-primary-600 text-white
      hover:bg-primary-700
      active:bg-primary-800
      focus-visible:ring-primary-500
      shadow-md hover:shadow-lg
    `,
    secondary: `
      bg-secondary-600 text-white
      hover:bg-secondary-700
      active:bg-secondary-800
      focus-visible:ring-secondary-500
    `,
    danger: `
      bg-red-600 text-white
      hover:bg-red-700
      active:bg-red-800
      focus-visible:ring-red-500
    `,
    ghost: `
```

```
      text-gray-600 hover:bg-gray-100
      dark:text-gray-400 dark:hover:bg-gray-800
      focus-visible:ring-gray-500
    `,
  };

  // サイズごとのスタイル
  const sizeStyles = {
    /* 小: 横12px 縦6px 文字12px */
    sm: "px-3 py-1.5 text-xs gap-1.5",
    /* 中: 横16px 縦10px 文字14px */
    md: "px-4 py-2.5 text-sm gap-2",
    /* 大: 横24px 縦12px 文字16px */
    lg: "px-6 py-3 text-base gap-2.5",
  };

  return (
    <button
      className={`
        inline-flex items-center justify-center
        rounded-lg font-medium
        transition-all duration-200 ease-out
        focus-visible:outline-hidden focus-visible:ring-2 focus-visible:ring-offset-2
        disabled:pointer-events-none disabled:opacity-50
        active:scale-[0.97]
        ${variantStyles[variant]}
        ${sizeStyles[size]}
        ${className}
      `}
      /* active:scale-[0.97] : クリック中（押している瞬間）に97%サイズに縮める → 押下感の演出 */
      /* 任意値構文 [ ] でTailwindにないサイズを直接指定できる */
      /* 処理中も無効化 */
      disabled={disabled || isLoading}
      /* onClick等の追加属性をスプレッドで渡す */
      {...props}
    >
      {/* isLoading が true の時だけスピナーを表示 */}
      {isLoading && (
        <svg
          /* animate-spin: 1秒ループ回転 */
          className="h-4 w-4 animate-spin"
          viewBox="0 0 24 24"
          fill="none"
          aria-hidden="true"
        >
          {/* 円弧（薄い背景の円） */}
          <circle
            /* 透明度25%（薄く） */
            className="opacity-25"
            /* 中心 x座標 */
            cx="12"
            /* 中心 y座標 */
            cy="12"
            /* 半径10 */
            r="10"
            stroke="currentColor"
            strokeWidth="4"
          />
          {/* 回転する濃い部分（パス） */}
          <path
            /* 透明度75%（濃いめ） */
```

```
            className="opacity-75"
            fill="currentColor"
            d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4z"
          />
        </svg>
      )}
      {children}
    </button>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: `<button>` の全属性に独自propsを足す（インターセクション型）

コピー

```
type AnimatedButtonProps = ButtonHTMLAttributes<HTMLButtonElement> & {
  variant?: "primary" | "secondary" | "danger" | "ghost";
  size?: "sm" | "md" | "lg";
  isLoading?: boolean;
  children: ReactNode;
};
```

- `ButtonHTMLAttributes<HTMLButtonElement>` は「`<button>` が本来持つ全属性（`onClick`・`type`・`disabled` など）の型」です。
- `&` （インターセクション型）で、それに `variant`・`size`・`isLoading` という**独自プロパティを足し合わせて**います。
- これにより、標準のボタン属性も独自のオプションも両方受け取れる、使い勝手のよいボタンになります。

> **用語: インターセクション型（`A & B`）** ...「Aの性質もBの性質も両方持つ」型。既存の型に独自プロパティを追加するときに使う。

解説2: 残りの属性をまとめて受け取って渡す（rest と spread）

コピー

```
export function AnimatedButton({
  variant = "primary",
  size = "md",
  isLoading = false,
  children,
  className = "",
  disabled,
  ...props
}: AnimatedButtonProps) {
  return (
    <button
      // ... className ...
      disabled={disabled || isLoading}
      {...props}
    >
```

- `...props`（rest演算子）は「分割代入で名前を付けなかった**残りの全属性**」をまとめて受け取ります（`onClick` など）。
- `{...props}`（spread構文）でそれらを `<button>` に**そのまま展開して渡します**。これで親が渡した `onClick` 等がボタンに反映されます。
- `disabled={disabled || isLoading}` は「明示的に無効化されている、**または処理中**なら無効化」。二重クリック（多重送信）を防ぐ工夫です。

> **用語: rest演算子 / spread構文（`...`）** … 同じ `...` 記号でも、受け取る側では「残りをまとめる（rest）」、渡す側では「展開する（spread）」働きをする。

解説3: variant/size でスタイルを切り替える

コピー

```
const variantStyles = {
  primary: `bg-primary-600 text-white hover:bg-primary-700 ...`,
  danger: `bg-red-600 text-white hover:bg-red-700 ...`,
  // ...
};
const sizeStyles = {
  sm: "px-3 py-1.5 text-xs gap-1.5",
  md: "px-4 py-2.5 text-sm gap-2",
  lg: "px-6 py-3 text-base gap-2.5",
};
// className 内で:
//   ${variantStyles[variant]}
//   ${sizeStyles[size]}
```

- `variantStyles` は色違い（primary/secondary/danger/ghost）、`sizeStyles` は大きさ（sm/md/lg）の**クラスをまとめたオブジェクト**です。
- `variantStyles[variant]` のように**propsの値をキーにして**該当スタイルを取り出し、className に差し込みます。
- `active:scale-[0.97]` も付いていて、「クリック中（押している瞬間）だけ97%に縮む」押下感の演出になります。

> **用語: ルックアップオブジェクト（lookup object）** … `スタイル[キー]` のようにキーで値を引くためのオブジェクト。長い `if` / `switch` の代わりに使えて見通しがよい。

解説4: 処理中だけスピナーを出す

コピー

```
{isLoading && (
  <svg className="h-4 w-4 animate-spin" ...>
    <circle className="opacity-25" cx="12" cy="12" r="10" stroke="currentColor" strokeWidth="4" />
    <path className="opacity-75" fill="currentColor" d="M4 12a8 8 0 018-8V0C5.373 0 0 5.373 0 12h4z" />
  </svg>
)}
```

- `{isLoading && (...)}` は「`isLoading` が `true` のときだけ」スピナーを描く短絡評価です。
- `animate-spin` は「1秒で1回転を無限に繰り返す」アニメ。薄い円（`opacity-25`）の上に濃い円弧（`opacity-75`）を重ねて、**回っているように見せる**定番の作りです。
- これと解説2の `disabled` を合わせて、「処理中はボタンを押せず、くるくる回って待たせる」UXを実現しています。

> **用語: スピナー（spinner）／animate-spin** … 処理中を示すくるくる回るアイコン。`animate-spin` はTailwindの回転アニメーションクラス。

---

5. ダークモード（発展）

5.1 Tailwind のダークモード設定

1.1 節の `globals.css` で `@custom-variant dark (&:where(.dark, .dark *));` を設定済みなので、`<html>` タグに `dark` クラスを付与することでダークモードが有効になります（v3 では `tailwind.config.ts` の `darkMode: "class"` で同じことを指定します）。

ダークモード実装には大きく2つのパターンがあります。

- 「**class**」**方式**: `<html class="dark">` のように、ユーザーの選択で手動切替する方式。本アプリで採用。
- 「**media**」**方式**: OS の設定（`prefers-color-scheme`）に自動連動する方式。シンプルだがユーザーが選べない。

コピー

```
<!-- ライトモード（dark クラスなし） -->
<html lang="ja">

<!-- ダークモード（dark クラスあり） -->
<!-- このクラスが付くと、すべての dark:〜 のスタイルが有効化される -->
<html lang="ja" class="dark">
```

本アプリの「CSS 変数を切り替える方式」で、ボタンを押してから画面の色が変わるまでの流れを図にすると次のとおりです。**部品（コンポーネント）側は何も書き換えず、変数の中身だけが入れ替わる**のがポイントです。照明のスイッチを1つ切り替えると、部屋の家具を1つずつ塗り直さなくても部屋全体の見え方が変わる、というイメージです。



図9-15: ダークモードが切り替わる流れ（CSS 変数方式）

> **つまずきポイント: v4 で `dark:` のクラスが、ボタンを押しても効かない**
>
> Tailwind v4 の `dark:` は、何も設定しないと **OS のダークモード設定に連動する**作りになっています。5.3 節の ThemeToggle で `<html class="dark">` を付けても、パソコンの設定がライトのままなら `dark:bg-…` は効きません。v3 の `darkMode: "class"` に当たる1行を、`globals.css` に足す必要があります。

コピー

```
/* NG: v4 で何も設定していない → dark: は OS の設定にしか反応しない */
@import "tailwindcss";
```

コピー

```
/* OK: dark: を「dark クラスが付いた要素とその中身」で効かせる */
@import "tailwindcss";
@custom-variant dark (&:where(.dark, .dark *));
```

`<div class="bg-white dark:bg-slate-900">本棚</div>` を CLI に通して、作られた CSS を比べると違いがはっきりします。

▼ **実行結果**（それぞれの出力の `dark:bg-slate-900` の部分）

コピー

```
---- NG: 「OS がダークのとき」という条件になっている ----
  @media (prefers-color-scheme: dark) {
    .dark\:bg-slate-900 {
      background-color: var(--color-slate-900);
    }
  }

---- OK: 「.dark の中にいるとき」という条件になっている ----
  .dark\:bg-slate-900:where(.dark, .dark *) {
    background-color: var(--color-slate-900);
  }
```

- NG の `@media (prefers-color-scheme: dark)` は「OS（パソコンやスマホ）の設定がダークのとき」という意味です。アプリのボタンでは切り替えられません。
- OK の `:where(.dark, .dark *)` は「`dark` クラスが付いた要素自身か、その中にある要素のとき」という意味です。`<html>` に `dark` を付ければ、ページの中の全部が当てはまります。

5.2 dark: クラスの使い方

Tailwind の `dark:` バリアントを使うと、ダークモード時に適用されるスタイルを簡単に記述できます。

▼ このコードがやること（先に日本語で）：ダークモード専用の見た目を指定する `dark:` 接頭辞の使い方を、いくつかのパターンで示します。基本は「通常のクラス」の後ろに `dark:〜` を並べるだけで、ダークモードのときだけそちらが適用されます。`dark:hover:〜` のようにホバーやフォーカスと組み合わせて書けるのもポイントです。実際の組み合わせ例を見て感覚をつかんでください。

コピー

```
{/* 基本パターン: 通常のクラスの後に dark: プレフィックスを付ける */}
{/* bg-white: 通常は白背景 / dark:bg-gray-900: ダーク時は濃灰背景 */}
<div className="bg-white dark:bg-gray-900">
  {/* text-gray-900: 通常は濃灰文字 / dark:text-gray-100: ダーク時は薄灰文字 */}
  <h1 className="text-gray-900 dark:text-gray-100">タイトル</h1>
  {/* 補足テキスト: 通常は中間グレー / ダーク時は薄め */}
  <p className="text-gray-600 dark:text-gray-400">本文テキスト</p>
</div>

{/* ボーダーの例: 通常は薄い枠 / ダーク時は濃い枠 */}
<div className="border border-gray-200 dark:border-gray-700">
  ...
</div>

{/* ホバーとの組み合わせ（dark: と hover: は並べて書ける） */}
<button className="
  bg-blue-500 hover:bg-blue-600
  dark:bg-blue-600 dark:hover:bg-blue-700
">
  {/* bg-blue-500           : 通常背景 */}
  {/* hover:bg-blue-600     : ホバー時に濃く */}
  {/* dark:bg-blue-600      : ダーク時の背景 */}
  {/* dark:hover:bg-blue-700: ダーク × ホバー時 */}
  ボタン
</button>

{/* リングとフォーカスとの組み合わせ */}
<input className="
  focus:ring-blue-500
  dark:focus:ring-blue-400
" />
{/* focus:ring-blue-500       : フォーカス時のリング色 */}
{/* dark:focus:ring-blue-400  : ダーク × フォーカス時のリング色 */}
```

ダークモードの実装パターンまとめ：

1. **CSS変数を切り替える方式**（本アプリで採用）：`:root` に変数を定義し、`.dark` で値を上書き。コンポーネント側は `bg-[var(--color-card)]` のように変数を参照するだけで自動切替。
2. **`dark:` クラスを毎回書く方式**: コンポーネントの各クラスに `dark:` 付きを並べる。直感的だが冗長になる。
3. **両者の併用**: 本アプリも基本色は CSS 変数、細かい補正は `dark:` で行うハイブリッド方式。

同じ書籍カードを、ライトとダークで並べた Before/After です。カードの HTML とクラスは**まったく同じ**で、`<html>` に `dark` クラスがあるかどうかだけが違います。



図9-16: Before/After — 同じ書籍カードのライト表示（左）とダーク表示（右）

左右を見比べると、ダークでは「背景だけ黒くする」のではなく、**カードは背景より少し明るい色**（**#1e293b**）、**補足の文字は少し明るい灰色**（**#94a3b8**）、**バッジは濃い地に明るい文字**と、すべての組み合わせが「暗い背景でも読みやすい」ように入れ替わっていることが分かります。1.1 節の `.dark { … }` で変数ごとに値を用意したのは、このためです。

ヒント: ダークモードの背景には真っ黒（ `#000000` ）より、少し青みのある濃い灰色（slate-900 の `#0f172a` など）を使うのが定番です。真っ黒に真っ白の文字だと明暗の差が強すぎて、長く読むと目が疲れやすくなります。

5.3 テーマ切り替えボタンの実装

▼ このコードがやること（先に日本語で）：ライト・ダーク・システム連動を切り替えるボタンを作ります。カギは、選んだテーマに応じて `<html>` タグに `dark` クラスを付け外しすることと、その選択を `localStorage`（ブラウザに残る保存場所）に覚えさせて次回も同じ見た目で開けるようにする点です。これで globals.css の CSS 変数が一斉に切り替わり、画面全体の色が変わります。

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
// components/ThemeToggle.tsx
// ↑ テーマ（ライト/ダーク/システム連動）を切り替えるボタンコンポーネント。

"use client";

import { useState, useEffect } from "react";

// Theme 型: 3つのテーマ候補
type Theme = "light" | "dark" | "system";

export function ThemeToggle() {
  // theme: 現在のユーザー選択（初期値は system）
  const [theme, setTheme] = useState<Theme>("system");
  // mounted: コンポーネントがマウント済みかどうか
  // → SSR（サーバーサイドレンダリング）とCSR（クライアント）で
  //   初期描画が異なるとReactが警告を出すので、それを避けるための仕組み
  const [mounted, setMounted] = useState(false);

  // コンポーネントがマウントされた後にテーマを読み込む
  // （SSR 時にミスマッチ＝ハイドレーションエラーが起きるのを防ぐため）
  useEffect(() => {
    // クライアントで初めて true になる
    setMounted(true);

    // localStorage: ブラウザに永続的に値を保存する仕組み（タブを閉じても消えない）
    // as Theme | null: TypeScript の型アサーション。null も許容
    const savedTheme = localStorage.getItem("theme") as Theme | null;
    if (savedTheme) {
      setTheme(savedTheme);
      applyTheme(savedTheme);
    } else {
      applyTheme("system");
    }
  // [] で初回マウント時のみ実行
  }, []);

  // システムのカラースキーム変更を監視
  // ユーザーが OS の設定でダーク⇔ライトを切り替えたら自動連動する
  useEffect(() => {
    // system モード以外なら監視不要
    if (theme !== "system") return;

    // matchMedia: メディアクエリをJSから扱う API
    // (prefers-color-scheme: dark): OSがダークモード設定かどうか
    const mediaQuery = window.matchMedia("(prefers-color-scheme: dark)");
    const handleChange = () => applyTheme("system");

    // OS設定変更時に handleChange を呼ぶ
    mediaQuery.addEventListener("change", handleChange);
    // クリーンアップ: コンポーネントがアンマウントされる時にリスナー解除
    return () => mediaQuery.removeEventListener("change", handleChange);
  // theme が変わるたびに再実行
  }, [theme]);

  // テーマを実際にDOMに適用する関数
  const applyTheme = (newTheme: Theme) => {
    // <html> 要素を取得
    const root = document.documentElement;
    // isDark: 実際にダークモードにすべきかの判定
    // - "dark" 選択時 → true
    // - "system" 選択時 → OS設定に応じて
    const isDark =
```

```
      newTheme === "dark" ||
      (newTheme === "system" &&
        window.matchMedia("(prefers-color-scheme: dark)").matches);

    // <html> の class に "dark" を付けたり外したり
    if (isDark) {
      root.classList.add("dark");
    } else {
      root.classList.remove("dark");
    }
  };

  // テーマを切り替えるボタンハンドラ
  // クリックするたび light → dark → system → light → ... と循環
  const toggleTheme = () => {
    const nextTheme: Theme =
      theme === "light" ? "dark" : theme === "dark" ? "system" : "light";

    // state 更新
    setTheme(nextTheme);
    // 永続化
    localStorage.setItem("theme", nextTheme);
    // DOM 反映
    applyTheme(nextTheme);
  };

  // マウント前はプレースホルダーを表示（ハイドレーションエラー防止）
  if (!mounted) {
    return (
      <button
        className="
          inline-flex h-10 w-10 items-center justify-center
          rounded-lg
          text-gray-500
        "
        aria-label="テーマ切り替え"
      >
        <div className="h-5 w-5" />
        {/* 中身は空（プレースホルダー） */}
      </button>
    );
  }

  // テーマに応じたアイコン
  const themeIcon = () => {
    switch (theme) {
      case "light":
        return (
          <svg
            className="h-5 w-5"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M12 3v2.25m6.364.386l-1.591 1.591M21 12h-2.25m-.386 6.364l-1.591-1.591M12 18.75V21m-4.773-
4.227l-1.591 1.591M5.25 12H3m4.227-4.773L5.636 5.636M15.75 12a3.75 3.75 0 11-7.5 0 3.75 3.75 0 017.5 0z"
            />
          </svg>
        );
      case "dark":
```

```
        return (
          <svg
            className="h-5 w-5"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M21.752 15.002A9.718 9.718 0 0118 15.75c-5.385 0-9.75-4.365-9.75-9.75 0-1.33.266-
2.597.748-3.752A9.753 9.753 0 003 11.25C3 16.635 7.365 21 12.75 21a9.753 9.753 0 009.002-5.998z"
            />
          </svg>
        );
      case "system":
        return (
          <svg
            className="h-5 w-5"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M9 17.25v1.007a3 3 0 01-.879 2.122L7.5 21h9l-.621-.621A3 3 0 0115 18.257V17.25m6-
12V15a2.25 2.25 0 01-2.25 2.25H5.25A2.25 2.25 0 013 15V5.25m18 0A2.25 2.25 0 0018.75 3H5.25A2.25 2.25 0 003
5.25m18 0V12a2.25 2.25 0 01-2.25 2.25H5.25A2.25 2.25 0 013 12V5.25"
            />
          </svg>
        );
    }
  };

  // テーマのラベル（画面表示と aria-label に使う）
  const themeLabel =
    theme === "light" ? "ライト" : theme === "dark" ? "ダーク" : "システム";

  return (
    <button
      onClick={toggleTheme}
      className="
        inline-flex h-10 items-center justify-center gap-2
        rounded-lg px-3
        text-gray-600
        transition-colors duration-200
        hover:bg-gray-100 hover:text-gray-900
        dark:text-gray-400
        dark:hover:bg-gray-800 dark:hover:text-gray-100
        focus-visible:outline-hidden focus-visible:ring-2
        focus-visible:ring-primary-500 focus-visible:ring-offset-2
      "
      /* ▼ クラスの読み解き ▼
         inline-flex h-10 ...     : 40px高さの横並びボタン
         gap-2                    : アイコンとラベルの間 8px
         rounded-lg px-3          : 角丸 + 左右パディング 12px
         transition-colors        : 色変化をトランジション
         hover/dark バリアントの組み合わせで多状態対応
      */
      aria-label={`テーマ切り替え（現在: ${themeLabel}モード）`}
```

```
      /* マウスを乗せたときのツールチップ */
      title={`${themeLabel}モード`}
    >
      {themeIcon()}
      {/* ラベルは小画面で隠す（hidden）、640px以上で inline 表示 */}
      <span className="hidden text-sm font-medium sm:inline">
        {themeLabel}
      </span>
    </button>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: 保存済みテーマを読み込む（`useEffect` + `localStorage`）

コピー

```
const [theme, setTheme] = useState<Theme>("system");
const [mounted, setMounted] = useState(false);

useEffect(() => {
  setMounted(true);
  const savedTheme = localStorage.getItem("theme") as Theme | null;
  if (savedTheme) {
    setTheme(savedTheme);
    applyTheme(savedTheme);
  } else {
    applyTheme("system");
  }
}, []);
```

- `localStorage` は「**タブを閉じても消えないブラウザの保存場所**」です。`getItem("theme")` で前回選んだテーマを読み出します。
- `mounted` を `true` にするのは、SSR（サーバー描画）とCSR（ブラウザ描画）の食い違い（ハイドレーションエラー）を避けるためです。マウント後にだけ本来の表示をします。
- 依存配列 `[]` なので、この `useEffect` は**初回マウント時に1回だけ**実行されます。

> **用語: localStorage／ハイドレーション** … localStorage はブラウザに永続保存する仕組み。ハイドレーションはサーバー生成HTMLにReactが後から機能を結びつける処理で、両者の表示差は警告の原因になる。

解説2: 実際にダークを適用する（`applyTheme`）

コピー

```
const applyTheme = (newTheme: Theme) => {
  const root = document.documentElement;
  const isDark =
    newTheme === "dark" ||
    (newTheme === "system" &&
      window.matchMedia("(prefers-color-scheme: dark)").matches);
  if (isDark) {
    root.classList.add("dark");
  } else {
    root.classList.remove("dark");
  }
};
```

- `document.documentElement` は `<html>` 要素そのものです。ここに `dark` クラスを付け外しすると、globals.css のCSS変数が一斉に切り替わります。
- `isDark` の判定は「**`dark` が選ばれている、または `system` かつOSがダーク設定**」のとき `true`。`matchMedia("(prefers-color-scheme: dark)")` でOSの設定を読み取ります。
- `classList.add("dark")` / `remove("dark")` でダークモードのオン・オフを切り替えます。

> **用語: matchMedia / prefers-color-scheme** … `matchMedia` はメディアクエリをJSから判定するAPI。`prefers-color-scheme: dark` はOSがダーク表示を望んでいるかを表す。

解説3: クリックでテーマを循環させる（`toggleTheme`）

コピー

```
const toggleTheme = () => {
  const nextTheme: Theme =
    theme === "light" ? "dark" : theme === "dark" ? "system" : "light";
  setTheme(nextTheme);
  localStorage.setItem("theme", nextTheme);
  applyTheme(nextTheme);
};
```

- 三項演算子を2つつなげて「light → dark → system → light → …」と**順番に循環**させています。
- `setTheme` で画面表示を更新し、`localStorage.setItem` で選択を保存（次回も同じテーマで開ける）、`applyTheme` で実際の `<html>` に反映、の3つを行います。
- このように「状態更新・永続化・DOM反映」をまとめて行うのがテーマ切替の基本形です。

> **用語: 三項演算子のネスト** … `A ? x : B ? y : z` のように三項演算子を連ねた書き方。3つ以上の分岐を1行で表せるが、深くしすぎると読みにくくなる。

解説4: マウント前はプレースホルダーを返す

コピー

```
if (!mounted) {
  return (
    <button className="inline-flex h-10 w-10 ..." aria-label="テーマ切り替え">
      <div className="h-5 w-5" />
    </button>
  );
}
```

- `if (!mounted)` は「まだクライアントでマウントされていない（＝サーバー描画段階）」のときの分岐です。
- ここで**中身が空のボタン（プレースホルダー）**を返すことで、サーバーとブラウザで初期表示を一致させ、ハイドレーションエラーを防ぎます。
- マウント後は `mounted` が `true` になり、テーマに応じたアイコン入りのボタンが描画されます。

> **用語: プレースホルダー（placeholder）** … 本来の中身が決まる前に置いておく仮の表示。ここではレイアウトのずれや警告を避けるための空ボタンとして使う。

ちらつき防止のスクリプト:

ページ読み込み時にダークモードが一瞬ライトモードで表示されるのを防ぐため、`<head>` 内にインラインスクリプトを設置します。

> **▼ このコードがやること（先に日本語で）**：ページを開いた一瞬だけダークモードがライト表示でちらつく問題を防ぐ小さなスクリプトを、`<head>` の中に仕込みます。ポイントは、画面が描かれる「前」に保存済みのテーマ設定を読み取り、すぐ `<html>` に `dark` クラスを付けてしまうこと。React の描画を待たずに先回りするため、あえて `<head>` 内の即時実行スクリプトとして書きます。

コピー

```
// app/layout.tsx の <head> 内に追加

// dangerouslySetInnerHTML: React の機能。普通は危険なので使うべきではないが、
// React のハイドレーション前に実行するインラインスクリプトを書きたい時に使う
<script
  dangerouslySetInnerHTML={{
    __html: `
      /* 即時実行関数（IIFE）: 定義と同時に呼び出す関数。グローバル変数を汚さない */
      (function() {
        try {
          /* localStorage から保存済みテーマを取得 */
          var theme = localStorage.getItem('theme');
          /* isDark の判定:
             - 明示的に 'dark' が保存されている
             - または theme 未保存 かつ OS がダーク設定 */
          var isDark = theme === 'dark' ||
            (!theme && window.matchMedia('(prefers-color-scheme: dark)').matches);
          if (isDark) {
            /* <html> に dark クラスを付与（React の描画前に間に合う） */
            document.documentElement.classList.add('dark');
          }
        } catch (e) {
          /* localStorage が使えないブラウザ等のためのフォールバック（何もしない） */
        }
      })();
    `,
  }}
/>
```

6. アクセシビリティ

6.1 aria 属性の追加

アクセシビリティを確保するために、各コンポーネントに適切な ARIA 属性を追加します。

ARIA（Accessible Rich Internet Applications、エイリア） とは、HTMLだけでは表現しきれない要素の意味や状態をスクリーンリーダー（screen reader：画面の内容を音声で読み上げるソフト）に伝えるための仕様です。`role`、`aria-label`、`aria-expanded` などの属性を使います。

▼ **このコードがやること（先に日本語で）**：サイト上部のヘッダーに、目の見えない方が使う読み上げソフト（スクリーンリーダー）向けの情報を足します。カギは `aria-label`（要素の意味を言葉で補う）や `aria-expanded`（メニューが開いているか）などの ARIA 属性で、見た目を変えずに「ここはナビゲーション」「このボタンで開閉する」と伝えられます。どの属性が何を伝えるかはコメントを参照してください。

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
// components/Header.tsx
// ↑ サイト全体のヘッダー（上部の固定ナビゲーション）。

"use client";

import { useState } from "react";
import Link from "next/link";
// 現在のURLパス取得
import { usePathname } from "next/navigation";
import { ThemeToggle } from "./ThemeToggle";

export function Header() {
  const pathname = usePathname();
  // モバイルメニューの開閉状態
  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false);

  // ナビゲーションアイテムを配列で管理。後でmapで展開する
  const navItems = [
    // 書籍一覧はトップページ（/）に置いている
    { href: "/", label: "書籍一覧" },
    { href: "/books/new", label: "新規登録" },
  ];

  return (
    <header
      className="
        sticky top-0 z-40
        border-b border-[var(--color-border)]
        glass
      "
      /* ▼ クラスの読み解き ▼
         sticky top-0    : スクロールしても上端に貼り付く
         z-40            : z-index 40（他要素より前面、トーストの50よりは後ろ）
         border-b        : 下側だけ枠線
         glass           : すりガラス効果（globals.css 定義）
      */
      /* ARIA: サイトのバナー領域（ヘッダー）であることを伝える */
      role="banner"
    >
      <div className="container flex h-16 items-center justify-between">
        {/* container: globals.css の @utility container（中央寄せ＋余白）
            flex h-16 items-center justify-between:
              横並び、高さ64px、縦中央、両端寄せ */}

        {/* ロゴ */}
        <Link
          href="/"
          className="
            flex items-center gap-2
            text-xl font-bold
            text-[var(--color-foreground)]
            transition-colors hover:text-primary-600
            dark:hover:text-primary-400
          "
          /* スクリーンリーダー用 */
          aria-label="BookShelf ホームへ"
        >
          <svg
            className="h-7 w-7 text-primary-600 dark:text-primary-400"
            fill="none"
            viewBox="0 0 24 24"
            stroke="currentColor"
```

```
            strokeWidth={2}
            aria-hidden="true"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              d="M12 6.042A8.967 8.967 0 006 3.75c-1.052 0-2.062.18-3 .512v14.25A8.987 8.987 0 016 18c2.305
0 4.408.867 6 2.292m0-14.25a8.966 8.966 0 016-2.292c1.052 0 2.062.18 3 .512v14.25A8.987 8.987 0 0018
18a8.967 8.967 0 00-6 2.292m0-14.25v14.25"
            />
          </svg>
          BookShelf
        </Link>

        {/* デスクトップナビゲーション */}
        <nav
          className="hidden items-center gap-1 md:flex"
          /* hidden    : 通常は非表示（モバイル時） */
          /* md:flex   : 768px以上で flex 表示に切替 */
          role="navigation"
          aria-label="メインナビゲーション"
        >
          {navItems.map((item) => {
            // 現在のパスと一致するかを判定
            const isActive = pathname === item.href;
            return (
              <Link
                key={item.href}
                href={item.href}
                className={`
                  rounded-lg px-3 py-2 text-sm font-medium
                  transition-colors duration-200
                  ${
                    isActive
                      ? "bg-primary-50 text-primary-700 dark:bg-primary-950 dark:text-primary-300"
                      : "text-gray-600 hover:bg-gray-100 hover:text-gray-900 dark:text-gray-400
dark:hover:bg-gray-800 dark:hover:text-gray-100"
                  }
                `}
                /* isActive ? 強調表示 : 通常表示 で切り替え */
                aria-current={isActive ? "page" : undefined}
                /* aria-current="page": スクリーンリーダーに「これが現在のページ」と伝える */
              >
                {item.label}
              </Link>
            );
          })}
        </nav>

        {/* 右側のアクション */}
        <div className="flex items-center gap-2">
          <ThemeToggle />

          {/* モバイルメニューボタン（ハンバーガー） */}
          <button
            className="
              inline-flex h-10 w-10 items-center justify-center
              rounded-lg
              text-gray-600
              transition-colors
              hover:bg-gray-100
              dark:text-gray-400 dark:hover:bg-gray-800
              md:hidden
            "
            /* md:hidden: 768px以上で非表示（モバイル専用ボタン） */
```

```
            onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}
            /* メニューが展開中かをスクリーンリーダーに通知 */
            aria-expanded={isMobileMenuOpen}
            /* このボタンが操作する対象要素のid */
            aria-controls="mobile-menu"
            aria-label={isMobileMenuOpen ? "メニューを閉じる" : "メニューを開く"}
          >
            {isMobileMenuOpen ? (
              <svg
                className="h-6 w-6"
                fill="none"
                viewBox="0 0 24 24"
                stroke="currentColor"
                strokeWidth={2}
                aria-hidden="true"
              >
                <path strokeLinecap="round" strokeLinejoin="round" d="M6 18L18 6M6 6l12 12" />
              </svg>
            ) : (
              <svg
                className="h-6 w-6"
                fill="none"
                viewBox="0 0 24 24"
                stroke="currentColor"
                strokeWidth={2}
                aria-hidden="true"
              >
                <path strokeLinecap="round" strokeLinejoin="round" d="M3.75 6.75h16.5M3.75 12h16.5m-16.5
5.25h16.5" />
              </svg>
            )}
          </button>
        </div>
      </div>

      {/* モバイルメニュー */}
      {isMobileMenuOpen && (
        <nav
          id="mobile-menu"
          className="
            border-t border-[var(--color-border)]
            bg-[var(--color-background)]
            p-4
            md:hidden
          "
          role="navigation"
          aria-label="モバイルナビゲーション"
        >
          <ul className="flex flex-col gap-1" role="list">
            {navItems.map((item) => {
              const isActive = pathname === item.href;
              return (
                <li key={item.href}>
                  <Link
                    href={item.href}
                    className={`
                      block rounded-lg px-4 py-3 text-sm font-medium
                      transition-colors duration-200
                      ${
                        isActive
                          ? "bg-primary-50 text-primary-700 dark:bg-primary-950 dark:text-primary-300"
                          : "text-gray-600 hover:bg-gray-50 dark:text-gray-400 dark:hover:bg-gray-800"
                      }
                    `}
                    aria-current={isActive ? "page" : undefined}
```

```
                    onClick={() => setIsMobileMenuOpen(false)}
                  >
                    {item.label}
                  </Link>
                </li>
              );
            })}
          </ul>
        </nav>
      )}
    </header>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: ヘッダーの領域を読み上げソフトに伝える（`role="banner"`）

コピー

```
<header
  className="sticky top-0 z-40 border-b border-[var(--color-border)] glass"
  role="banner"
>
```

- `sticky top-0` は「スクロールしても**画面上端に貼り付く**」配置。`z-40` で他の要素より前面（ただしトーストの `z-50` よりは後ろ）に置きます。
- `glass` は globals.css のすりガラス効果（半透明＋背景ぼかし）です。
- `role="banner"` は「ここはサイトのヘッダー領域だ」とスクリーンリーダーに伝えるARIAロール。見た目は変わらず、意味だけを補います。

> **用語: ARIA role（banner）** … 要素の役割を読み上げソフトに伝える属性。`banner` はページ上部のヘッダー（ロゴやナビ）領域を表す。

解説2: ナビ配列を map し、現在地を強調（`aria-current`）

コピー

```
{navItems.map((item) => {
  const isActive = pathname === item.href;
  return (
    <Link
      key={item.href}
      href={item.href}
      className={`rounded-lg px-3 py-2 ... ${
        isActive
          ? "bg-primary-50 text-primary-700 dark:bg-primary-950 dark:text-primary-300"
          : "text-gray-600 hover:bg-gray-100 ..."
      }`}
      aria-current={isActive ? "page" : undefined}
    >
      {item.label}
    </Link>
  );
})}
```

- `navItems` 配列を `map` で展開し、各リンクを作ります。`isActive = pathname === item.href` で「今いるページか」を判定します。
- 三項演算子で、現在ページのリンクだけ**背景色付きで強調**し、他は通常色＋ホバー時の薄い背景にしています。
- `aria-current={isActive ? "page" : undefined}` は、現在ページにだけ「これが現在地」と読み上げソフトに伝えます。該当しなければ `undefined`（属性を付けない）にします。

> **用語: aria-current="page"** … 同種リンク群の中で「今いるページ」を示すARIA属性。視覚的な強調と合わせて使うとアクセシブルになる。

解説3: ハンバーガーボタンの開閉状態を伝える（`aria-expanded` / `aria-controls`）

コピー

```
<button
  onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}
  aria-expanded={isMobileMenuOpen}
  aria-controls="mobile-menu"
  aria-label={isMobileMenuOpen ? "メニューを閉じる" : "メニューを開く"}
>
```

- `onClick={() => setIsMobileMenuOpen(!isMobileMenuOpen)}` は「**現在の開閉状態を反転**」させる処理。`!` で true↔false を切り替えます。

- `aria-expanded={isMobileMenuOpen}` は「メニューが今**開いているか閉じているか**」を読み上げソフトに伝えます。`aria-controls="mobile-menu"` は「このボタンが操作する対象のid」を示します。
- `aria-label` も開閉状態で文言を変え、ボタンの目的を明確にしています。

> **用語: aria-expanded / aria-controls** … `aria-expanded` は開閉式の要素が今開いているかを示す。`aria-controls` はそのボタンが制御する要素のidを結びつける。

解説4: モバイルメニューを条件付きで表示

コピー

```
{isMobileMenuOpen && (
  <nav id="mobile-menu" className="border-t ... md:hidden" role="navigation" aria-label="モバイルナビゲーション">
    <ul className="flex flex-col gap-1" role="list">
      {navItems.map((item) => (
        // ... 各リンク。onClick で setIsMobileMenuOpen(false)
      ))}
    </ul>
  </nav>
)}
```

- `{isMobileMenuOpen && (...)}` で「開いているときだけ」メニューを描画します（短絡評価）。
- `id="mobile-menu"` は解説3の `aria-controls` と対応し、`md:hidden` で「768px以上（PC）では非表示」にします。モバイル専用のメニューです。
- 各リンクの `onClick` で `setIsMobileMenuOpen(false)` を呼び、**リンクを押したらメニューが閉じる**ようにしています。

> **用語: 条件付きレンダリング（`条件 && JSX`）** … 条件が真のときだけ要素を描く書き方。開いているときだけメニューを出すなどの出し分けに使う。

6.2 キーボードナビゲーション

キーボードナビゲーション（keyboard navigation：マウスを使わずキーボードだけで操作できるようにすること） は、視覚障害のあるユーザーや、マウスが使いづらい状況のユーザーにとって必須の機能です。

主な操作:

- **Tab キー**: 次のフォーカス可能要素へ移動
- **Shift + Tab**: 前のフォーカス可能要素へ移動
- **Enter / Space**: 選択中の要素を実行（クリック相当）
- **Escape**: モーダルやポップアップを閉じる
- **矢印キー**: メニュー内の項目選択（必要に応じて自分で実装）

フォーカストラップ（focus trap：フォーカスを特定範囲内に閉じ込める仕組み） は、モーダル表示中に Tab キーで背景の要素にフォーカスが飛んでしまうのを防ぐテクニックです。

> **▼ このコードがやること（先に日本語で）**：マウスを使わずキーボードだけで操作できるようにする対応を、カードに足します。ポイントは、Tab キーで要素を順にフォーカスでき、Enter/Space で実行、Escape で閉じる、という基本操作に対応すること。さらにモーダル表示中はフォーカスを枠内に閉じ込める「フォーカストラップ」で、背景に迷子にならないようにします。

コピー

```
// components/BookCard.tsx に追加するキーボード対応

// カードが Link で囲まれている場合、Enter/Space で遷移が可能（Link のデフォルト動作）
// それ以外のインタラクティブ要素にも対応する

// 例: 削除確認ダイアログのキーボード対応
// components/ConfirmDialog.tsx
// ↑ モーダル（modal：背景を暗くして他操作を遮るダイアログ）型の確認ボックス。
//   削除など重要操作の前にユーザーに最終確認を求める用途。

"use client";

import { useEffect, useRef, type ReactNode } from "react";

type ConfirmDialogProps = {
  // ダイアログを表示するかどうか
  isOpen: boolean;
  // タイトル
  title: string;
  // 本文メッセージ
  message: string;
  // 確認ボタンの文字（既定: "確認"）
  confirmLabel?: string;
  // キャンセルボタンの文字（既定: "キャンセル"）
  cancelLabel?: string;
  // 確認ボタンの色
  variant?: "danger" | "primary";
  // 確認時のコールバック
  onConfirm: () => void;
  // キャンセル時のコールバック
  onCancel: () => void;
};

export function ConfirmDialog({
  isOpen,
  title,
  message,
  confirmLabel = "確認",
  cancelLabel = "キャンセル",
  variant = "primary",
  onConfirm,
  onCancel,
}: ConfirmDialogProps) {
  // useRef: DOM要素への参照を保持する Hook
  // .current で実際の DOM ノードにアクセスできる
  // ダイアログ本体への参照
  const dialogRef = useRef<HTMLDivElement>(null);
  // キャンセルボタンへの参照
  const cancelButtonRef = useRef<HTMLButtonElement>(null);
```

```
// ダイアログが開いたらキャンセルボタンにフォーカス
// ※ 確認ボタンではなくキャンセルに当てるのは「誤操作で確定しない」ための配慮
useEffect(() => {
  if (isOpen) {
    // ?. は null/undefined チェック付きアクセス
    cancelButtonRef.current?.focus();
  }
}, [isOpen]);

// Escape キーで閉じる + Tab キーでのフォーカストラップ（ダイアログ内に閉じ込める）
useEffect(() => {
  const handleKeyDown = (e: KeyboardEvent) => {
    if (!isOpen) return;

    // Escape キーでキャンセル
    if (e.key === "Escape") {
      onCancel();
    }

    // Tab キーのフォーカストラップ
    // → Tab/Shift+Tab を押してもダイアログの外に出ないようにする
    if (e.key === "Tab" && dialogRef.current) {
      // ダイアログ内でフォーカス可能な要素を全て取得
      // querySelectorAll: CSS セレクタで子要素を検索する API
      const focusableElements = dialogRef.current.querySelectorAll<HTMLElement>(
        'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])'
      );
      // 最初
      const firstElement = focusableElements[0];
      // 最後
      const lastElement = focusableElements[focusableElements.length - 1];

      if (e.shiftKey) {
        // Shift+Tab: 逆方向に移動
        // 最初の要素でさらに戻ろうとしたら、最後の要素にラップ
        if (document.activeElement === firstElement) {
          e.preventDefault();
          lastElement.focus();
        }
      } else {
        // Tab: 順方向
        // 最後の要素でさらに進もうとしたら、最初の要素にラップ
        if (document.activeElement === lastElement) {
          e.preventDefault();
          firstElement.focus();
        }
      }
    }
  };
```

```
    // document 全体で keydown を監視
    document.addEventListener("keydown", handleKeyDown);
    return () => document.removeEventListener("keydown", handleKeyDown);
  }, [isOpen, onCancel]);

  // body のスクロールを無効化
  // モーダル表示中に背景をスクロールできてしまうと混乱するため
  useEffect(() => {
    if (isOpen) {
      // スクロール禁止
      document.body.style.overflow = "hidden";
    } else {
      // 元に戻す
      document.body.style.overflow = "";
    }
    return () => {
      // クリーンアップでも復元
      document.body.style.overflow = "";
    };
  }, [isOpen]);

  // 閉じている時は何も描画しない
  if (!isOpen) return null;

  return (
    <div
      className="
        fixed inset-0 z-50
        flex items-center justify-center
        p-4
      "
      /* fixed inset-0 : 画面全体に固定（top:0 right:0 bottom:0 left:0）
         z-50         : 最前面
         flex ... center: 中身を縦横中央に
         p-4          : 画面端からの余白 */
      role="dialog"
      /* 背景操作を遮るモーダルであると伝える */
      aria-modal="true"
      /* タイトルとなる要素のid */
      aria-labelledby="dialog-title"
      /* 説明文となる要素のid */
      aria-describedby="dialog-description"
    >
      {/* オーバーレイ（背景を暗くする半透明の幕） */}
      <div
        className="
          absolute inset-0
          bg-black/50
          animate-fade-in
          backdrop-blur-sm
        "
```

```
        /* bg-black/50      : 黒の透明度50%
           backdrop-blur-sm : 背景を少しぼかす */
        /* オーバーレイクリックでキャンセル */
        onClick={onCancel}
        /* スクリーンリーダーから隠す（装飾要素） */
        aria-hidden="true"
      />

      {/* ダイアログ本体 */}
      <div
        /* DOMへの参照を取得 */
        ref={dialogRef}
        className="
          relative
          w-full max-w-md
          animate-scale-in
          rounded-xl
          bg-[var(--color-card)]
          p-6
          shadow-xl
        "
        /* relative                  : オーバーレイより手前に重ねる
           w-full max-w-md           : 幅100%（ただし最大448px）
           animate-scale-in          : 拡大しながら表示
           rounded-xl                : 大きめ角丸
           bg-[var(--color-card)]    : カード背景色（ダーク対応） */
      >
        <h2
          /* aria-labelledby で参照される */
          id="dialog-title"
          className="text-lg font-semibold text-[var(--color-foreground)]"
        >
          {title}
        </h2>

        <p
          /* aria-describedby で参照される */
          id="dialog-description"
          className="mt-2 text-sm text-[var(--color-muted)]"
        >
          {message}
        </p>

        <div className="mt-6 flex justify-end gap-3">
          {/* mt-6: 上マージン 24px / justify-end: 右寄せ / gap-3: ボタン間 12px */}
          <button
            /* ボタンへの参照（初期フォーカス用） */
            ref={cancelButtonRef}
            onClick={onCancel}
            /* 控えめなボタン */
            className="btn-ghost"
```

```
          >
            {cancelLabel}
          </button>

          <button
            onClick={onConfirm}
            /* variant に応じて btn-danger（赤）または btn-primary（インディゴ） */
            className={variant === "danger" ? "btn-danger" : "btn-primary"}
          >
            {confirmLabel}
          </button>
        </div>
      </div>
    </div>
  );
}
```

▼ コードを1つずつ分解して解説

解説1: DOM要素への参照を持つ（`useRef`）

コピー

```
const dialogRef = useRef<HTMLDivElement>(null);
const cancelButtonRef = useRef<HTMLButtonElement>(null);

useEffect(() => {
  if (isOpen) {
    cancelButtonRef.current?.focus();
  }
}, [isOpen]);
```

- `useRef` は「**実際のDOM要素への参照**」を保持するHookです。`.current` でその要素にアクセスできます。`<div ref={dialogRef}>` のように要素に紐付けます。
- ダイアログが開いたら `cancelButtonRef.current?.focus()` で**キャンセルボタンに自動でフォーカス**を当てます。`?.` は「nullでなければ実行」の安全アクセスです。
- 確認ボタンではなくキャンセルに当てるのは、Enter連打などで**誤って確定させない**ための配慮です。

> **用語: useRef** … 再描画を起こさずに値やDOM参照を保持するReact Hook。`.current` で中身を読み書きする。

解説2: Escapeで閉じ、Tabを閉じ込める（フォーカストラップ）

コピー

```
const handleKeyDown = (e: KeyboardEvent) => {
  if (!isOpen) return;
  if (e.key === "Escape") { onCancel(); }
  if (e.key === "Tab" && dialogRef.current) {
    const focusableElements = dialogRef.current.querySelectorAll<HTMLElement>(
      'button, [href], input, select, textarea, [tabindex]:not([tabindex="-1"])'
    );
    const firstElement = focusableElements[0];
    const lastElement = focusableElements[focusableElements.length - 1];
    if (e.shiftKey) {
      if (document.activeElement === firstElement) { e.preventDefault(); lastElement.focus(); }
    } else {
      if (document.activeElement === lastElement) { e.preventDefault(); firstElement.focus(); }
    }
  }
};
```

- `e.key === "Escape"` のとき `onCancel()` を呼び、**Escapeキーで閉じられる**ようにします。
- Tabキーのときは、`querySelectorAll` でダイアログ内の**フォーカス可能な要素を全部集め**、最初と最後を取り出します。
- 最後の要素でさらにTabを押したら最初へ、最初の要素でShift+Tabを押したら最後へ「ラップ」させます。これで**フォーカスがダイアログの外に逃げない**＝フォーカストラップが完成します。

> **用語: フォーカストラップ（focus trap）** … モーダル表示中、Tab移動の対象を枠内に閉じ込める仕組み。背景の要素に誤ってフォーカスが移るのを防ぐ。

解説3: 背景のスクロールを止める（`document.body.style.overflow`）

コピー

```
useEffect(() => {
  if (isOpen) {
    document.body.style.overflow = "hidden";
  } else {
    document.body.style.overflow = "";
  }
  return () => {
    document.body.style.overflow = "";
  };
}, [isOpen]);
```

- ダイアログが開いている間 `document.body.style.overflow = "hidden"` で**ページ全体のスクロールを禁止**します。モーダルの後ろが動くと混乱するためです。

- 閉じたら `""`（空文字）に戻して、元のスクロール可能な状態に復帰させます。
- `return () => { ... }` のクリーンアップでも復元しておくことで、コンポーネントが消えてもスクロールが固まったままにならないようにします。

> **用語: クリーンアップ関数** … `useEffect` が返す関数。次の実行前やアンマウント時に呼ばれ、イベント解除やスタイル復元などの後片付けを担う。

解説4: モーダルの意味づけ（`role="dialog"` / `aria-modal`）

コピー

```
<div
  role="dialog"
  aria-modal="true"
  aria-labelledby="dialog-title"
  aria-describedby="dialog-description"
>
  <div className="... bg-black/50 ..." onClick={onCancel} aria-hidden="true" />
  <div ref={dialogRef} className="relative ...">
    <h2 id="dialog-title">{title}</h2>
    <p id="dialog-description">{message}</p>
    {/* ... ボタン ... */}
  </div>
</div>
```

- `role="dialog"` と `aria-modal="true"` で「これは**背景操作を遮るモーダルダイアログ**だ」とスクリーンリーダーに伝えます。
- `aria-labelledby="dialog-title"` / `aria-describedby="dialog-description"` は、見出し（`id="dialog-title"`）と説明文（`id="dialog-description"`）を**ダイアログの名前・説明として結びつけます**。
- 背景の幕（`bg-black/50`）は `onClick={onCancel}` で「外側クリックで閉じる」、`aria-hidden="true"` で読み上げ対象から除外しています。

> **用語: role="dialog" / aria-modal** … ダイアログであることと、背景が操作不可のモーダルであることを伝えるARIA。`aria-labelledby / describedby` で見出し・説明を関連付ける。

6.3 色のコントラスト

WCAG（Web Content Accessibility Guidelines、ダブリュー・シー・エー・ジー：W3C が策定するWebアクセシビリティ指針） 2.1 のコントラスト比（色の明暗差）基準を満たすための指針です。コントラストが弱いと、視覚障害のあるユーザーや明るい場所でスマホを見るユーザーが文字を読み取れなくなります。

レベル	コントラスト比	対象
AA（通常テキスト）	4.5:1 以上	本文、ラベル、プレースホルダ
AA（大きいテキスト）	3:1 以上	見出し（18px以上、または14px太字以上）
AAA（通常テキスト）	7:1 以上	最高レベルの可読性が求められる場面

本アプリで採用しているカラーのコントラスト比:

組み合わせ	ライトモード	ダークモード	基準
メインテキスト / 背景	17.9:1 (#0f172a / #ffffff)	17.1:1 (#f8fafc / #0f172a)	AA
ミュートテキスト / 背景	4.8:1 (#64748b / #ffffff)	7.0:1 (#94a3b8 / #0f172a)	AA
プライマリボタン文字 / ボタン背景	6.3:1 (#ffffff / #4f46e5)	6.3:1（ライトと同じ色）	AA
エラーテキスト / エラー背景	7.6:1 (#991b1b / #fef2f2)	5.3:1 (#fca5a5 / #7f1d1d)	AA

> **注意:** 上の表の数値は、下の「試してみよう」のプログラムで計算し直した値です（以前の版の数値には誤りがあったため修正しました）。どの組み合わせも AA の基準を満たしている、という結論は変わりません。

確認ツール:

- Chrome DevTools: Elements パネルの Color Contrast 表示
- WebAIM Contrast Checker
- axe DevTools ブラウザ拡張

数字だけだとピンと来ないので、実際の色で並べてみます。



図9-17: コントラスト比の見本（数字が大きいほど、文字と背景の明るさの差が大きく読みやすい）

試してみよう: コントラスト比を自分で計算する

> **▼ このコードがやること（先に日本語で）**：「文字の色」と「背景の色」の16進数カラーを受け取り、WCAG（Webのアクセシビリティ指針）が決めている計算式でコントラスト比を求めます。本アプリで使っている色の組み合わせを一度に調べ、4.5:1 以上なら「合格」と表示します。上の表の数値は、このプログラムで計算したものです。

コピー

```
// contrast.ts
// "#4f46e5" のような16進数カラーを、0〜1 の「明るさ（相対輝度）」に変える
function luminance(hex: string): number {
  // "#4f46e5" → [0x4f, 0x46, 0xe5] → 0〜1 の範囲に直す
  const [r, g, b] = [1, 3, 5].map(
    (i) => parseInt(hex.slice(i, i + 2), 16) / 255,
  );
  // 人の目の感じ方に合わせて補正する（WCAG の計算式）
  const linear = (c: number) =>
    c <= 0.04045 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
  // 緑を一番明るく、青を一番暗く感じるので重みを変えて足す
  return 0.2126 * linear(r) + 0.7152 * linear(g) + 0.0722 * linear(b);
}

// 2色のコントラスト比（1〜21）を求める
function contrast(text: string, background: string): number {
  const a = luminance(text);
  const b = luminance(background);
  const light = Math.max(a, b);
  const dark = Math.min(a, b);
  return (light + 0.05) / (dark + 0.05);
}

// 調べたい「文字色 / 背景色」の組み合わせ
const pairs: [string, string, string][] = [
  ["本文（ライト）", "#0f172a", "#ffffff"],
  ["本文（ダーク）", "#f8fafc", "#0f172a"],
  ["補足文（ライト）", "#64748b", "#ffffff"],
  ["補足文（ダーク）", "#94a3b8", "#0f172a"],
  ["ボタン文字", "#ffffff", "#4f46e5"],
  ["エラー文字", "#991b1b", "#fef2f2"],
  ["エラー文字（ダーク）", "#fca5a5", "#7f1d1d"],
  ["白文字 / blue-500 のボタン", "#ffffff", "#3b82f6"],
  ["薄すぎる灰色の文字", "#9ca3af", "#ffffff"],
];

for (const [label, text, background] of pairs) {
  const ratio = contrast(text, background);
  // 本文の合格ラインは 4.5:1 以上（WCAG 2.1 AA）
  const result = ratio >= 4.5 ? "合格" : "不合格";
  // 比率を右寄せ5桁で揃えてから、判定と名前を後ろに並べる
  console.log(`${ratio.toFixed(2).padStart(5, " ")}:1  ${result}  ${label}`);
}
```

▼ **実行結果**（`node contrast.ts` で実行）

コピー

```
17.85:1  合格  本文（ライト）
17.06:1  合格  本文（ダーク）
 4.76:1  合格  補足文（ライト）
 6.96:1  合格  補足文（ダーク）
 6.29:1  合格  ボタン文字
 7.60:1  合格  エラー文字
 5.28:1  合格  エラー文字（ダーク）
 3.68:1  不合格  白文字 / blue-500 のボタン
 2.54:1  不合格  薄すぎる灰色の文字
```

▼ コードを1つずつ分解して解説

解説1: 16進数カラーを 0〜1 の数に直す

コピー

```
const [r, g, b] = [1, 3, 5].map(
  (i) => parseInt(hex.slice(i, i + 2), 16) / 255,
);
```

- `"#4f46e5"` の1〜2文字目が赤、3〜4文字目が緑、5〜6文字目が青の強さです。`hex.slice(i, i + 2)` で2文字ずつ取り出し、`parseInt(…, 16)` で16進数を普通の数（0〜255）に直します。
- 最後に `/ 255` で 0〜1 の範囲にそろえます。`[1, 3, 5].map(…)` で、赤・緑・青の3つを一度に計算しています。

解説2: 人の目の感じ方に合わせて「明るさ」を出す

コピー

```
const linear = (c: number) =>
  c <= 0.04045 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4;
return 0.2126 * linear(r) + 0.7152 * linear(g) + 0.0722 * linear(b);
```

- 画面の色の数値と、人が感じる明るさは比例しません。`linear` はその「ずれ」を直す WCAG の決まった式です（`**` は「〜乗」）。
- 人の目は**緑をいちばん明るく、青をいちばん暗く**感じるので、緑に 0.7152、青に 0.0722 という重みを掛けて足します。これが「相対輝度」と呼ばれる明るさです。

> **用語: 相対輝度（そうたいきど、relative luminance）** … 色の明るさを「真っ黒 = 0、真っ白 = 1」の範囲で表した数。コントラスト比の計算に使う。

解説3: 明るいほうと暗いほうの比を取る

コピー

```
const light = Math.max(a, b);
const dark = Math.min(a, b);
return (light + 0.05) / (dark + 0.05);
```

- 2色のうち明るいほうを分子、暗いほうを分母にして割り算します。`+ 0.05` は、真っ黒（0）で割ってしまわないための決まりの数です。
- 結果は「白と黒」の **21:1** が最大、「同じ色どうし」の **1:1** が最小になります。実行結果を見ると、`blue-500` の背景に白い文字は 3.68:1 しかなく、ふつうの大きさの文字には足りないことが分かります。

> **つまずきポイント:「おしゃれに見えるから」と薄い色を選んでしまう**
>
> 薄い灰色の文字（ `text-gray-400` など）や、`bg-blue-500` に白い小さな文字は、すっきりして見えますが**コントラストが足りません**。明るい屋外でスマホを見る人や、視力の弱い人には読めなくなります。補足の文字は `text-gray-500`（4.8:1）より濃い色を、白い文字を載せるボタンは `600` 以上の背景を選びましょう。

コピー

```
<!-- NG: 薄い灰色の文字（2.5:1）と、blue-500 に白い小さな文字（3.7:1） -->
<p class="text-gray-400">2024年に登録</p>
<button class="bg-blue-500 text-sm text-white">保存</button>
```

コピー

```
<!-- OK: 補足文は gray-600（7.6:1）、ボタンは blue-600（5.2:1）にする -->
<p class="text-gray-600">2024年に登録</p>
<button class="bg-blue-600 text-sm text-white">保存</button>
```

7. 最終的な画面の説明

7.1 トップページ（書籍一覧）



ヘッダー:

- 画面上部に `sticky` で固定。スクロールしても常に表示される
- `glass` クラスによる半透明のすりガラス効果（`backdrop-blur-md`）
- 左にロゴ（本のアイコン + "BookShelf"）、中央にナビゲーションリンク、右にテーマ切り替えボタン

- モバイルではハンバーガーメニューに切り替わる

メインコンテンツ:

- ページタイトルと書籍数を表示。右端に「新規登録」ボタン
- 検索バーとフィルター（ステータス、並び順）
- 書籍カードがレスポンシブグリッドで並ぶ。各カードはホバーで浮き上がるアニメーション付き
- カードにはサムネイル、タイトル（最大2行で省略）、著者名、星評価を表示

フッター:

- 著作権表示、外部リンク

7.2 新規登録ページ



レイアウト:

- フォームは `max-w-2xl` で中央配置。カードスタイルで背景と分離
- 「← 書籍一覧に戻る」のパンくずリンクをフォーム上部に配置
- 必須フィールドには `*` マーク。バリデーションエラー時はフィールド下に赤文字でメッセージ表示

- 評価は星アイコンをクリックして選択するインタラクティブ UI
- 「登録する」ボタンは送信中にローディングスピナーを表示
- 送信成功時にトースト通知が右上に表示される

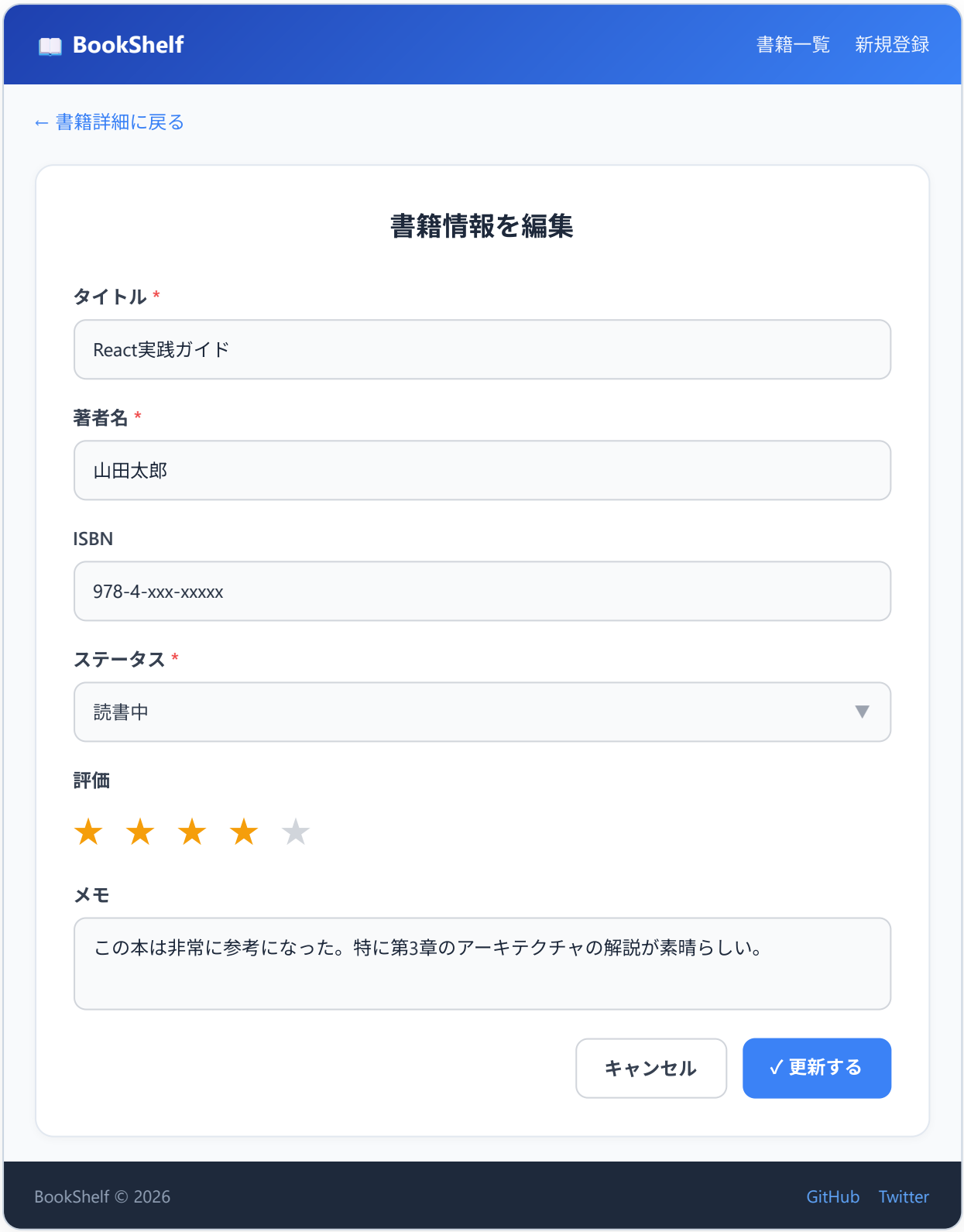
7.3 詳細ページ



レイアウト:

- 上段は2カラム: 左に表紙画像（`aspect-[3/4]`）、右に書籍情報
- モバイルでは1カラムに切り替わり、画像が上、情報が下に配置される
- ステータスバッジが右上に表示される
- 「編集」ボタン（`btn-outline`）と「削除」ボタン（`btn-danger`）を横に並べる
- 削除ボタン押下時は `ConfirmDialog` が表示される
- メモは別のカードセクションとして表示

7.4 編集ページ



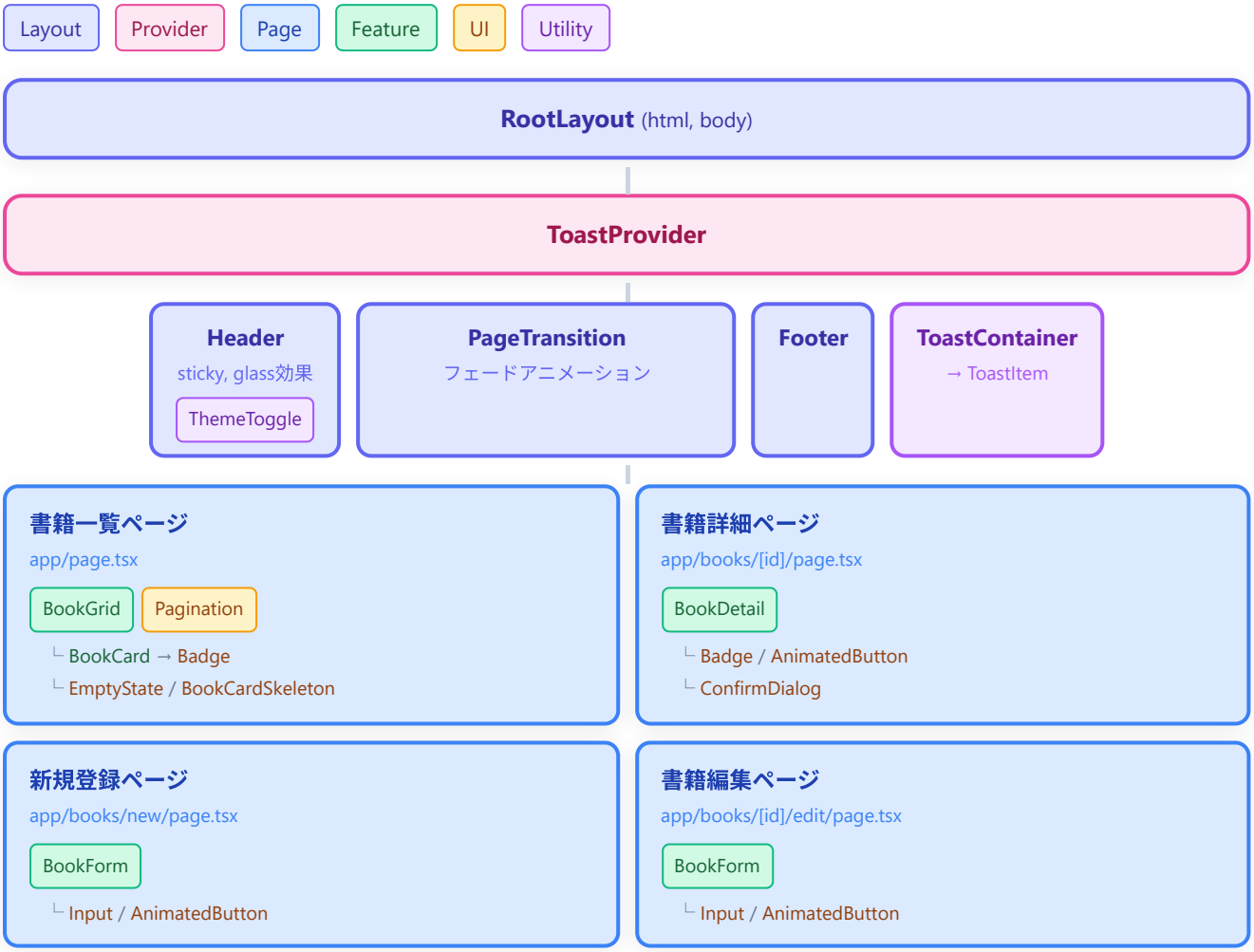
レイアウト:

- 新規登録ページとほぼ同じフォームレイアウトを再利用
- 違いは: タイトルが「書籍情報を編集」、ボタンラベルが「更新する」、パンくずが「← 書籍詳細に戻る」
- フォームの各フィールドには既存データが pre-fill される

- 更新成功時にトースト通知が表示され、詳細ページにリダイレクト

---

8. コンポーネント構成図（最終版）



構成図の読み方:

色	カテゴリ	説明
青	Page Components	Next.js のルートに対応するページコンポーネント
緑	Feature Components	特定の機能を担うコンポーネント（書籍グリッド、フォーム等）
黄	UI Components	再利用可能な汎用 UI 部品
紫	Utility Components	トースト通知やテーマ切替などの横断的機能
インディゴ	Layout Components	ヘッダー、フッター、ページ遷移など画面の骨格
ピンク	Providers	React Context による状態管理

まとめ

この章で実装した内容を振り返ります。

ここまでで、書籍管理アプリは「動くだけのアプリ」から「整ったデザイン・滑らかなアニメーション・ダークモード対応・誰でも使えるアクセシビリティ」を備えた本格的なWebアプリに進化しました。Tailwind CSS のユーティリティファースト思想に慣れると、CSSを書く時間より「何を作りたいか」を考える時間に集中できるようになります。

項目	実装内容	ファイル
グローバルスタイル	`@theme` のカラートークン・影・アニメーション、`.dark` の上書き、ベース／コンポーネントレイヤー、`@utility`	`app/globals.css`
Tailwind 設定	ダークモードの方式（`@custom-variant`）、カスタムカラー、アニメーション、フォント、シャドウ	`app/globals.css`（v3 では `tailwind.config.ts`）
v3 との違い	設定ファイルの引っ越し、名前が変わったクラスの対応表	1.1.1 節 / 1.3 節（参考）
レスポンシブグリッド	1列→2列→3列→4列のブレークポイント対応	`components/BookGrid.tsx`
トースト通知	Context API ベースの成功/エラー通知	`contexts/ToastContext.tsx`, `components/Toast.tsx`
ページネーション	ページ番号計算ロジック付きナビゲーション	`components/Pagination.tsx`
空状態	アイコン + メッセージ + CTA ボタン	`components/EmptyState.tsx`
スケルトン	シマーアニメーション付きローディング	`components/BookCardSkeleton.tsx`
カードアニメーション	ホバーで浮き上がり + 画像ズーム + オーバーレイ	`components/BookCard.tsx`
ページ遷移	フェードイン・スライドアップ	`components/PageTransition.tsx`
ダークモード	class ベース切替 + ちらつき防止スクリプト	`components/ThemeToggle.tsx`
アクセシビリティ	aria属性、キーボードナビ、フォーカストラップ、コントラスト比	各コンポーネント
確認ダイアログ	モーダル + フォーカストラップ + Escape 対応	`components/ConfirmDialog.tsx`

次の章では、テストの書き方とデプロイについて学びます。その前に、章末の「練習問題」で、この章のつまずきやすいポイントを確認しておきましょう。

発展: アプリでは使っていない重要なスタイリング機能

> このアプリでは、グリッドやホバー演出など一部の機能だけを使いました。でも Tailwind CSS には、これ以外にも「画面のレイアウトを自由に組む」ための定番機能がたくさんあります。
>
> ここでは、本編では深く触れなかったものの、実際の開発で**ほぼ毎日使う**5つのテーマを、初心者向けに最小サンプルで紹介します。すべて Tailwind のクラスだけで書けるので、HTMLに `className` を足すだけで試せます。どれも「いつ使うか」「どんな見た目になるか」をセットで説明します。

Flexbox（横並び・縦並びを自在に組む）

> **▼ このコードがやること（先に日本語で）**：複数の箱を「横一列」や「縦一列」にきれいに並べる仕組みを紹介します。並べる方向、はみ出したときの折り返し、上下や左右の揃え方、そして「余ったすき間を1つの箱に吸わせて画面いっぱいに広げる」やり方までを、1つの小さな例でまとめて見ていきます。ボタンを横に並べたり、ロゴと右側のメニューを両端に寄せたりといった、画面の至るところで使う基本テクニックです。

コピー

```
{/*
  ▼ Flexbox（フレックスボックス）の基本セット
     ・flex           : 中の子要素を「横並び」にする
     ・items-center   : 縦方向（高さ方向）の中央に揃える
     ・justify-between : 左右の端に寄せて、間を空ける
     ・flex-wrap      : 横幅が足りなくなったら自動で折り返す
     ・flex-1         : 余ったすき間を吸い込んで広がる
*/}
<div className="flex flex-wrap items-center justify-between gap-3">
  {/* 左側：タイトル。flex-1 で余白を吸って広がる */}
  <h2 className="flex-1 text-lg font-bold">本棚</h2>

  {/* 右側：ボタンを2つ横並びに。間に gap-2 のすき間 */}
  <div className="flex items-center gap-2">
    <button className="rounded-md bg-gray-200 px-3 py-1">並び替え</button>
    <button className="rounded-md bg-blue-500 px-3 py-1 text-white">追加</button>
  </div>
</div>
```

▼ コードを1つずつ分解して解説

解説1: `flex` で「横並び」にする

コピー

```
<div className="flex ...">
```

- `flex` を付けると、その箱の**中の子要素が横一列に並びます**。何も付けないと子要素は縦に積み上がるので、横に並べたいときの第一歩がこれです。
- 「ロゴとメニューを横に」「ボタンを2つ横に」など、**横並びにしたい場面のほぼすべて**で使います。

> **用語: Flexbox（フレックスボックス）** … 箱の中身を「横並び」または「縦並び」に整列させ、すき間や揃え方を細かく指定できるCSSのレイアウト方式。

解説2: `items-*` と `justify-*` で揃え方を決める

コピー

```
<div className="flex items-center justify-between ...">
```

- `items-center` は**縦方向（並びと垂直の向き）の揃え**です。高さの違う要素も中央でそろい、見た目がぴしっとします。他に `items-start`（上揃え）、`items-end`（下揃え）があります。
- `justify-between` は**横方向（並びと同じ向き）の配置**です。`between` は「両端に寄せて、間を均等に空ける」配置で、左にタイトル・右にメニュー、という両端レイアウトの定番です。他に `justify-center`（中央寄せ）、`justify-end`（右寄せ）があります。
- 見た目のイメージ: `[タイトル ・・・・・・・・・・ ボタン ボタン]` のように、左右の端に内容が分かれます。

> **用語: items / justify** … `items-*` は「並びと垂直な向き」の揃え、`justify-*` は「並びと同じ向き」の配置を決めるクラス。横並び（`flex`）なら items=縦、justify=横になる。

この「並びと同じ向き」を**主軸（しゅじく）**、「並びと垂直な向き」を**交差軸（こうさじく）**と呼びます。電車の線路にたとえると、主軸は「電車が進む向き」、交差軸は「線路を横切る向き」です。`justify-*` はいつも主軸、`items-*` はいつも交差軸を担当します。

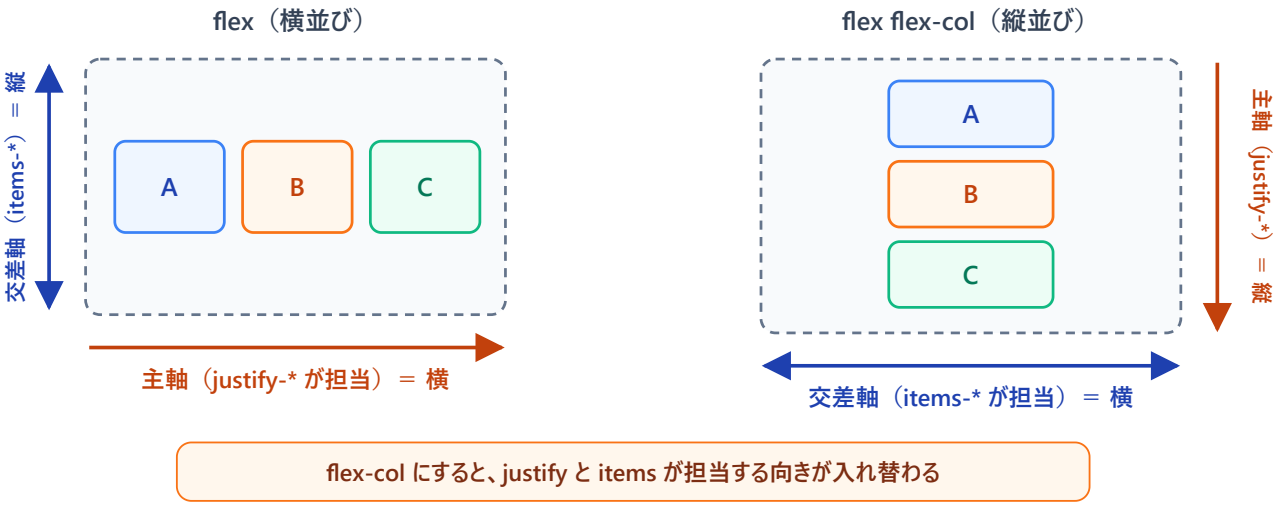


図9-18: Flexbox の主軸と交差軸（横並びと縦並びで、justify-* と items-* の向きが逆になる）

つまずきポイント: `flex-col` にしたら `justify-center` で横の中央にならない

縦並び（ `flex-col` ）にすると主軸が**縦**になるので、 `justify-center` は「縦方向の中央」寄せになり、横方向には効きません。横方向は初期値の「横いっぱいに引き伸ばす（ `stretch` ）」のままなので、ボタンが箱の幅いっぱいに広がってしまいます。ボタンを**横方向の中央**に小さく置きたいなら、交差軸を担当する `items-center` を使います（図9-18の右側）。

コピー

```
{/* NG: 縦並びのボタンを「横の中央」に寄せたくて justify-center を付けた */}
{/* → flex-col の主軸は縦なので横には効かず、ボタンは横いっぱいに伸びたまま */}
<div className="flex flex-col justify-center gap-2">
  <button>保存</button>
  <button>キャンセル</button>
</div>
```

コピー

```
{/* OK: flex-col のときの「横」は交差軸 → items-center を使う */}
{/* → ボタンが中身の幅に縮み、横方向の真ん中にそろう */}
<div className="flex flex-col items-center gap-2">
  <button>保存</button>
  <button>キャンセル</button>
</div>
```

解説3: `flex-wrap` で折り返す

コピー

```
<div className="flex flex-wrap ...">
```

- `flex-wrap` は「横幅が足りなくなったら、子要素を**次の行へ自動で折り返す**」指定です。
- これが無いと、画面が狭いスマホで要素が押しつぶされたり、はみ出したりします。スマホ対応では付けておくと安心です。
- 見た目のイメージ: PCでは1行に収まっていたボタンが、スマホでは2行に折り返して表示されます。

解説4: `flex-1` で「余ったすき間を吸わせる」

コピー

```
<h2 className="flex-1 ...">本棚</h2>
```

- `flex-1` を付けた要素は、**横並びの中で余ったすき間をすべて吸い込んで広がります。**
- ここではタイトルに `flex-1` を付けたので、タイトルが左側いっぱいに伸び、その結果ボタン群が右端へ押しやられます。`justify-between` と似た両端レイアウトを、別のやり方で作れます。
- 「片方だけ広げて、もう片方は中身の幅のまま」にしたいときに便利です。

> **用語: flex-1** … 横並びの中で「余白を引き受けて伸び縮みする」指定。複数の要素に付けると、すき間を等分に分け合う。

CSS Grid（格子状にきっちり並べる）

> **▼ このコードがやること（先に日本語で）**：写真ギャラリーやカード一覧のように、**縦横の格子（マス目）にきっちり並べる**やり方を紹介します。「何列にするか」「マス同士のすき間」を指定するだけで、あとは自動で整列します。さらに、画面の広さに合わせて列数が勝手に増減する「自動調整」も見ていきます。本編のグリッドより一歩踏み込んだ使い方です。

コピー

```
{/*
  ▼ CSS Grid（グリッド）の基本セット
     ・grid          : 格子レイアウトにする
     ・grid-cols-3   : 列を3つに分ける
     ・gap-4         : マス同士のすき間を 16px に
*/}
<div className="grid grid-cols-3 gap-4">
  <div className="rounded bg-blue-100 p-4">1</div>
  <div className="rounded bg-blue-100 p-4">2</div>
  <div className="rounded bg-blue-100 p-4">3</div>
  <div className="rounded bg-blue-100 p-4">4</div>
  <div className="rounded bg-blue-100 p-4">5</div>
  <div className="rounded bg-blue-100 p-4">6</div>
</div>

{/*
  ▼ 列の自動調整（画面幅に応じて列数が変わる）
     ・grid-cols-[repeat(auto-fill,minmax(8rem,1fr))]
       → 「最低 8rem ある列を、入るだけ自動で並べる」指定
*/}
<div className="grid grid-cols-[repeat(auto-fill,minmax(8rem,1fr))] gap-4">
  <div className="rounded bg-green-100 p-4">A</div>
  <div className="rounded bg-green-100 p-4">B</div>
  <div className="rounded bg-green-100 p-4">C</div>
</div>
```

▼ コードを1つずつ分解して解説

解説1: `grid` と `grid-cols-*` で列数を決める

コピー

```
<div className="grid grid-cols-3 ...">
```

- `grid` を付けると、その箱は**格子（マス目）レイアウト**になります。`grid-cols-3` は「列を3つに分ける」指定です。
- すると中の子要素は、左上から順に **1行に3つずつ** 並び、4つ目から自動で次の行へ折り返します。
- カード一覧・画像ギャラリー・ダッシュボードなど、「同じ大きさの箱をきれいに整列させたい」ときに最適です。

> **用語: CSS Grid（グリッド）** … 画面を縦横の格子に区切って要素を配置するレイアウト方式。Flexbox が「1方向の並び」が得意なのに対し、Grid は「縦横2方向の整列」が得意。

解説2: `gap-*` でマスのすき間を空ける

コピー

```
<div className="grid grid-cols-3 gap-4 ...">
```

- `gap-4` は、マス目同士の**縦と横のすき間**をまとめて 16px に空ける指定です。
- `margin`（外側の余白）を1つずつ足すより簡単で、端っこに余分なすき間ができないのが利点です。
- 数字を変えると間隔が変わります（`gap-2` =8px、`gap-6` =24px など）。

解説3: 列の自動調整（`auto-fill` / `minmax`）

コピー

```
<div className="grid grid-cols-[repeat(auto-fill,minmax(8rem,1fr))] ...">
```

- `grid-cols-[...]` の角かっこは、**Tailwind に用意されていない細かい値を自分で書く**ための書き方（任意の値）です。
- 中身の `repeat(auto-fill, minmax(8rem, 1fr))` は「**最低 8rem（約128px）の幅を持つ列を、入るだけ自動で並べる**」という意味です。`minmax(8rem, 1fr)` は「最小8rem・最大は余白を分け合った幅」を表します。
- これを使うと、`sm:` `lg:` のような接頭辞をいくつも書かなくても、**画面が広ければ列が増え、狭ければ自動で減ります**。見た目のイメージ: PCでは1行に5〜6個、スマホでは1〜2個に自然に変化します。

> **用語: 任意の値（arbitrary value）** … `grid-cols-[...]` のように角かっこの中に好きな値を直接書く Tailwind の機能。用意されたクラスでは足りないときの「逃げ道」。

2つのグリッドの並び方を図にすると次のとおりです。左は「列の数を決める」やり方、右は「1列の最小の幅を決めて、列の数はおまかせ」にするやり方です。

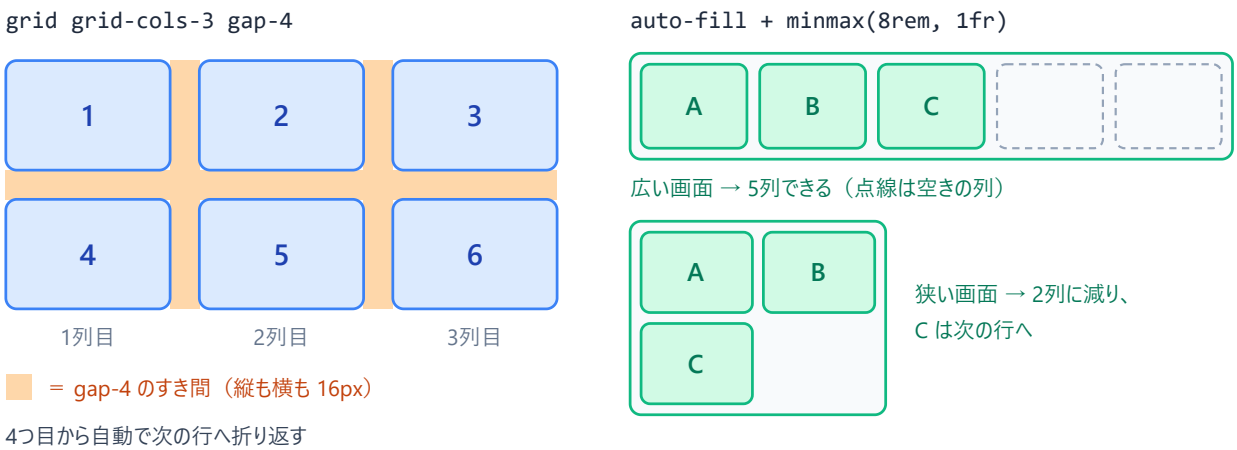


図9-19: Grid の並び方（左: 列の数を指定 / 右: 列の最小幅を指定して、列の数は画面幅まかせ）

擬似クラス・状態（hover・focus・before/after）

> **▼ このコードがやること（先に日本語で）**：マウスを乗せたとき・クリックして入力中のときなど、「**ある状態のときだけ**見た目を変える」やり方を紹介します。さらに、HTMLには書いていない「飾りの文字や印」を、CSSの力だけで前後にくっつける小技も見ていきます。ボタンの反応や、入力欄を選んだときの強調など、操作している感じを伝えるのに欠かせません。

なぞってコピーすると「…」が別の文字になります。正しくは → コピー

```
{/*
  ▼ 状態に応じた見た目の変化
    ・hover:bg-blue-600 ： マウスを乗せた時だけ濃い青に
    ・focus:ring-2      ： クリック/選択した時だけ枠を光らせる
    ・before:/after:    ： HTMLに無い「飾り」を前後に足す
*/}
<button className="rounded bg-blue-500 px-4 py-2 text-white hover:bg-blue-600 focus:ring-2 focus:ring-blue-300">
  保存
</button>

<input
  className="rounded border px-3 py-2 focus:border-blue-500 focus:outline-hidden"
  placeholder="検索…"
/>

{/* 必須マーク（赤い * ）を after で自動的に付ける */}
<label className="after:ml-1 after:text-red-500 after:content-['*']">
  タイトル
</label>
```

▼ コードを1つずつ分解して解説

解説1: `hover:` でマウスを乗せたときの見た目

コピー

```
<button className="bg-blue-500 hover:bg-blue-600 ...">
```

- `hover:` を付けたクラスは、**マウスのカーソルがその要素の上にあるときだけ**効きます。
- ここでは普段は `bg-blue-500`（青）、乗せたときだけ `hover:bg-blue-600`（少し濃い青）になります。「押せそう」という反応をユーザーに伝えられます。
- 見た目のイメージ: マウスを近づけるとボタンの色がすっと濃くなります。

> **用語: 擬似クラス（pseudo-class）** … `hover`（乗せた時） `focus`（選択中） `active`（押している時）など、「要素がある状態のときだけ」スタイルを当てるための目印。Tailwind では `hover:` のように接頭辞で書く。

解説2: `focus:` で「選択中」を強調する

コピー

```
<input className="... focus:border-blue-500 focus:outline-hidden" />
<button className="... focus:ring-2 focus:ring-blue-300">
```

- `focus:` は、**その要素が選ばれている（入力欄をクリックした、キーボードで移動してきた）ときだけ**効きます。
- `focus:border-blue-500` は選択中の入力欄の枠を青くし、「今ここに入力できますよ」と知らせます。
  `focus:ring-2` はボタンのまわりに光る輪を出します。
- これは見た目だけでなく、**キーボードだけで操作する人がどこを触っているか分かる**ようにする大切な配慮です（アクセシビリティ）。

解説3: `before:` / `after:` で飾りを足す（擬似要素）

コピー

```
<label className="after:ml-1 after:text-red-500 after:content-['*']">
  タイトル
</label>
```

- `after:` は、その要素の**中身の「後ろ」に、HTMLには書いていない飾りを足す**指定です（前に足すなら `before:`）。
- `after:content-['*']` で「`*`（アスタリスク）」という文字を後ろに付け、`after:text-red-500` で赤く、`after:ml-1` で少し左に余白を空けています。結果として「タイトル *」*のように必須マーク*が表示されます。
- HTMLを汚さずに、ラベルの装飾や区切り記号などを足せるのが便利です。

> **用語: 擬似要素（pseudo-element）** … `before` / `after` のように、HTMLには存在しない「飾り用の中身」をCSSで前後に作り出す仕組み。アイコンや記号の追加によく使う。

ポジショニング（重ね・固定・追従）

> **▼ このコードがやること（先に日本語で）**：要素を「**普通の流れから外して好きな場所に置く**」やり方を紹介します。たとえば、画像の右上にバッジを重ねたり、スクロールしても見出しを画面上部に貼り付けたままにしたりできます。重なり順の調整方法もあわせて見ていきます。通知の数字バッジや、追従するヘッダーなどでよく使います。

コピー

```
{/*
  ▼ ポジショニングの基本セット
     ・relative : 子要素の「基準」になる箱（自分は動かない）
     ・absolute : 基準の箱を起点に、好きな位置へ浮かせて置く
     ・inset-*  : 上下左右からの距離をまとめて指定
     ・z-*      : どれを手前に表示するか（重なり順）
*/}
<div className="relative inline-block">
  {/* 商品画像（基準になる箱） */}
  <img src="/book.jpg" alt="表紙" className="rounded" />

  {/* 右上に重ねる「NEW」バッジ */}
  <span className="absolute -top-2 -right-2 z-10 rounded-full bg-red-500 px-2 py-0.5 text-xs text-white">
    NEW
  </span>
</div>

{/*
  ▼ sticky : スクロールしても画面上部に貼り付く見出し
     ・top-0 : 上端 0px の位置で固定される
*/}
<h3 className="sticky top-0 bg-white py-2 font-bold">あ行</h3>
```

▼ コードを1つずつ分解して解説

解説1: `relative` は「基準」をつくる

コピー

```
<div className="relative inline-block">
```

- `relative` を付けた箱は、**それ自体はその場から動きません。**役割は「中にある `absolute` の要素の**位置の基準**になる」ことです。
- つまり「この箱を基準にして、中のバッジを置きたい」というときの土台です。`relative` を付け忘れると、バッジが画面全体を基準にずれてしまうので、セットで覚えます。

> **用語: relative（相対配置）** … 自分は動かず、中の `absolute` 要素にとっての「位置の起点」になる指定。「ここを基準にして子を浮かせる」宣言。

解説2: `absolute` と `inset-*` で浮かせて置く

コピー

```
<span className="absolute -top-2 -right-2 ...">NEW</span>
```

- `absolute` は、その要素を**普通の流れから外して、好きな位置に浮かせて置く**指定です。場所は一番近い `relative` の箱が基準になります。
- `-top-2 -right-2` は「上から少し外側へ・右から少し外側へ」ずらす指定で（マイナスなので箱の外側へはみ出します）、画像の右上角にバッジが乗ります。
- 位置の指定には `top-* / right-* / bottom-* / left-*` のほか、4辺をまとめて指定する `inset-0`（上下左右すべて0＝親いっぱいに広げる）も便利です。
- 見た目のイメージ: 画像の右上角に赤い丸の「NEW」が乗っかります。

> **用語: absolute（絶対配置）** … 通常の並びから外して、基準の箱の中で座標を指定して置く方式。バッジ・吹き出し・オーバーレイなど「重ねたい」ものに使う。`inset-*` は上下左右の距離をまとめて書くクラス。

`relative` と `absolute` はセットで使うものなので、片方を忘れると次のような失敗が起きます。

> **つまずきポイント: `absolute` のバッジが、とんでもない場所に飛んでいく**
>
> `absolute` の要素は「外側に向かって探して、**最初に見つかった `relative`（など位置の指定がある）箱**」を基準にします。画像を包む `div` に `relative` を付け忘れると、基準がさらに外側（最終的にはページ全体）になり、バッジが画像から離れた画面の角に表示されてしまいます。

コピー

```
{/* NG: 画像を包む div に relative が無い */}
{/* → バッジの基準がページ全体になり、画面の右上に飛んでいく */}
<div className="inline-block">
  <img src="/book.jpg" alt="表紙" className="rounded" />
  <span className="absolute -top-2 -right-2 rounded-full bg-red-500 px-2 text-xs text-white">
    NEW
  </span>
</div>
```

コピー

```
{/* OK: 画像を包む div に relative を付けて「ここが基準」と宣言する */}
{/* → バッジが画像の右上の角にぴったり乗る */}
<div className="relative inline-block">
  <img src="/book.jpg" alt="表紙" className="rounded" />
  <span className="absolute -top-2 -right-2 rounded-full bg-red-500 px-2 text-xs text-white">
    NEW
  </span>
</div>
```

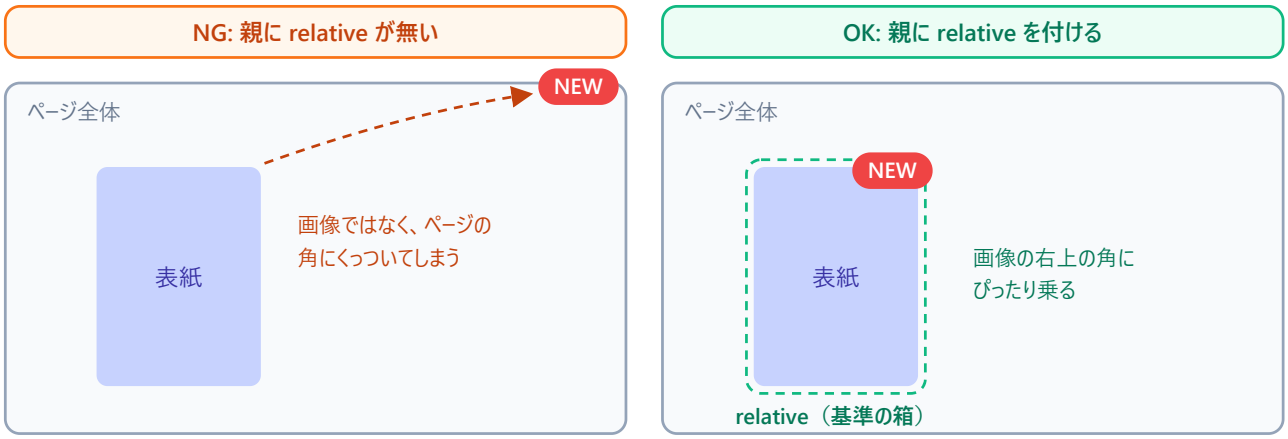


図9-20: absolute の基準（左: relative が無いと遠くの箱が基準になる / 右: relative の箱が基準になる）

解説3: `z-*` で重なり順を決める

コピー

```
<span className="absolute ... z-10 ...">NEW</span>
```

- `z-10` は**重なったときにどれを手前にするか**（重なり順）を決める指定です。数字が大きいほど手前に来ます。
- バッジが画像の下に隠れてしまうようなときは、バッジに大きい `z-` を付けると前面に出ます。
- メニューやモーダル（前面に出る小窓）など、「他の上に必ず出したい」要素で重要になります。

> **用語: z-index（ゼットインデックス）** … 要素の前後（奥行き）の重なり順を表す数値。大きいほど手前。Tailwind では `z-10` `z-50` のように書く。

解説4: `sticky` でスクロールに追従させる

コピー

```
<h3 className="sticky top-0 ...">あ行</h3>
```

- `sticky` は、**普段は普通の位置にいて、スクロールして指定位置に来たらそこで貼り付く**という、`relative` と固定の「いいとこ取り」の指定です。
- `top-0` と組み合わせると、見出しが画面の一番上まで来たときにそこで止まり、続きの内容だけがその下を流れていきます。
- 五十音やカテゴリごとの見出し、表のヘッダーを「常に見える」ようにするのに最適です。

> **用語: sticky（粘着配置）** … 普段は流れに従い、スクロールで一定位置に達すると画面に貼り付く配置。長いリストの見出しを残すのに便利。

---

transform（拡大・回転・移動とアニメーション）

> **▼ このコードがやること（先に日本語で）**：要素を「**大きくする・回す・動かす**」見た目の変化を紹介します。さらに、その変化を一瞬で切り替えるのではなく**なめらかにつなぐ**設定を組み合わせて、触ったときに気持ちよく反応するボタンを作ります。レイアウトを崩さずに見た目だけ動かせるので、ホバー演出の定番です。

長い行を折り返しているので、なぞってコピーすると行が分かれます。正しくは → コピー

```
{/*
  ▼ transform（変形） ＋ transition（なめらかな変化）
    ・transition    ： 変化を一瞬ではなく滑らかにつなぐ
    ・duration-300  ： 変化にかける時間（300ミリ秒）
    ・hover:scale-105 ： 乗せた時に 1.05 倍に拡大
    ・hover:-rotate-2 ： 乗せた時に少し左へ傾ける
    ・hover:-translate-y-1 ： 乗せた時に少し上へ浮かせる
*/}
<button className="rounded-lg bg-purple-500 px-5 py-2 text-white transition duration-300 hover:scale-105
hover:-rotate-2 hover:-translate-y-1">
  お気に入り
</button>
```

▼ コードを1つずつ分解して解説

解説1: `scale-*` で拡大・縮小する

コピー

```
<button className="... hover:scale-105 ...">
```

- `scale-105` は要素を**1.05倍（5%大きく）**にする指定です。`hover:` と組み合わせているので、マウスを乗せたときだけ少し大きくなります。
- ポイントは、`scale` は**まわりのレイアウトを崩さずに見た目だけ拡大する**ことです（実際の場所取りは変わりません）。だからホバーで気軽に使えます。
- 見た目のイメージ: マウスを乗せるとボタンがぷくっと少し膨らみます。

> **用語: transform（変形）** … 拡大（scale）・回転（rotate）・移動（translate）など、要素の「見た目」を変えるCSSの機能。場所取りは変えずに表示だけ変形する。

解説2: `rotate-*` で回転、`translate-*` で移動

コピー

```
<button className="... hover:-rotate-2 hover:-translate-y-1 ...">
```

- `-rotate-2` は要素を**少しだけ左に傾ける**指定です（マイナスで反時計回り）。数字を大きくすると傾きが強くなります。
- `-translate-y-1` は**少し上へ動かす**指定です（`y` は縦方向、マイナスで上）。「浮き上がる」感じを出せます。
- これらは `scale` と同じく場所取りを変えないので、複数を組み合わせても周りがガタつきません。

解説3: `transition` と `duration-*` でなめらかにつなぐ

コピー

```
<button className="... transition duration-300 ...">
```

- `transition` を付けないと、`hover:` の変化が**パッと一瞬で切り替わって**しまい、ぎこちなく見えます。
- `transition` を付けると変化が**なめらかにつながり**、`duration-300` でその所要時間を300ミリ秒（約0.3秒）に指定します。数字が大きいほどゆっくり変化します。
- この「変形（transform）＋なめらか（transition）」の組み合わせが、**気持ちよく反応するボタン**の黄金パターンです。

> **用語: transition（トランジション）** … プロパティの変化を、指定した時間をかけて滑らかにつなぐ機能。`transform` と組み合わせると自然な動きの演出になる。

練習問題（解答・解説つき）

この章の内容を確かめる問題です。まず自分で考えてから、下の解答・解説を読みましょう。手を動かせる問題は、Tailwind Play（`https://play.tailwindcss.com`）や 0.3 節の練習用フォルダで実際に試すと理解が深まります。

問題1: 窮屈なボタンを広げる（ボックスモデル）

次のボタンは、文字のまわりに余白が無くて窮屈です。**青い部分の内側**を「上下 8px・左右 16px」広げるには、どのクラスを足せばよいでしょうか。

コピー

```
<button class="bg-blue-600 text-white">保存</button>
```

解答: `py-2 px-4` を足します。

コピー

```
<!-- py-2 = 上下 8px、px-4 = 左右 16px の「内側の余白」 -->
<button class="bg-blue-600 px-4 py-2 text-white">保存</button>
```

解説:

- 「青い部分の**内側**」を広げたいので、使うのは padding（`p-`）です。`m-`（margin）を足しても青い部分は大きくならず、外側に透明なすき間ができるだけでした（0.1 節のつまずきポイント）。
- `py-` は上下（y方向）、`px-` は左右（x方向）です。数字は「×4px」なので、8px は `2`、16px は `4` になります（図9-6）。

問題2: 画面幅ごとの列数（レスポンシブ）

`class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3"` の箱は、画面の幅が **(a) 500px (b) 800px (c) 1100px** のとき、それぞれ何列になるでしょうか。

解答: (a) 1列 (b) 2列 (c) 3列

解説:

- `md:` は 768px 以上、`lg:` は 1024px 以上で効きます。500px ではどちらも効かないので、接頭辞なしの `grid-cols-1` だけが残ります。
- 800px は 768px 以上・1024px 未満なので `md:grid-cols-2` まで。1100px では `md:` と `lg:` の両方が条件を満たしますが、**より大きい画面向けの `lg:` が勝ちます**（図9-12）。
- 2.1 節の `breakpoints.ts` の `good` をこのクラスに、幅の配列を `[500, 800, 1100]` に書き換えて実行すると、OK 列に `grid-cols-1` / `grid-cols-2` / `grid-cols-3` と表示されて確かめられます。

問題3: スマホだけ表示したいボタン（`sm:` の意味）

「スマホではハンバーガーボタンを表示し、**768px 以上では隠したい**」とき、ボタンに付けるクラスとして正しいものはどれでしょうか。

- (A) `sm:block`
- (B) `md:hidden`
- (C) `sm:hidden md:block`

解答: (B) `md:hidden`

解説:

- 接頭辞の無い状態ではボタンは表示されています。`md:hidden` で「768px 以上になったら隠す」と上書きするので、スマホでだけ見えるボタンになります。6.1 節のヘッダーと同じ書き方です。
- (A) は「640px 以上で表示」で、もともと表示されているので何も変わりません。

- (C) は「640〜767px で隠し、768px 以上でまた表示」になり、狙いと正反対です。`sm:` を「スマホ用」と読んでしまうと、こうした間違いが起きます（図9-13）。

問題4: 色が付かないボタン（動的クラス名）

次のボタンは、`variant` に何を渡しても背景色が付きません。**理由**と**直し方**を答えてください。

コピー

```
type Props = { variant: "primary" | "danger" };

export function Button({ variant }: Props) {
  const color = variant === "primary" ? "indigo" : "red";
  return <button className={`bg-${color}-600 text-white`}>OK</button>;
}
```

解答: Tailwind はファイルを文字として読むだけなので、`bg-indigo-600` や `bg-red-600` という**完成したクラス名**がファイルの中に無く、その CSS が作られないためです。完成したクラス名を辞書に書いておく形に直します。

コピー

```
type Props = { variant: "primary" | "danger" };

// 完成したクラス名を辞書に書いておく
const variantClasses = {
  primary: "bg-indigo-600",
  danger: "bg-red-600",
};

export function Button({ variant }: Props) {
  return <button className={`${variantClasses[variant]} text-white`}>OK</button>;
}
```

▼ **実行結果**（直す前と後のファイルを CLI に通した、`@layer utilities` の部分）

コピー

```
---- 直す前: text-white しか作られていない ----
@layer utilities {
  .text-white {
    color: var(--color-white);
  }
}

---- 直した後: 2つの背景色が作られた ----
@layer utilities {
  .bg-indigo-600 {
    background-color: var(--color-indigo-600);
  }
  .bg-red-600 {
    background-color: var(--color-red-600);
  }
  .text-white {
    color: var(--color-white);
  }
}
```

解説:

- 直した後も `` `${variantClasses[variant]} text-white` `` のように `${}` を使っていますが、問題ありません。大事なのは**クラス名そのもの（ `bg-indigo-600` ）が、どこかに完成した形で書いてあること**です。
- 0.4 節の「試してみよう」と同じ仕組みです。

問題5: v4 の globals.css を書く（Tailwind v4 の設定）

Tailwind v4 のプロジェクトで、次の2つを実現したいです。`app/globals.css` に何を書けばよいでしょうか。

1. ThemeToggle で `<html class="dark">` を付けたときに、`dark:` のクラスが効くようにする
2. `#0ea5e9` という色を `brand-500` という名前で登録し、`bg-brand-500` を使えるようにする

解答:

コピー

```
@import "tailwindcss";

/* (1) dark: を dark クラスで切り替える */
@custom-variant dark (&:where(.dark, .dark *));

/* (2) 自分の色を登録する */
@theme {
  --color-brand-500: #0ea5e9;
}
```

▼ **実行結果**（`<div class="bg-brand-500 dark:bg-slate-900"></div>` を CLI に通した、`@layer utilities` の部分）

コピー

```
@layer utilities {
  .bg-brand-500 {
    background-color: var(--color-brand-500);
  }
  .dark\:bg-slate-900:where(.dark, .dark *) {
    background-color: var(--color-slate-900);
  }
}
```

解説:

- (1) v4 の `dark:` は、何もしないと OS の設定に連動します。`@custom-variant dark (…)` の1行で「`dark` クラスの中にいるとき」に変えます（5.1 節のつまずきポイント）。
- (2) `@theme` の中に `--color-` で始まる変数を書くと、その名前の色クラス（`bg-brand-500`・`text-brand-500` など）が作られます。v3 のように `tailwind.config.ts` に書いても、v4 では `@config` を書かない限り読まれません（1.1.1 節）。

問題6: 影がパッと消えるカード（transition）

次のカードは、マウスを乗せると影がなめらかに大きくなるのに、**離すと影がパッと元に戻って**しまいます。どう直せばよいでしょうか。

コピー

```
<article class="rounded-xl bg-white shadow-sm hover:shadow-lg hover:transition-shadow hover:duration-300">
  リーダブルコード
</article>
```

解答: `transition-shadow` と `duration-300` を `hover:` の外に出します。

コピー

```
<!-- transition は「いつでも」付けておき、変えたい影だけを hover: にする -->
<article class="rounded-xl bg-white shadow-sm transition-shadow duration-300 hover:shadow-lg">
  リーダブルコード
</article>
```

解説:

- `transition` は「変化をなめらかにする」設定なので、**変化が起きる前と後の両方で**要素に付いている必要があります。

- `hover:transition-shadow` だと、マウスを離した瞬間に transition も外れるため、戻る動きだけが一瞬になります（4.1 節のつまずきポイント）。

問題7: この灰色は本文に使える？（コントラスト）

白い背景（`#ffffff`）に、`#94a3b8` の文字で本文を書こうとしています。これは WCAG 2.1 AA の基準を満たすでしょうか。満たさない場合、どうすればよいでしょうか。

解答: 満たしません（約 2.6:1 で、本文の基準 4.5:1 に届かない）。`#64748b`（4.8:1）や `#475569`（7.6:1）のように、もっと濃い灰色を使います。

解説:

- 6.3 節の `contrast.ts` の `pairs` に `["問題7", "#94a3b8", "#ffffff"]` を足して実行すると、`2.56:1 不合格` と表示されます。
- 面白いのは、**同じ `#94a3b8` でも、ダークモードの背景 `#0f172a` の上では 6.96:1 で合格**になることです。本アプリのダークモードで補足文に `#94a3b8` を使っているのはこのためで、「色の合否は背景との組み合わせで決まる」ことが分かります。

問題8: 縦並びのまま横の中央へ（Flexbox）

ボタンを縦に並べたまま、**横方向の真ん中**にそろえたいです。`???` に入るクラスはどれでしょうか。

コピー

```
<div className="flex flex-col ??? gap-2">
  <button>保存</button>
  <button>キャンセル</button>
</div>
```

- (A) `justify-center`
- (B) `items-center`
- (C) `text-center`

解答: (B) `items-center`

解説:

- `flex-col` のときの主軸は**縦**、交差軸は**横**です。横方向の揃えは交差軸の担当なので `items-center` を使います（図9-18）。
- (A) `justify-center` は主軸（縦）方向の中央寄せです。横方向は初期値の `stretch`（横いっぱいに伸ばす）のままなので、ボタンが箱の幅いっぱいに広がります。
- (C) `text-center` は「箱の中の**文字**を中央に寄せる」クラスで、ボタンという箱そのものの幅や位置は変わりません（横いっぱいに伸びたまま）。